

Multi-Client Attribute-Based and Predicate Encryption, Revisited

Robert Schädlich 

DIENS, École normale supérieure, PSL University, CNRS, Inria
robert.schaedlich@ens.fr

Abstract

Multi-client Attribute-Based Encryption (ABE) is a generalization of key-policy ABE where attributes can be independently encrypted across several ciphertexts w.r.t. labels, and a joint decryption of these ciphertexts is possible if and only if (1) all ciphertexts share the same label, and (2) the combination of attributes satisfies the policy of the decryption key. All encryptors have their own secret key and security is preserved even if some of them are known to the adversary.

Very recently, Pointcheval et al. (TCC 2024) presented a semi-generic construction of MC-ABE for restricted function classes, e.g., NC^0 and constant-threshold policies. We identify an abstract criterion common to all their policy classes which suffices to present the construction in a fully black-box way and allows for a slight strengthening of the supported policy classes. The construction of Pointcheval et al. is based on pairings. We additionally provide a new lattice-based instantiation from (public-coin) evasive LWE.

Furthermore, we revisit existing constructions for policies that can be viewed as a conjunction of local policies (one per encryptor). Existing constructions from MDDH (Agrawal et al., CRYPTO 2023) and LWE (Francati et al., EUROCRYPT 2023) do not support encryption w.r.t. different labels. We show how this feature can be included at the cost of additionally relying on random oracles. Notably, the security model of Francati et al. additionally guarantees attribute-hiding but does not capture collusions. Our new construction is also attribute-hiding and provides resilience against any polynomially bounded number of collusions which must be fixed at the time of setup.

Contents

1	Introduction	3
1.1	Related Work	3
1.2	Our Results	4
1.3	Relation to Witness Encryption	5
2	Technical Overview	5
2.1	MC-ABE for Succinctly Enumerable Function Classes	7
2.2	MC-ABE for Conjunction Policies	10
3	Preliminaries	14
3.1	Notational Conventions	14
3.2	Computational Models	15
3.3	Pairing Groups	15
3.4	Lattices	16
3.5	Exact Linear Garbling	16
3.6	Noisy Linear Garbling	17
3.7	Multi-Client Attribute-Based Encryption	19
3.8	Attribute-Based Inner-Product Functional Encryption	20
3.9	Slotted Identity-Based Inner-Product Functional Encryption	23
3.10	Attribute-Based Encryption	25
3.11	Other Preliminaries	26
4	MC-ABE for Succinctly Enumerable Function Classes	26
4.1	Construction from Slotted IPFE	28
4.2	Proof of Security	30
4.3	Construction from Evasive IPFE	32
4.4	Proof of Security	34
5	MC-ABE for Conjunction Predicates	37
5.1	Construction	38
5.2	Proof of Security – Function-Hiding Case	39
5.3	Proof of Security – Non-Function-Hiding Case	41
A	AB-MIFE for Inner Products Against Bounded Collusions	49
A.1	Definitions for MIFE with Corruptions	49
A.2	Construction of AB-MIFE Against Legitimate Keys	50
A.3	Upgrade to Any Keys	52

1 Introduction

Attribute-Based Encryption. Attribute-based encryption (ABE) [SW05, GPSW06] is an elevated version of traditional public-key encryption that enables fine-grained access control on encrypted data. A ciphertext CT_x encrypting a message μ is generated with respect to a public attribute x , while a secret decryption key DK_f is generated with respect to a policy f . The decryption key DK_f is authorized to decrypt the ciphertext CT_x if and only if the attribute x satisfies the policy f , i.e., $f(x) = 0$. Security requires resilience to collusion attacks. That is, a set of decryption keys $\{DK_{f_i}\}_i$ does not disclose any information about μ unless one of the keys DK_{f_i} is individually authorized to decrypt. A strengthening of this notion, traditionally referred to as Predicate Encryption (PE) [BW07, SBC⁺07], requires ciphertexts to not only hide messages but also their associated attributes.

Decentralized Encryption. Until recently, ABE and PE were solely studied in the centralized setting, i.e., the considered policies f have arity one. The notions of Multi-Input ABE (MI-ABE) [BJK⁺18, AYY22] and Multi-Input PE (MI-PE) [FFMV23] overcome this limitation by considering n encryptors who each encrypt their inputs $(x_0, \mu_0), \dots, (x_{n-1}, \mu_{n-1})$ independently using uncorrelated random coins. The key generator provides decryption keys for arity n functions f , and a joint decryption recovers $(\mu_0, \dots, \mu_{n-1})$ if and only if $f(x_0, \dots, x_{n-1}) = 0$. As in the single-input case, the security model of MI-ABE only guarantees to hide the message encoded in a ciphertext while MI-PE additionally hides the associated attribute.

A very recent work [PS24] initiates the formal study of Multi-Client ABE (MC-ABE) and Multi-Client PE (MC-PE) by adding the usual features of Multi-Client Functional Encryption (MCFE) compared to Multi-Input Functional Encryption (MIFE) [GGG⁺14]. First, MC-ABE separates the secret keys of the slots and guarantees security even if some of them are known to the adversary. Second, encryption proceeds with respect to a label and a joint decryption of ciphertexts is possible only if all of them share the same label.²

1.1 Related Work

The notion of MI-ABE had been studied first by Brakerski et al. [BJK⁺18] as a new pathway for achieving witness encryption. However, they did not consider strong security notions nor did they provide any constructions. In [AYY22], Agrawal et al. provided the first constructions for 2-input ABE for NC¹ from LWE and a nonstandard assumption on pairings. They also gave heuristic constructions for 2-input ABE for P and 3-input ABE for NC¹. Additionally, they gave a compiler that lifts a constant-input ABE scheme to a PE scheme for the same arity and policy class, using a sophisticated nesting technique of lockable obfuscation which can be based on LWE. In an independent work, Francati et al. [FFMV23] built MI-PE for *conjunctions* of (bounded) polynomial-depth circuits from LWE.³ Notably, they can support a polynomial number of inputs. Furthermore, when restricting to constant arity, they provide a construction that remains secure under user corruptions. On the negative side, neither of their MI-PE constructions can be proven secure under collusions. Agrawal et al. [ARYY23] presented a constant-arity MI-ABE for NC¹ whose security is based on (private-coin) evasive LWE. When additionally assuming tensor LWE, they can upgrade their scheme to support arbitrary policies in P. A follow-up work [AKY24] removes the dependency on tensor LWE and enables any polynomial arity. Very recently, Pointcheval and Schadlich [PS24] provided pairing-based constructions for several restricted function classes including NC⁰ and constant-threshold policies. They can support polynomial arity and prove security in the stronger MC-ABE model. As a side result, they also show how to capture all of NC¹ when restricting to one-time labels. Furthermore, they generalize the PE compiler of [AYY22] to the multi-client setting. In a concurrent work [CLW25], Cini et al. present a MC-ABE construction for P from (public-coin) evasive LWE and prove security in the one-time label setting against a restricted form of corruptions.

²We use the term “multi-client” as a generalization of “multi-input” by allowing multiple uses of labels. In contrast, a weaker notion of multi-client has been considered in the literature [CDG⁺18] where each label can be used only once, thus it does not imply the corresponding multi-input notion. We call this weaker form *one-time* labels.

³A policy f is said to be a *conjunction* of a policy class $\mathcal{F}$ if there exist policies $f_0, \dots, f_{n-1} \in \mathcal{F}$ such that $f(x_0, \dots, x_{n-1}) = f_0(x_0) \wedge \dots \wedge f_{n-1}(x_{n-1})$.

MI-ABE and MI-PE can also be viewed as a special case of the more general MIFE primitive. MIFE has been the subject of extensive studies resulting in a large body of works with various trade-offs between expressiveness, security, underlying assumptions and efficiency, e.g., [GGG⁺14, AJ15, AGRW17, DOT18, ACF⁺18, CDG⁺18, Tom19, ABKW19, ABG19, LT19, AGT21a, AGT22]. As MIFE for NC^1 is known to imply indistinguishability Obfuscation (iO) [BGI⁺01, GGH⁺13], it remains an important area of research to build MIFE schemes for simpler function classes from assumptions not known to imply iO. Some of these function classes still encompass MI-ABE. Specifically, Nguyen et al. [NPP22] build the first attribute-based MCFE with one-time labels for inner products, where policies are conjunctions of Linear Secret Sharings (LSS), implying MC-ABE for the same function class. Their construction uses pairings and security is proven under the SXDH assumption. In a very recent follow-up [NPP24], the authors allow reusing the same label for several private inputs (i.e. the inner-product vectors) and prove security in a strong model: adversaries can submit adaptive oracle queries and the admissibility condition excludes fewer attacks than previous works. In [ATY23], Agrawal et al. presented the first attribute-based MIFE for Attribute-Weighted Sums (AWS). The supported policies are conjunctions of Arithmetic Branching Programs (ABPs) for a polynomial number of slots. Their construction uses pairings and is proven secure under the matrix DDH assumption (MDDH).

1.2 Our Results

We provide new MC-ABE constructions for previously unknown combinations of policy classes and computational assumptions. Please see Table 1 for a comparison with known results. For succinctness, the table does not include MC-ABE constructions that support only one-time labels.

MC-ABE Beyond Conjunctions. Very recently, [PS24] presented a semi-generic MC-ABE scheme for several restricted function classes that, nevertheless, cannot be viewed as conjunctions. We revisit their construction and enhance it in the following directions.

- **Abstract Function Class.** We first identify an abstract criterion that we dub *succinct enumerability*. It is common to all the function classes considered in [PS24] and implicitly required by their construction to work properly. While the formal definition of succinct enumerability seems quite artificial, it enables a fully black-box presentation of the construction. It also helps us to identify slightly stronger function classes that were not considered in [PS24]. Notably, the following classes are succinctly enumerable:
 - **Constant Bit Dependency.** Succinct enumerability captures all functions with constant bit dependency on slots $1, \dots, n-1$, but arbitrary dependency on the 0-th slot. This includes NC^0 (considered in [PS24]) as a special case since NC^0 policies have a constant bit dependency on *all* slots. In addition, succinct enumerability captures some special NC^1 predicates where most of the relevant bits (in particular, more than constant) are part of the 0-th input and the bit dependency on the other $n-1$ inputs is still constant.
 - **Polynomial-Size Accepting Sets.** Succinct enumerability also captures policies with a polynomial-size accepting set on slots $1, \dots, n-1$ for each slot-0 input. As a special case, this includes constant-threshold policies (considered in [PS24]) as these policies have *globally* poly-size accepting sets.
- **Lattice-Based Construction.** The construction of [PS24] uses slotted Inner-Product Functional Encryption (IPFE) as a building block which is only known to exist assuming MDDH on pairings [LL20]. To obtain a lattice-based scheme, we instantiate the construction from another IPFE primitive called evasive IPFE [HLL24]. In this way, we obtain MC-ABE for succinctly enumerable function classes from (public-coin) evasive LWE. We note that the previous work [ARYY23] presented MI-ABE from (private-coin) evasive LWE for bounded-depth circuits. While their function class is certainly much stronger than ours, their construction considers the weaker MI-ABE model and can only deal with constant arity.

MC-ABE for Conjunctions. We present the first constructions for conjunction policies that provide all features of MC-ABE.

- **MC-ABE for Conjunctions of ABPs from MDDH.** We first revisit the AB-MIFE scheme for AWS by [ATY23]. When ignoring the AWS functionality, this gives an MI-ABE scheme supporting conjunctions of ABPs. Notably, the construction is already secure against corruptions. We modify the scheme to additionally enable encryption with respect to labels, thus obtaining a full-fledged MC-ABE scheme. Security is proven under the MDDH assumption on pairings (as in [ATY23]) and additionally relies on the ROM. On a technical level, our construction is a compiler that turns any (single-input) Attribute-Based IPFE (AB-IPFE) scheme for a policy class $\mathcal{F}$ with function-hiding security into an MC-ABE for conjunctions of $\mathcal{F}$.
- **Bounded-Collusion Secure MC-ABE for Conjunctions of P from LWE.** In the second step, we attempt to provide a construction that does not use pairings and is secure under LWE. Since lattice-based IPFE schemes with function-hiding security are unlikely⁴ to exist, an MC-ABE scheme from LWE can only rely on an AB-IPFE with non-function-hiding security. In fact, the only existing AB-IPFE scheme with security from LWE [LLW21] is only secure against bounded collusions. We therefore modify the previous compiler to turn non-function-hiding, bounded-collusion secure AB-IPFE into bounded-collusion secure MC-ABE. When instantiated with the AB-IPFE of [LLW21], this yields a bounded-collusion secure MC-ABE for conjunctions of P. As a side result, we also obtain the first bounded-collusion secure AB-MIFE for inner products assuming the hardness of LWE.

All our MC-ABE constructions can be plugged into the compiler of [PS24] which converts constant-input MC-ABE into constant-input MC-PE for the same policy class. In particular, this yields the first constant-input, bounded-collusion secure MC-PE for conjunctions of P and improves on the MC-PE of [FFMV23] (no collusions, no labels) at the cost of relying on random oracles.

1.3 Relation to Witness Encryption

A *witness encryption* (WE) scheme for an NP relation $\mathcal{R}$ defined over a language $\mathcal{L}$ allows a sender to efficiently encrypt a message μ with respect to a problem instance x . A receiver holding a witness w can recover the message μ if $(x, w) \in \mathcal{R}$. Security requires that ciphertexts for messages μ^0 and μ^1 are computationally indistinguishable if $x \notin \mathcal{L}$.

In [BJK⁺18], the authors show that n -input ABE for a policy class $\mathcal{F}$ implies WE for relations with length n witnesses whose verification algorithm is in $\mathcal{F}$. If $n = \text{poly}(\lambda)$ and $\mathcal{F} = \text{P}$, we obtain WE for all NP relations. But since there are NP relations that can be verified in NC^1 (e.g., 3-SAT), even MI-ABE for NC^1 policies would imply WE for certain NP relations. Unfortunately, our constructions fail to imply WE because conjunction policies and classes of succinctly enumerable functions are not powerful enough to verify an NP language. The work [FFMV23] mentions an interesting alternative pathway towards WE. If the MI-ABE is secure under corruptions, then a two-input scheme for conjunctions of some policy class $\mathcal{F}$ implies WE for any relation whose verification algorithm lies in $\mathcal{F}$. Importantly, the conversion of [FFMV23] works only if the first slot has a wildcard while the second slot does not. However, as previous works on conjunction policies [FFMV23, ATY23], our construction runs into the problem that all slots have wildcards.

2 Technical Overview

We start by recalling the definition of MC-ABE. The 0-th client (a.k.a. the “encryptor”) runs an algorithm Enc which takes as input a label L_0 , an attribute x_0 and a message μ to create a ciphertext CT_{L_0, x_0} . The other

⁴Strictly speaking, the existence of lattice-based IPFE schemes with function-hiding security is not proven to be impossible, but there are strong structural limitations ruling out most natural constructions [Üna20, TÛ24].

Work	Arity	Attribute ¹	Collusion	Corruption	Labels	Policy Class	Assumptions, (Model)
[Ayy22]	2	private	✓	✗	✗	NC ¹	KOALA ² , LWE
[ARYY23]						NC ¹	private-coin evasive LWE, LWE
	const	private	✓	✗	✗	P	private-coin evasive LWE, tensor LWE, LWE
[AKY24]	const	private	✓	✗	✗	P	private-coin evasive LWE
	poly						non-uniform evasive LWE ³ , LWE
[PS24]	poly	public	✓	✓	✓	NC ⁰ or const threshold	SXDH
	const	private					SXDH, LWE
Sec. 4	poly	public	✓	✓	✓	succinctly enumerable	MDDH or [LWE, public-coin evasive LWE]
Sec. 4 + [PS24]	const	private					LWE, [MDDH or public-coin evasive LWE]
[FFMV23]	poly	private	✗	✗	✗	conjunctions of P	LWE
	const			✓			
[ATY23]	poly	public	✓	✓	✗	conjunctions of NC ¹	MDDH
[ATY23] + [PS24]	const	private					MDDH, LWE
Sec. 5	poly	public	✓	✓	✓	conjunctions of NC ¹	MDDH, LWE, (ROM)
Sec. 5 + [PS24]	const	private					
Sec. 5	poly	public	poly ⁴	✓	✓	conjunctions of P	LWE, (ROM)
Sec. 5 + [PS24]	const	private					

¹ Public attributes correspond to ABE and private attributes to PE.

² KOALA is a nonstandard knowledge type assumption on pairings.

³ Non-uniform κ -evasive LWE is a strengthening of (private-coin) evasive LWE.

⁴ The scheme supports any polynomial number of collusions, but the number must be fixed at the time of setup.

Table 1: Comparison with existing works in MI-ABE and MI-PE

clients $1, \dots, n-1$ (a.k.a. the “attribute key generators”) run an algorithm AKeyGen which takes only a label L_i and an attribute x_i to generate a decryption key DK_{L_i, x_i} . Policy decryption keys DK_f for a policy f are generated by a central authority which runs an algorithm PKeyGen . CT_{L_0, x_0} can be decrypted using $\{\text{DK}_{L_i, x_i}\}_i$ and DK_f if $L_0 = \dots = L_{n-1}$ and $f(x_0, \dots, x_{n-1}) = 0$. For security, we require the usual ciphertext indistinguishability against collusion attacks under corruptions. We summarize the syntax as follows:

$$\begin{aligned}
\text{Client } 0: & \quad \text{Enc}(\text{MPK}, L, x_0, \mu) \rightarrow \text{CT}_{x_0} \\
\text{Client } i \in [n-1]: & \quad \text{AKeyGen}(\text{SK}_i, L, x_i) \rightarrow \text{DK}_{x_i} \\
\text{Authority:} & \quad \text{PKeyGen}(\text{MSK}, f) \rightarrow \text{DK}_f
\end{aligned}$$

For completeness, we mention that MC-ABE can also be considered in an n -message setting where not only the 0-th but all slots encrypt a message. In [Ayy22], it was shown that a single-message MI-ABE scheme can be generically lifted to an n -message scheme by running n single-message schemes in parallel with rotated slots. The technique generalizes to MC-ABE.

2.1 MC-ABE for Succinctly Enumerable Function Classes

Ingredients to our Constructions. Let $\mathbb{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, g_1, g_2, g_t, e, p)$ be a pairing group. For $i \in \{1, 2, t\}$ and $a \in \mathbb{Z}_p$, we write $\llbracket a \rrbracket_i = g_i^a$ and use additive notations for the group operations.

- An *identity-based IPFE* (IB-IPFE) scheme based on $\mathbb{G}$ enables the generation of ciphertexts $\text{iCT}(i, \llbracket \mathbf{x} \rrbracket_1)$ associated with identities i and vectors $\mathbf{x} \in \mathbb{Z}_p^N$ encoded in $\mathbb{G}_1$, and the generation of (secret) decryption keys $\text{iDK}(i', \llbracket \mathbf{y} \rrbracket_2)$ for identities i' and vectors $\mathbf{y} \in \mathbb{Z}_p^N$ encoded in $\mathbb{G}_2$. Decryption of $\text{iCT}(i, \llbracket \mathbf{x} \rrbracket_1)$ with $\text{iDK}(i', \llbracket \mathbf{y} \rrbracket_2)$ reveals the inner product $\llbracket \mathbf{x} \mathbf{y}^T \rrbracket_t$ of $\mathbf{x}$ and $\mathbf{y}$ encoded in $\mathbb{G}_t$ if $i = i'$ and hides all other information about $\mathbf{x}$ whereas $\llbracket \mathbf{y} \rrbracket_2$ is usually public. If $i \neq i'$, then no information about $\mathbf{x}$ is revealed.
- In the same vein, *identity-based encryption* (IBE) allows creating ciphertexts $\text{idCT}(i, \mu)$ associated with an identity i for a message μ , and decryption keys $\text{idDK}(i')$ associated with an identity i' . Decryption is possible if and only if $i = i'$.
- A *linear secret sharing* (LSS) scheme allows to decompose a secret $s \in \mathbb{Z}_p$ into shares

$$\{s_{j,b}\}_{j \in [k], b \in \{0,1\}} \leftarrow \text{Share}(s, f)$$

with respect to a policy $f: \{0,1\}^k \rightarrow \{0,1\}$ such that s can be reconstructed from a subset of shares $\{s_{j,\mathbf{x}[j]}\}_{j \in [k]}$ for $\mathbf{x} \in \{0,1\}^k$ if and only if $f(\mathbf{x}) = 0$. For this reconstruction, there exists an efficient algorithm $\text{FindCoef}(f, \mathbf{x})$ that outputs coefficients $\omega_1, \dots, \omega_k$ such that $\sum_{j \in [k]} \omega_j s_{j,\mathbf{x}[j]} = s$.

Recap: KP-ABE from LSS Schemes and Pairing-Based IPFE. Our starting point is a well-known technique of constructing ABE from LSS schemes and pairing-based IPFE. To encrypt a message μ under an attribute vector $\mathbf{x} \in \{0,1\}^k$, one picks a random scalar $r \xleftash \mathbb{Z}_p$ and encodes it in IPFE ciphertexts in a way that reflects the choice of $\mathbf{x}$: for each $j \in [k]$, encrypt $(r, 0)$ under identity j if $\mathbf{x}[j] = 0$ and $(0, r)$ otherwise. To generate a key for a policy f , one generates secret shares $\{s_{j,b}\}_{j \in [k], b \in \{0,1\}} \leftarrow \text{Share}(s, f)$ for $s \xleftash \mathbb{Z}_p$ and encodes them in the IPFE secret keys such that IPFE decryption recovers the shares $\{r \cdot s_{j,\mathbf{x}[j]}\}_{j \in [k]}$ corresponding to $\mathbf{x}$ randomized by r .

$$\left. \begin{array}{l} \text{ABE.CT}_{\mathbf{x}}: \text{iCT}(0, \llbracket r, \mu \rrbracket_1), \{\text{iCT}(j, \llbracket 0^{\mathbf{x}[j]}, r, 0^{1-\mathbf{x}[j]} \rrbracket_1)\}_{j \in [k]} \\ \text{ABE.DK}_f: \text{iDK}(0, \llbracket s, 1 \rrbracket_2), \{\text{iDK}(j, \llbracket s_{j,0}, s_{j,1} \rrbracket_2)\}_{j \in [k]} \end{array} \right\} \Rightarrow \begin{array}{l} \llbracket r \cdot s + \mu \rrbracket_t, \\ \{\llbracket r \cdot s_{j,\mathbf{x}[j]} \rrbracket_t\}_{j \in [k]} \end{array}$$

IPFE decryption yields target group encodings of $r \cdot s + \mu$ and $r \cdot s_{j,\mathbf{x}[j]}$ for all $j \in [k]$. If $f(\mathbf{x}) = 0$, one can run $\text{FindCoef}(f, \mathbf{x}) \rightarrow \{\omega_j\}_j$ and recover the message μ encoded in $\mathbb{G}_t$:

$$\llbracket r \cdot s + \mu \rrbracket_t - \sum_{j \in [k]} \omega_j \llbracket r \cdot s_{j,\mathbf{x}[j]} \rrbracket_t = \llbracket r \cdot s + \mu \rrbracket_t - \left\llbracket r \cdot \sum_{j \in [k]} \omega_j s_{j,\mathbf{x}[j]} \right\rrbracket_t = \llbracket \mu \rrbracket_t.$$

MC-ABE for Short Inputs. To turn the above KP-ABE into a MC-ABE scheme, we have to distribute the generation of the ciphertext $\text{CT}_{\mathbf{x}}$. We decompose $\mathbf{x} = (\mathbf{x}_0, \dots, \mathbf{x}_{n-1})$ where $\mathbf{x}_0, \dots, \mathbf{x}_{n-1} \in \mathbb{Z}_p^k$. To begin with, let us assume that the total length of the input is short, i.e., $kn = O(\log \lambda)$. In this case, the encryptor, who is given $\mathbf{x}_0$ as input, can generate an independent KP-ABE ciphertext for each possible choice of $\mathbf{x}_{\geq 1} = (\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in \{0,1\}^{(n-1)k}$. Since $2^{kn} = \text{poly}(\lambda)$, the total number of ciphertexts to generate is still polynomial. If the encryptor provided all these ciphertexts without further protection, there would be no need for attribute decryption keys from the other slots since decryption is already possible. Therefore, instead of providing these ciphertexts for the other slots “in the clear”, the encryptor hides them with an additional layer of identity-based encryption

on top. Specifically, each IPFE ciphertext $\text{iCT}((i, j), \llbracket 0^{\mathbf{x}_i[j]}, r, 0^{1-\mathbf{x}_i[j]} \rrbracket_1)$ is encrypted with respect to the identity $(L, \mathbf{x}_i)$ using the public key of client i , where L denotes the label with respect to which we want to encrypt. Correspondingly, client $i \in [n-1]$ can issue identity-based decryption keys $\text{idDK}_i(L, \mathbf{x}_i)$ to remove the IBE layer added by client 0, thereby controlling for which attributes $\mathbf{x}_i$ the IPFE ciphertexts are disclosed. This results in the following construction:

$$\begin{aligned} \text{CT}_{\mathbf{x}_0} &: \left\{ \begin{array}{l} \text{iCT}(\llbracket 0, r_{\mathbf{x}_{\geq 1}}, \mu \rrbracket_1), \\ \{\text{iCT}((0, j), \llbracket 0^{\mathbf{x}_0[j]}, r_{\mathbf{x}_{\geq 1}}, 0^{1-\mathbf{x}_0[j]} \rrbracket_1)\}_{j \in [k]}, \\ \{\text{idCT}_i((L, \mathbf{x}_i), \text{iCT}((i, j), \llbracket 0^{\mathbf{x}_i[j]}, r_{\mathbf{x}_{\geq 1}}, 0^{1-\mathbf{x}_i[j]} \rrbracket_1))\}_{i \in [n-1], j \in [k]} \end{array} \right\}_{\mathbf{x}_{\geq 1} := (\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in \{0, 1\}^{(n-1)k}} \\ \text{DK}_{\mathbf{x}_i} &: \text{idDK}_i(L, \mathbf{x}_i) \\ \text{DK}_f &: \text{idDK}_0(\llbracket s, 1 \rrbracket_2), \{\text{idDK}((i, j), \llbracket s_{i,j,0}, s_{i,j,1} \rrbracket_2)\}_{i \in [0; n-1], j \in [k]} \end{aligned}$$

Here, $r_{\mathbf{x}_{\geq 1}} \xleftarrow{\$} \mathbb{Z}_p$ is a fresh random scalar for each $\mathbf{x}_{\geq 1} \in \{0, 1\}^{(n-1)k}$. Given $\text{CT}_{\mathbf{x}_0}$ and $\text{DK}_{\mathbf{x}_1}, \dots, \text{DK}_{\mathbf{x}_{n-1}}$, the decryptor first runs IBE decryption to recover the IPFE ciphertexts corresponding to $\mathbf{x}_1, \dots, \mathbf{x}_{n-1}$. In the second step, using the IPFE keys in DK_f , it runs IPFE decryption to obtain $\llbracket r_{\mathbf{x}_{\geq 1}} \cdot s + \mu \rrbracket_t$ and $\{\llbracket r_{\mathbf{x}_{\geq 1}} \cdot s_{j, \mathbf{x}_i[j]} \rrbracket_t\}_{i \in [0; n-1], j \in [k]}$. Finally, it is possible to recover $\llbracket \mu \rrbracket_t$ using LSS reconstruction as in the KP-ABE case.

Succinct Enumerability. This naive MC-ABE construction becomes inefficient if k or n are chosen too large. The authors of [PS24] observe however that the size of the ciphertexts can be kept polynomial for particular function classes even when $k, n = \text{poly}(\lambda)$. As an intuitive example, let us consider NC^0 policies. The output of a function that can be computed by an NC^0 circuit depends only on a set $\mathfrak{s} \subseteq [0; n-1] \times [k]$ of size $\tau = O(1)$ out of the total of kn inputs. As the remaining $kn - \tau$ inputs can be chosen arbitrarily without changing the output, it suffices to consider vectors $\mathbf{x}_{\geq 1}$ that are 0 outside $\mathfrak{s}$. There are $\binom{kn}{\tau} \leq (kn)^\tau = \text{poly}(\lambda)$ possible sets $\mathfrak{s}$ and for each choice we only need to consider $2^\tau = O(1)$ vectors $\mathbf{x}_{\geq 1}$. So the total size of the ciphertexts remains polynomial.

Let us take a closer look at the concrete properties that make the construction work even for $k, n = \text{poly}(\lambda)$. Initially, the encryptor needs to pick a subset $V \subseteq \{0, 1\}^{[n-1] \times [k]}$ for which it provides independent ciphertexts. This set must be of polynomial size and cannot depend on inputs $\mathbf{x}_1, \dots, \mathbf{x}_{n-1}$ or a policy f as these values are unknown to the encryptor. To guarantee correctness, the set V must be chosen in such a way that, for each combination of attributes $(\mathbf{x}_0, \dots, \mathbf{x}_{n-1})$ and function f such that $f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) = 0$, there exists an element $\mathbf{v} \in V$ that coincides with $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1})$ on a “sufficiently large” subset of the indices $[n-1] \times [k]$ such that we can recover enough shares to run the LSS reconstruction algorithm. In a bit more detail, to make sure that reconstruction works even without holding a share for each coordinate, we view the policies as functions with a restricted domain by ignoring all inputs that do not have an impact on the output. Note that these restricted domains may be different for each function of the class. While this domain restriction guarantees correctness, we must be careful not to violate security. In particular, we have to make sure that we do not give the adversary the opportunity to change the output of the function when restricting the domain too much.

This leads us to the following condition dubbed *succinct enumerability* which is sufficient for a policy class to be captured by our construction. Let us introduce the following notations. We set $X_0 = \{0\}$, $X_1 = \{1\}$ and $X_\star = \{0, 1\}$. Given a vector $\mathbf{v} \in \{0, 1, \star\}^k$, we denote $X_{\mathbf{v}} = X_{\mathbf{v}[1]} \times \dots \times X_{\mathbf{v}[k]}$. Furthermore, for concatenated vectors $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in \{0, 1, \star\}^{[n-1] \times [k]}$, we write

$$X_{\mathbf{v}} = X_{\mathbf{v}_1} \times \dots \times X_{\mathbf{v}_{n-1}} \quad \text{and} \quad J_{\mathbf{v}} = \{(i, j) \in [n-1] \times [k] : \mathbf{v}_i[j] \neq \star\}.$$

On an intuitive level, the set $X_{\mathbf{v}}$ contains all $\mathbf{x}_{\geq 1} = (\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in \{0, 1\}^{[n-1] \times [k]}$ that coincide with $\mathbf{v}$ in the $\{0, 1\}$ -coordinates and have an arbitrary value in $\mathbf{v}$'s $\star$ -coordinates. $J_{\mathbf{v}}$ contains the indices of all non- $\star$ coordinates of $\mathbf{v}$. A policy class $\mathcal{F}$ is said to be efficiently enumerable if there exists a fixed set $V \subseteq \{0, 1, \star\}^{[n-1] \times [k]}$ of polynomial size with the following properties. For each policy $f \in \mathcal{F}$ there exists an index set $J_f \subseteq [n-1] \times [k]$ such that

1. for each $(\mathbf{x}_0, \mathbf{x}_{\geq 1}) \in \{0, 1\}^{\{0\} \times [k]} \times \{0, 1\}^{\{n-1\} \times [k]}$, if $f(\mathbf{x}_0, \mathbf{x}_{\geq 1}) = 0$ then there exists $\mathbf{v} \in V$ such that $J_{\mathbf{v}} = J_f$ and $\mathbf{x}_{\geq 1} \in X_{\mathbf{v}}$, and
2. for each $\mathbf{x}_0 \in \{0, 1\}^{\{0\} \times [k]}$ and $\mathbf{v} \in V$, if $J_{\mathbf{v}} = J_f$ then the function f is constant on the set $\{(\mathbf{x}_0, \mathbf{x}_{\geq 1}) : \mathbf{x}_{\geq 1} \in X_{\mathbf{v}}\}$.

The first condition is needed for correctness to ensure that we can always recover a set of shares sufficient for reconstruction. The second property is used during the security proof to justify the restriction of the policy f to the smaller domain J_f . When adapting the construction from the NC^0 case to a general succinctly enumerable function class, we obtain the following scheme.

Construction 1 (MC-ABE for Succinctly Enumerable Policy Classes – Informal). Let $\text{iFE} = (\text{iSetup}, \text{iEnc}, \text{iKeyGen}, \text{iDec})$ be a pairing-based IB-IPFE scheme and $\text{idFE} = (\text{idSetup}, \text{idEnc}, \text{idKeyGen}, \text{idDec})$ be an IBE scheme.

Setup(1^λ): Sample $\text{iMSK} \leftarrow \text{iSetup}(1^\lambda)$ and $\text{idMSK}_i \leftarrow \text{idSetup}(1^\lambda)$ for $i \in [n-1]$. Then output $\text{MPK} = (\text{iMPK}, \{\text{idMPK}_i\}_{i \in [n-1]}, \{\text{SK}_i = \text{idMSK}_i\}_{i \in [n-1]})$ and $\text{MSK} = \text{iMSK}$.

Enc($\text{MPK}, L, \mathbf{x}_0, \llbracket \mu \rrbracket_t$) Sample $\llbracket r^{\mathbf{v}} \rrbracket_1 \xleftarrow{\$} \mathbb{G}_1^K$ for each $\mathbf{v} \in V$, set $\mathbf{v}_0 = \mathbf{x}_0$ and run

$$\begin{aligned} \{\text{iCT}_0^{\mathbf{v}} &\leftarrow \text{iEnc}(\text{iMPK}, 0, \llbracket r^{\mathbf{v}} \rrbracket_1, \mu)\}_{\mathbf{v} \in V} \\ \{\text{iCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{iEnc}(\text{iMPK}, (i, j), \llbracket 0^{\mathbf{v}_i[j]} \rrbracket_1, r^{\mathbf{v}}, 0^{(1-\mathbf{v}_i[j])})\}_{(i,j) \in J_{\mathbf{v}} \cup (\{0\} \times [k])}^{\mathbf{v}=(\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V} \\ \{\text{idCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{idEnc}(\text{idMPK}_i, (L, \mathbf{v}_i), \text{iCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in J_{\mathbf{v}}}^{\mathbf{v}=(\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V}. \end{aligned}$$

$$\text{Output CT} = (\llbracket d \rrbracket_t, \{\text{iCT}_0^{\mathbf{v}}\}_{\mathbf{v} \in V}, \{\text{iCT}_{0,j}^{\mathbf{v}}\}_{j \in [k]}, \{\text{idCT}_{i,j}^{\mathbf{v}}\}_{(i,j) \in J_{\mathbf{v}}}).$$

AKeyGen($\text{SK}_i, L, \mathbf{x}_i$) Define $V_{\mathbf{x}_i} = \{\mathbf{v}_i : (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V, \mathbf{x}_i \in X_{\mathbf{v}_i}\}$ and run

$$\{\text{idDK}_i^{\mathbf{v}_i} \leftarrow \text{idKeyGen}(\text{idMSK}_i, (L, \mathbf{v}_i))\}_{\mathbf{v}_i \in V_{\mathbf{x}_i}}.$$

$$\text{Output DK}_i = \{\text{idDK}_i^{\mathbf{v}_i}\}_{\mathbf{v}_i \in V_{\mathbf{x}_i}}.$$

PKeyGen(MSK, f) Sample $s \xleftarrow{\$} \mathbb{Z}_p^K$, generates shares $\{s_{i,j,b}\}_{(i,j) \in J_f, b \in \{0,1\}} \leftarrow \text{Share}(f, s)$ and run

$$\begin{aligned} \text{idK}_0 &\leftarrow \text{iKeyGen}(\text{iMSK}, 0, \llbracket s, 1 \rrbracket_2) \\ \text{idK}_{i,j} &\leftarrow \text{iKeyGen}(\text{iMSK}, (i, j), \llbracket s_{i,j,0}, s_{i,j,1} \rrbracket_2) \quad \text{for each } (i, j) \in J_f \cup (\{0\} \times [k]). \end{aligned}$$

$$\text{Output DK}_f = (\text{idK}_0, \{\text{idK}_{i,j}\}_{(i,j) \in J_f}).$$

Dec($\text{DK}_f, \{\text{DK}_i\}_{i \in [n-1]}, \text{CT}$) Pick $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V$ such that $J_{\mathbf{v}} = J_f$ and $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in X_{\mathbf{v}}$. Run

$$\begin{aligned} \{\text{iCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{idDec}(\text{idDK}_i^{\mathbf{v}_i}, \text{iCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in J_{\mathbf{v}}} \\ \{\llbracket d_{i,j} \rrbracket_t &\leftarrow \text{idDec}(\text{idK}_{i,j}, \text{iCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in J_{\mathbf{v}} \cup (\{0\} \times [k])} \\ \llbracket d_0 \rrbracket_t &\leftarrow \text{idDec}(\text{idK}_0, \text{iCT}_0^{\mathbf{v}}). \end{aligned}$$

Run LSS reconstruction on the set $\{\llbracket d_{i,j} \rrbracket_t\}_{(i,j) \in J_{\mathbf{v}} \cup (\{0\} \times [k])}$ to find $\llbracket d' \rrbracket_t = \llbracket r \cdot s \rrbracket_t$ and output $\llbracket d_0 - d' \rrbracket_t$.

The first property of succinctly enumerable function classes guarantees that there exists $\mathbf{v} \in V$ such that $J_{\mathbf{v}} = J_f$ and $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in X_{\mathbf{v}}$. This implies that all idFE ciphertexts $\{\text{idCT}_{i,j}^{\mathbf{v}}\}_{(i,j) \in J_{\mathbf{v}}}$ and secret keys $\{\text{idDK}_i^{\mathbf{v}_i}\}_{(i,j) \in J_{\mathbf{v}}}$ used in the first step of decryption are contained in CT_L and $\{\text{DK}_i\}_{i \in [n-1]}$, respectively. It then follows that $d_0 = \mu + r \cdot s$ and $d_{i,j} = r \cdot s_{i,j, \mathbf{x}_i[j]}$. If $f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) = 0$, the LSS reconstruction correctly recovers $\llbracket r \cdot s \rrbracket_t$ and $\llbracket \mu \rrbracket_t$ is revealed.

Security. The security proof follows a well-established technique. We use a primitive called slotted IPFE [LL20] which is a mix between public-key and private-key IPFE that provides standard security on the public part and additionally hides the function vectors on the private part. While the real scheme only uses the public part, the private part helps us to move the scalars r from the message vectors encoded in $\mathbb{G}_1$ into a hidden coordinate of the function vectors in $\mathbb{G}_2$. The only values that the adversary learns after this transformation are

- encodings $\llbracket s \rrbracket_2$ of random scalars $s \xleftarrow{\$} \mathbb{Z}_p$ sampled during the key generation and their corresponding shares $\{\llbracket s_{i,j,b} \rrbracket_2\}_{i \in J_f \cup (\{0\} \times [k])}$, and
- encodings of the form $\llbracket r \cdot s + \mu \rrbracket_2$ (iFE identity 0) and $\llbracket r \cdot s_{i,j,x_i[j]} \rrbracket_2$ (iFE identity (i, j)) for random scalars $r \xleftarrow{\$} \mathbb{Z}_p$ sampled during the encryption of the challenge message.

Importantly, nothing about r is leaked in $\mathbb{G}_1$. So we can rely on the DDH assumption to obtain a fresh looking mask $\llbracket \tilde{s} \rrbracket_t \approx_c \llbracket r \cdot s \rrbracket_t$ with fresh shares $\llbracket \tilde{s}_{i,j} \rrbracket_t \approx_c \llbracket \{\llbracket s_{i,j,x_i[j]} \rrbracket_2\}_{i \in J_f \cup (\{0\} \times [k])} \rrbracket_2$ for each combination of r and s . Then we can exploit the one-time, information-theoretic security provided by the LSS scheme to replace the individual secret sharings with random values: by the admissibility of the adversary and the second item of the succinct enumerability property, there does not exist any pair (r, s) such that the adversary has sufficient information about a subset of shares that allows her to recover the mask $\llbracket r \cdot s \rrbracket_t$. Instead, they look uniformly random and, thus, perfectly hide the challenge message.

Lattice-Based Construction. Slotted IPFE is only known from the MDDH assumption on pairings. A lattice-based instantiation of Construction 1 must therefore employ a different IPFE primitive. The crucial step in the above sketch of the security proof happens when applying the DDH assumption: at this point, we switch from *one* set of secret shares that appears in the decryption results with all ciphertexts to *many* independent sharings, one for each ciphertext. In the lattice setting, we need an IPFE primitive which allows us to perform a similar step while relying on LWE. Very recently, Hsieh et al. [HLL24] introduced a new variant of IPFE called *evasive* IPFE for precisely this purpose. Using this new IPFE primitive and a noisy variant of LSS schemes also introduced in that work, everything follows quite readily. Notably, we can rely on a restricted security requirement of evasive IPFE in which case security can be reduced to public-coin evasive LWE. We omit further details here and refer the reader to Section 4.3.

2.2 MC-ABE for Conjunction Policies

The work of Agrawal et al. [ATY23], henceforth referred to as ATY, presents an AB-MIFE for the Attribute-Weighted Sums (AWS) functionality. We recall their construction in the special case of inner products, i.e. the functionality $F: (\mathcal{X} \times \mathbb{Z}_p^d)^n \rightarrow \mathbb{G}_t$ specified by $((f_0, \mathbf{v}_0), \dots, (f_{n-1}, \mathbf{v}_{n-1})) \in (\mathcal{F} \times \mathbb{Z}_p^d)^n$ and defined as

$$F((x_0, \mathbf{u}_0), \dots, (x_{n-1}, \mathbf{u}_{n-1})) = \begin{cases} \sum_{i \in [0; n-1]} \llbracket \mathbf{u}_i \mathbf{v}_i^\top \rrbracket_t & \text{if } f_i(x_i) = 0 \text{ for all } i \in [0; n-1], \\ \perp & \text{otherwise.} \end{cases}$$

The construction's main building block is an AB-IPFE scheme iFE = (iSetup, iEnc, iKeyGen, iDec) with (partially) function-hiding security. Specifically, $\text{iCT} \leftarrow \text{iEnc}(\text{MSK}, x, \mathbf{u}')$ takes as input the master secret key MSK, a public attribute $x \in \mathcal{X}$ and a vector $\mathbf{u}' \in \mathbb{Z}_p^{d+1}$; $\text{iDK} \leftarrow \text{iKeyGen}(\text{MSK}, f, \mathbf{v}')$ takes MSK, a policy $f \in \mathcal{F}$ and a vector $\mathbf{v}' \in \mathbb{Z}_p^{d+1}$. If $f(x) = 0$, then $\text{iDec}(\text{iDK}, \text{iCT})$ returns $\mathbf{u}' \cdot (\mathbf{v}')^\top$ (but reveals nothing else about $\mathbf{u}'$ and $\mathbf{v}'$). Conversely, if $f(x) \neq 0$, then $\mathbf{u}'$ and $\mathbf{v}'$ are completely hidden.

Recap: The AB-MIFE of ATY. In the AB-MIFE construction of ATY, the setup algorithm provides each slot $i \in [0; n-1]$ with their own secret key $\text{SK}_i := \text{iMSK}_i \leftarrow \text{iSetup}(1^\lambda)$ and defines the corresponding master secret key $\text{MSK} = \{\text{iMSK}_i\}_{i \in [0; n-1]}$. Given $(x_i, \mathbf{u}_i)$, the encryptor for slot $i \in [0; n-1]$ runs $\text{iCT}_i \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, \mathbf{u}'_i)$ for $\mathbf{u}'_i = (\mathbf{u}_i, 1)$ and outputs $\text{CT}_i := \text{iCT}_i$. Given $\{(f_i, \mathbf{v}_i)\}_{i \in [0; n-1]}$, the key generator samples $t_1, \dots, t_{n-1} \xleftarrow{\$} \mathbb{Z}_p$,

sets $t_0 = -\sum_{i \in [n-1]} t_i$, runs $iDK_i \leftarrow iKeyGen(iMSK_i, f_i, \mathbf{v}'_i)$ for $\mathbf{v}'_i = (\mathbf{v}_i, t_i)$ and outputs $DK = \{iDK_i\}_{i \in [0; n-1]}$. If $f_i(x_i) = 0$ for all $i \in [0; n-1]$, decryption outputs

$$\sum_{i \in [0; n-1]} iDec(iDK_i, iCT_i) = \sum_{i \in [0; n-1]} [\mathbf{u}_i \mathbf{v}_i^\top]_t,$$

where the equality follows from the fact that $\sum_{i \in [0; n-1]} t_i = 0$. The random element t_i is used during the security proof to hide partial decryption values $\mathbf{u}_i \mathbf{v}_i^\top$. Importantly, this scheme is only secure if the adversary does not possess so-called illegitimate keys. A decryption key for a function $F = ((f_0, \mathbf{v}_0), \dots, (f_{n-1}, \mathbf{v}_{n-1}))$ is said to be *illegitimate* if, for some slot i , the adversary did not query for a slot- i ciphertext encoding $(x_i, \star)$ such that $f_i(x_i) = 0$. We refer the reader to ATY for a detailed discussion why the scheme becomes insecure in the presence of illegitimate keys. Importantly for us, ATY presents a generic compiler to remove this restriction from the security model and achieve security against any keys.

From MI-ABE to MC-ABE. The scheme of ATY directly implies an MI-ABE for conjunction policies. Let $d = 1$. Given a message $[\mu]_t \in \mathbb{G}_t$, the encryptor (slot 0) samples a random masking term $r \xleftarrow{\$} \mathbb{Z}_p$, encrypts $iCT_0 \leftarrow iEnc(iMSK_0, x_0, (r, 1))$ and outputs $CT := (iCT_0, [c]_t = [r + \mu]_t)$. The attribute key generators (slots $i \in [n-1]$) proceed in the same way as in the AB-MIFE scheme except that they always set $\mathbf{u}_i = 0$. If $f_i(x_i) = 0$ for all $i \in [0; n-1]$, then iFE decryption reveals $\sum_{i \in [0; n-1]} iDec(iDK_i, iCT_i) = [r]_t$, and we can recover $[\mu]_t = [c - r]_t$. We note that this yields an MI-ABE in the secret-key setting, where the encryptor takes a secret key SK_0 instead of a public key MPK. As the security relies on the *function-hiding* property of iFE, it is not possible to replace iFE with a public-key AB-IPFE which might allow to disclose the encryption key for slot 0.⁵

We next discuss how to lift this MI-ABE to MC-ABE. Fortunately for us, the AB-MIFE of ATY (and thus the implied MI-ABE) are already secure against corruptions of secret keys. So the only missing feature towards MC-ABE is managing different labels. For the case of security against legitimate keys, such a scheme can be obtained by generalizing known techniques for building MCFE for inner products [ABG19] to the attribute-based setting. Specifically, we use a fresh set of random scalars $t_{L,0}, \dots, t_{L,n-1} \xleftarrow{\$} \mathbb{Z}_p$ for each label L and move them from the key vectors $\mathbf{v}'_i$ into the ciphertext vectors $\mathbf{u}'_i$, i.e., $\mathbf{u}'_i = (\mathbf{u}_i, t_{L,i})$. Correspondingly, key generation samples a random scalar $s \xleftarrow{\$} \mathbb{Z}_p$ and places it in the coordinates facing $t_{L,i}$, i.e., $\mathbf{v}'_i = (\mathbf{v}_i, s)$. The scalars $\{t_{L,i}\}_{i \in [0; n-1]}$ can be simulated “on the fly” by using a pseudorandom function PRF with key space $\mathcal{K}$. For each $i, j \in [0; n-1]$, $i < j$, sample $K_{i,j} \xleftarrow{\$} \mathcal{K}$ and set $K_{i,j} = K_{j,i}$ for $j > i$. Then we let

$$t_{L,i} = \sum_{j \in [0; n-1] \setminus \{i\}} (-1)^{(j < i)} \text{PRF}_{K_{i,j}}(L).$$

It is not hard to see that $\sum_{i \in [0; n-1]} t_{L,i} = 0$ for any label L . Pseudorandomness follows from the security of PRF.

Security Against Any Keys. While this AB-MCFE scheme trivially follows from existing constructions, it only provides security against legitimate keys. The actual hurdle lies in achieving security against illegitimate keys because the ATY compiler from the AB-MIFE setting is not able to handle different labels. We do not know how to deal with this issue in the general case of AB-MCFE. Let us therefore focus on how to deal with illegitimate keys in the special case of MC-ABE.

In this case, the legitimacy condition simplifies as there is only one slot that encrypts a message and thus receives left-or-right queries during the security game. Specifically, a key for a policy described by $(f_0, \dots, f_{n-1})$ is regarded illegitimate in the spirit of the prior definition if there exists a slot $i \in [n-1]$ and a label L such that

1. $\mathcal{A}$ does obtain a ciphertext for (L, x_0, μ) such that $f_0(x_0) = 0$, but
2. $\mathcal{A}$ does not obtain an attribute decryption key DK_i for L and an attribute x_i such that $f_i(x_i) = 0$.

⁵In more detail, if the adversary held an IPFE encryption key, she could always generate ciphertexts of all unit vectors by herself which allow recovering the coordinates of the key vector one by one. Therefore, function-hiding IPFE cannot exist in the public-key setting.

In this case, the adversary is supposed to learn nothing about μ as globally the policy is not satisfied. Looking at our construction, however, we can observe that this is not true. To see why, let us consider a two-client scheme where an adversary obtains ciphertexts of (L, x_0, μ_1) and (L, x_0, μ_2) for slot 0 (denoted by $\text{CT}_1 = (\text{iCT}_1, \llbracket c_1 \rrbracket_t = \llbracket \mu_1 + r_1 \rrbracket_t)$ and $\text{CT}_2 = (\text{iCT}_2, \llbracket c_2 \rrbracket_t = \llbracket \mu_2 + r_2 \rrbracket_t)$) and a policy decryption key for $F = (f_0, f_1)$ denoted by $\text{SK}_F = \{\text{iDK}_i\}_{i \in [0,1]}$ such that $f_0(x_0) = 0$. In this case, the adversary should not obtain any information about μ_1 and μ_2 since she does not possess an attribute decryption key $\text{DK}_{L,1}$ satisfying f_1 . However, the adversary can compute

$$\llbracket r_\Delta \rrbracket_t = \text{iDec}(\text{iDK}_0, \text{iCT}_1) - \text{iDec}(\text{iDK}_0, \text{iCT}_2) = \llbracket (st_{L,0} + r_1) - (st_{L,0} + r_2) \rrbracket_t = \llbracket r_1 - r_2 \rrbracket_t,$$

where we recall that iCT_j , for $j \in \{1,2\}$, encrypts a vector $(t_{L,0}, r_j)$ and iDK_0 encodes a vector $(s, 1)$. Thus, the adversary can learn $\llbracket c_1 - c_2 - r_\Delta \rrbracket_t = \llbracket \mu_1 - \mu_2 \rrbracket_t$. This is leakage we need to avoid.

At this point, our observation is that even though the adversary can learn certain relations between the (potentially many) masking terms r , the resulting linear system is always underdetermined. More precisely, an analysis of this linear system shows that the marginal distribution of each r is still uniform. Therefore, we can use a random oracle to turn the dependent random variables into independent variables which can perfectly hide the messages μ_1 and μ_2 . The resulting scheme can be summarized as follows. It provides selective security in the ROM assuming MDDH. Note that we can simplify the generation of the elements $t_{L,i}$ as there is only one message-encryption slot in MC-ABE.

Construction 2 (MC-ABE from Function-Hiding IPFE in ROM – Informal). Let $\text{iFE} = (\text{iSetup}, \text{iEnc}, \text{iKeyGen}, \text{iDec})$ be an AB-IPFE scheme with function-hiding security on the inner-product vectors, PRF be a pseudorandom function with key space $\mathcal{K}$, and $H: \mathbb{G}_t \rightarrow \mathbb{G}_t$ be a hash function modeled as a random oracle.

Setup(1^λ): Sample $\text{iMSK}_i \leftarrow \text{iSetup}(1^\lambda)$ for $i \in [0; n-1]$, and $K_i \xleftarrow{\$} \mathcal{K}$ for $i \in [n-1]$. Output $\text{SK}_0 := (\text{iMSK}_0, \{K_i\}_{i \in [n-1]})$, $\{\text{SK}_i := (\text{iMSK}_i, K_i)\}_{i \in [n-1]}$ and $\text{MSK} := \{\text{iMSK}_i\}_{i \in [0; n-1]}$.

Enc($\text{SK}_0, L, x_0, \llbracket \mu \rrbracket_t$): Sample $r \xleftarrow{\$} \mathbb{Z}_p$, set $t_{L,0} = -\sum_{i \in [n-1]} \text{PRF}_{K_i}(L)$, run $\text{iCT}_0 \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, \llbracket t_{L,0}, r \rrbracket_1)$ and output $\text{CT} := (\text{iCT}_0, \llbracket c \rrbracket_t = \llbracket \mu \rrbracket_t + H(\llbracket r \rrbracket_t))$.

AKeyGen(SK_i, L, x_i): Set $t_{L,i} = \text{PRF}_{K_i}(L)$, run $\text{iCT}_i \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, \llbracket t_{L,i}, 0 \rrbracket_1)$ and output $\text{DK}_i := \text{iCT}_i$.

PKeyGen($\text{MSK}, f = (f_0, \dots, f_{n-1})$): Run $\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}, f_i, \llbracket s, 1 \rrbracket_2)$ for each $i \in [0; n-1]$, where $s \xleftarrow{\$} \mathbb{Z}_p$. Output $\text{DK}_f := \{\text{iDK}_i\}_{i \in [0; n-1]}$.

Dec($\text{DK}_f, \{\text{DK}_i\}_{i \in [n-1]}, \text{CT}$): For $i \in [0; n-1]$, run $\llbracket d_i \rrbracket_t \leftarrow \text{iDec}(\text{iDK}_i, \text{iCT}_i)$. Output $\llbracket c \rrbracket_t - H(\sum_{i \in [0; n-1]} \llbracket d_i \rrbracket_t)$.

Security in the Random Oracle Model. Let us analyze why the random oracle H suffices to prove security against illegitimate keys. Suppose the adversary obtains a policy decryption key $\text{DK}_f := \{\text{iDK}_i\}_{i \in [0; n-1]}$ for some policy f and ciphertexts $\text{CT}_j := (\text{iCT}_{j,0}, \llbracket c_j \rrbracket_t = \llbracket \mu \rrbracket_t + H(\llbracket r_j \rrbracket_t))$ for $j \in [q]$, where $r_j \xleftarrow{\$} \mathbb{Z}_p$ is sampled for the reply to the j -th encryption query $\text{QEnc}(L, x_{j,0}, \llbracket \mu_j \rrbracket_t)$ with respect to a label L such that $f(x_j) = 0$. Roughly speaking, this allows the adversary to learn the values $\llbracket r'_j \rrbracket_t = \llbracket st_{L,0} + r_j \rrbracket_t = \llbracket -s \sum_{i \in [n-1]} t_{L,i} + r_j \rrbracket_t$ by running iFE decryption. Hence, for proving security we must argue that the masks $H(\llbracket r_j \rrbracket_t)$ are pseudorandom even if the adversary learns the terms $\llbracket r'_j \rrbracket_t$. By item 2, it follows that there exists a slot $i \in [n-1]$ such that the adversary does not learn $\llbracket st_{L,i} \rrbracket_t$, and we note that $\llbracket st_{L,i} \rrbracket_t \approx_c \llbracket s' \rrbracket_t$ for $s' \xleftarrow{\$} \mathbb{Z}_p$ under the DDH assumption. So it suffices to show that the following distributions are indistinguishable

$$\{\{\llbracket s' + r_j \rrbracket_t, H(\llbracket r_j \rrbracket_t)\}\}_{j \in [q]} \approx_c \{\{\llbracket s' + r_j \rrbracket_t, \llbracket y_j \rrbracket_t\}\}_{j \in [q]},$$

where $y_1, \dots, y_q \xleftarrow{\$} \mathbb{Z}_p$. For a single query to the random oracle, the probability to have asked an r_j is bounded by q/p . This allows the adversary to exclude up to q^2 points using the constraints on $\{s' + r_j\}_{j \in [q]}$. Thus, after $t = \text{poly}(\lambda)$ queries, the probability to have queried the random oracle on one of the r_j is at most $qt/(p - q^2 t)$ which is negligible in λ .

Lattice-Based Construction. We now focus on a construction that does not use pairings and can be proven secure assuming LWE. To our knowledge, there is currently no lattice-based AB-MIFE scheme for inner products in the literature. We therefore go back to the original AB-MIFE of ATY and attempt to build a lattice-version for this functionality first. Subsequently, we can follow the same path towards MC-ABE as described in the pairing-based setting since these ideas do not exploit the underlying group or pairing structure.

The biggest hurdle in the lattice-based setting is that we can no longer rely on AB-IPFE with function-hiding security, so the ATY compiler from (single-input) AB-IPFE to AB-MIFE cannot be instantiated anymore. As observed by ATY, existing MIFE constructions for inner products (without access control) can roughly be classified into two categories depending on which property of the employed IPFE primitive is used for the security proof: (1) ciphertext homomorphism, e.g., [AGRW17, CDG⁺18, ACF⁺18, ABG19], and (2) function-hiding security, e.g., [DOT18, Tom19, AGT21b, SV23]. Here, ciphertext homomorphism means that both message space and ciphertext space form a group, and taking the sum of two ciphertexts encrypting messages $\mathbf{u}_1$ and $\mathbf{u}_2$ yields a ciphertext for $\mathbf{u}_1 + \mathbf{u}_2$. In the case of AB-MIFE, we evaluate non-linear policies such as ABPs on the public input. It is therefore hopeless trying to equip an AB-IPFE scheme with ciphertext homomorphism. This rules out all lattice-based MIFE constructions for inner products that we could potentially generalize to the attribute-based setting.⁶ It therefore seems inevitable to somehow mimic the proof technique of the function-hiding case even though we cannot rely on function-hiding security.

To our knowledge, the only existing AB-IPFE scheme in the lattice setting was presented by Lai et al. in [LLW21] and is only secure against *bounded collusions*, i.e., the adversary obtains only a polynomial number Q of decryption keys with Q being fixed at the time of setup. Given that even in the single-input case the best known construction is only secure against bounded collusions, we restrict ourselves to the same level of security in the multi-input case.

It turns out that this limitation helps a lot. To see why, we recall that the AB-MIFE of ATY encrypts $(x_i, \mathbf{u}_i)$ by running $\text{iCT}_i \leftarrow \text{iEnc}(\text{iMSK}_0, x_i, \mathbf{u}'_i)$ for the vector $\mathbf{u}'_i = (\mathbf{u}_i, 1)$. To generate a decryption key for policy-vector pairs $\{(f_i, \mathbf{v}_i)\}_{i \in [0; n-1]}$, one samples $t_0, \dots, t_{n-1} \xleftarrow{\$} \mathbb{Z}_p$ such that $\sum_{i \in [0; n-1]} t_i = 0$ and computes $\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}_i, f_i, \mathbf{v}'_i)$ for all $i \in [0; n-1]$ for the vectors $\mathbf{v}'_i = (\mathbf{v}_i, t_i)$. Suppose that the adversary submits a slot- i encryption query $\text{QEnc}(i, x_i, \mathbf{u}_i^0, \mathbf{u}_i^1)$ for an attribute x_i and message vectors $(\mathbf{u}_i^0, \mathbf{u}_i^1)$ along with a key generation query $\text{QKeyGen}(\{(f_{i'}, \mathbf{v}_{i'})\}_{i' \in [0; n-1]})$ for policy-vector pairs $(f_{i'}, \mathbf{v}_{i'})$. If $f_i(x_i) = 0$, then the adversary can learn $\mathbf{u}'_i(\mathbf{v}'_i)^\top = \mathbf{u}_i \mathbf{v}_i^\top + t_i$, where the random scalar t_i is used to hide the local inner product $\mathbf{u}_i \mathbf{v}_i$ from the adversary. More precisely, during the security proof the scalar t_i is used to compensate for the difference between the left and the right inner product $\delta_i := \mathbf{u}_i^0 \mathbf{v}_i^\top - \mathbf{u}_i^1 \mathbf{v}_i^\top$ in slot i . Since t_i is placed in the key vectors, this compensation step requires function-hiding security. Without function-hiding, the only choice is to flip the construction and put the scalar t_i in the ciphertexts instead. (Note that we already did this in our pairing-based MC-ABE scheme for different reasons). While this seemingly solves the issue with the lack of function-hiding security, it immediately leads to a new problem. This is because the value δ_i does not only depend on the slot index i , but also on the vector $\mathbf{v}_i$, i.e., δ_i can be different for every key query. In the pairing-based setting, this issue can be dealt with by placing a random scalar s in the key vector in the coordinate facing t_i . Under a DDH-style assumption, we can obtain a different randomization of t_i for each key. But we do not have an analogous assumption in the lattice-based setting. So we cannot randomize t_i during the security proof anymore.

Security Against Bounded Collusions. This is the point where the restriction to a fixed number Q of decryption keys comes to the rescue as it limits the total number of different δ_i 's. It is therefore possible to embed sufficiently many independent random scalars $t_i^1, t_i^2, \dots$ into the ciphertext at the time of encryption. Naively, we could use Q different t_i 's, one for each decryption key. But this would require a stateful key generation algorithm which maintains a counter to keep track of which t_i 's were already used for previously issued keys. To avoid this, we rely on a probabilistic argument using cover-free sets, inspired by [Agr17, LLW21]. Intuitively,

⁶The lattice-based construction of Libert and Titu [LT19] is somewhat different in the sense that it does not really fit in one of the two categories. However, we do not know how to enhance their construction with access control.

for a fixed $Q = \text{poly}(\lambda)$, we can choose κ, τ such that independently sampled subsets $\Delta_1, \dots, \Delta_Q \subset [\kappa]$ of size τ satisfy $\Delta_i \setminus (\bigcup_{j \in [Q] \setminus \{i\}} \Delta_j) \neq \emptyset$ for all i with high probability.

We can use this result in the following way. During encryption, we replace $t_i \xleftarrow{\$} \mathbb{Z}_p$ with a vector $\mathbf{t}_i$ of dimension κ , i.e. we let $\mathbf{u}'_i = (\mathbf{u}_i, \mathbf{t}_i)$. During key generation, we sample a random subset $\Delta \subset [\kappa]$, define the vector $\mathbf{e}_\Delta = \sum_{k \in \Delta} \mathbf{e}_k \in \mathbb{Z}_p^\kappa$ and set $\mathbf{v}'_i = (\mathbf{v}_i, \mathbf{e}_\Delta)$. For the security proof, we use the fact that for each key $j \in [Q]$, there exists $k_j \in [\kappa]$ such that $\mathbf{t}_i[k_j]$ appears *only* when decrypting a ciphertext with the j -th decryption key. Hence, we can use $\mathbf{t}_i[k_j]$ to compensate for the value δ_i with respect to the j -th decryption key. The resulting AB-MIFE scheme can be summarized as follows:

Construction 3 (AB-MIFE Against Bounded Collusions – Informal). Let $\text{iFE} = (\text{iSetup}, \text{iEnc}, \text{iKeyGen}, \text{iDec})$ be an AB-IPFE scheme with standard security.

Setup(1^λ): Sample $\text{iMSK}_i \leftarrow \text{iSetup}(1^\lambda)$ and $\mathbf{t}_i \xleftarrow{\$} \mathbb{Z}_p^\kappa$ for $i \in [0; n-1]$ such that $\sum_{i \in [0; n-1]} \mathbf{t}_i = \mathbf{0}$. Output $\{\text{SK}_i := (\text{iMSK}_i, \mathbf{t}_i)\}_{i \in [0; n-1]}$ and $\text{MSK} := \{\text{iMSK}_i\}_{i \in [0; n-1]}$.

Enc($\text{SK}_i, x_i, \mathbf{u}_i$): Run $\text{iCT}_i \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, (\mathbf{u}_i, \mathbf{t}_i))$ and output $\text{CT}_i := \text{iCT}_i$.

KeyGen($\text{MSK}, \{(f_i, \mathbf{v}_i)\}_{i \in [0; n-1]}$): Sample $\Delta \subseteq [\kappa]$ of size τ and run $\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}, f_i, (\mathbf{v}_i, \mathbf{e}_\Delta))$ for each $i \in [0; n-1]$. Output $\text{DK} := \{\text{iDK}_i\}_{i \in [0; n-1]}$.

Dec($\text{DK}_f, \{\text{DK}_i\}_{i \in [0; n-1]}, \text{CT}$): For $i \in [0; n-1]$, run $d_i \leftarrow \text{iDec}(\text{iDK}_i, \text{iCT}_i)$ and output $d = \sum_{i \in [0; n-1]} d_i$.

A detailed description of the AB-MIFE can be found in Section A. Applying the same modification as in the pairing-based case yields our MC-ABE for conjunction policies from LWE.

3 Preliminaries

3.1 Notational Conventions

Let $\lambda \in \mathbb{N}$ be the security parameter. Except in the definitions, we will suppress λ in subscripts for brevity. A nonnegative function $\varepsilon: \mathbb{N} \rightarrow \mathbb{R}$ is negligible if $\varepsilon(\lambda) = O(\lambda^{-n})$ for all $n \in \mathbb{N}$. An algorithm is said to be *efficient* if it runs in probabilistic polynomial time (PPT) in the security parameter.

To avoid confusion, we always write vectors $\mathbf{v}$ and matrices $\mathbf{A}$ in boldface and use uppercase letters for the latter. Scalars s are written in italics. Unless otherwise stated, all vectors $\mathbf{v}$ are viewed as row vectors. The corresponding column vector is denoted by $\mathbf{v}^\top$.

Security Experiments and Distributions. Let $\mathbf{Exp}$ be an interactive *experiment* that interacts with an algorithm $\mathcal{A}$ (called the *adversary*), depends on the security parameter λ and has binary outcome. We also refer to such objects as *games* or *hybrids*. We let “ $\mathbf{Exp}_{\mathcal{A}}(1^\lambda) \rightarrow 1$ ” denote the event that the outcome of running $\mathbf{Exp}$ with $\mathcal{A}$ on security parameter λ is 1. For two experiments $\mathbf{Exp}^0$ and $\mathbf{Exp}^1$, we define the distinguishing advantage of $\mathcal{A}$ against the tuple $(\mathbf{Exp}^0, \mathbf{Exp}^1)$ as

$$\text{Adv}_{\mathbf{Exp}^0, \mathbf{Exp}^1, \mathcal{A}}(\lambda) := |\Pr[\mathbf{Exp}^1_{\mathcal{A}}(1^\lambda) \rightarrow 1] - \Pr[\mathbf{Exp}^0_{\mathcal{A}}(1^\lambda) \rightarrow 1]|.$$

We write $\mathbf{Exp}^0 \approx_c \mathbf{Exp}^1$ if the experiments are *computationally indistinguishable*, i.e., their distinguishing advantage is negligible for all efficient adversaries $\mathcal{A}$. We write $\mathbf{Exp}^0 \approx_s \mathbf{Exp}^1$ if the experiments are *statistically indistinguishable*, i.e., their distinguishing advantage is negligible for all (even unbounded) adversaries. We write $\mathbf{Exp}^0 \equiv \mathbf{Exp}^1$ if the experiments are *identically distributed*, i.e., their distinguishing advantage is 0 for all (even unbounded) adversaries. By default, the term *indistinguishable* refers to computational indistinguishability.

More general, the same notations can be used for sequences of distributions. Let $D^0 = \{D_\lambda^0\}_{\lambda \in \mathbb{N}}$ and $D^1 = \{D_\lambda^1\}_{\lambda \in \mathbb{N}}$ be two sequences of distributions. For $b \in \{0, 1\}$, we define $\mathbf{Exp}^b_{\mathcal{A}}(1^\lambda)$ as follows: sample $x \xleftarrow{\$} D_\lambda^b$, run $\mathcal{A}(1^\lambda, x)$ and use the output of $\mathcal{A}$ as the outcome of the experiment. Then we write $D^0 \approx_c D^1$ (resp. $D^0 \approx_s D^1$, $D^0 \equiv D^1$) if $\mathbf{Exp}^0_{\mathcal{A}} \approx_c \mathbf{Exp}^1_{\mathcal{A}}$ (resp. $\mathbf{Exp}^0_{\mathcal{A}} \approx_s \mathbf{Exp}^1_{\mathcal{A}}$, $\mathbf{Exp}^0_{\mathcal{A}} \equiv \mathbf{Exp}^1_{\mathcal{A}}$).

Sets and Indexing. We denote by $\mathbb{Z}$ and $\mathbb{N}$ the sets of integers and natural numbers (positive integers). For integers m and n , we write $[m; n]$ to denote the set $\{z \in \mathbb{Z} : m \leq z \leq n\}$ and let $[n] := [1; n]$. For a prime number p , $\mathbb{Z}_p$ denotes the finite field of integers modulo p . For a finite set S , we let 2^S denote the power set of S .

To index a vector or the columns of a matrix, we write $\mathbf{v}[i]$ and $\mathbf{A}[j]$. In contrast, objects of some collection that is not regarded as a vector or matrix are indexed using subscripts (or superscripts in some cases). For instance, $\mathbf{v}_i$ represents a vector, not a component of some vector. If i runs through some index set $[n]$, it means that there are n vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$. If the n objects are scalars (or not explicitly vectors), we will write $v_1, \dots, v_n$ instead.

For convenience, objects might be indexed by arbitrary sets, not just integers. For finite sets $\mathfrak{s}, A$, we write $A^\mathfrak{s} := \{(\mathbf{v}[i])_{i \in \mathfrak{s}} : \mathbf{v}[i] \in A\}$ for the set of vectors whose entries are in A and indexed by $\mathfrak{s}$, e.g., $\mathbb{Z}_p^{[n]}$ is just $\mathbb{Z}_p^n$. Suppose $\mathfrak{s}_1, \mathfrak{s}_2$ are two index sets with $\mathfrak{s}_1 \subseteq \mathfrak{s}_2$. For a vector $\mathbf{v} \in \mathbb{Z}_p^{\mathfrak{s}_2}$, we denote by $\mathbf{u} = \mathbf{v}|_{\mathfrak{s}_1}$ its canonical projection onto $\mathbb{Z}_p^{\mathfrak{s}_1}$, i.e., $\mathbf{u} \in \mathbb{Z}_p^{\mathfrak{s}_1}$ and $\mathbf{u}[i] = \mathbf{v}[i]$ for all $i \in \mathfrak{s}_1$. Conversely, for any vector $\mathbf{u} \in \mathbb{Z}_p^{\mathfrak{s}_1}$, we write $\mathbf{v} = \mathbf{u}^{\mathfrak{s}_2}$ for its zero-extension into $\mathbb{Z}_p^{\mathfrak{s}_2}$, i.e., $\mathbf{v} \in \mathbb{Z}_p^{\mathfrak{s}_2}$ and $\mathbf{v}[i] = \mathbf{u}[i]$ if $i \in \mathfrak{s}_1$, and $\mathbf{v}[i] = 0$ if $i \in \mathfrak{s}_2 \setminus \mathfrak{s}_1$.

3.2 Computational Models

Arithmetic Branching Programs (ABP). An ABP $f: \mathbb{Z}_p^n \rightarrow \mathbb{Z}_p$ is defined by a tuple $(V, E, s, t, p, n, \sigma)$ consisting of a directed acyclic graph (V, E) with two distinguished vertices $s, t \in V$, a prime p , an arity n and a labelling function $\sigma: E \rightarrow \mathcal{F}^{\text{aff}}$, where $\mathcal{F}^{\text{aff}}$ contains all affine functions $g: \mathbb{Z}_p^n \rightarrow \mathbb{Z}_p$. Let P be the set of all paths from s to t . The output of the ABP on input $\mathbf{x} \in \mathbb{Z}_p^n$ is defined as $f(\mathbf{x}) = \sum_{p \in P} \prod_{e \in p} \sigma(e)(\mathbf{x})$. We refer to $|V| + |E|$ as the size of f and denote by $\mathcal{F}_n^{\text{abp}}$ the class of all functions $f: \mathbb{Z}_p^n \rightarrow \mathbb{Z}_p$ that can be computed by an ABP.

Bounded-Depth (Arithmetic) Circuits. We denote by $\mathcal{F}_{n,d}^{\text{circ}}$ the class of functions $f: \{0, 1\}^n \rightarrow \{0, 1\}$ such that there exists a binary circuit C (using only NAND gates) of depth no more than d satisfying $C(\mathbf{x}) = f(\mathbf{x})$ for all $\mathbf{x} \in \{0, 1\}^n$.

3.3 Pairing Groups

Our constructions use a sequence of pairing groups

$$\mathbb{G} = \{\mathbb{G}_\lambda = (\mathbb{G}_{\lambda,1}, \mathbb{G}_{\lambda,2}, \mathbb{G}_{\lambda,t}, g_{\lambda,1}, g_{\lambda,2}, g_{\lambda,t}, e_\lambda, p_\lambda)\}_{\lambda \in \mathbb{N}},$$

where $\mathbb{G}_{\lambda,1}$ (resp. $\mathbb{G}_{\lambda,2}, \mathbb{G}_{\lambda,t}$) is a cyclic group of order p_λ generated by $g_{\lambda,1}$ (resp. $g_{\lambda,2}, g_{\lambda,t}$), and $e_\lambda: \mathbb{G}_{\lambda,1} \times \mathbb{G}_{\lambda,2} \rightarrow \mathbb{G}_{\lambda,t}$ is the pairing operation satisfying $e_\lambda(g_{\lambda,1}^a, g_{\lambda,2}^b) = g_{\lambda,t}^{ab}$ for all integers a, b . The group operations and the pairing map are required to be efficiently computable. Following the implicit notation in [EHK⁺13], we write $[a]_i$ to denote $g_{\lambda,i}^a$ for $i \in \{1, 2, t\}$. This notation extends component-wise to matrices and vectors having entries in $\mathbb{Z}_p$. Equipped with these notations, group operations are written additively and the pairing operation multiplicatively, e.g., $[\mathbf{A}]_1 - [\mathbf{B}][\mathbf{C}]_1 \mathbf{D} = [\mathbf{A} - \mathbf{BCD}]_1$ and $[\mathbf{A}]_1 [\mathbf{B}]_2 = [\mathbf{AB}]_t$. We rely on the MDDH assumption defined in [EHK⁺13].

Definition 4 (MDDH Assumption). Let $\alpha \in \{1, 2, t\}$, $k \geq 1$ an integer constant and $\ell = \ell(\lambda), q = q(\lambda)$ some polynomials. For a sequence of pairing groups $\mathbb{G}$, the $\text{MDDH}_{k,\ell}^q$ assumption holds in $\mathbb{G}_\alpha$ if

$$\{(1^\lambda, [\mathbf{A}]_\alpha, [\mathbf{AS}]_\alpha)\}_{\lambda \in \mathbb{N}} \approx_c \{(1^\lambda, [\mathbf{A}]_\alpha, [\mathbf{U}]_\alpha)\}_{\lambda \in \mathbb{N}}$$

for $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_{p_\lambda}^{\ell \times k}$, $\mathbf{S} \xleftarrow{\$} \mathbb{Z}_{p_\lambda}^{k \times q}$ and $\mathbf{U} \xleftarrow{\$} \mathbb{Z}_{p_\lambda}^{\ell \times q}$.

In [EHK⁺13], it was shown that k -Lin implies $\text{MDDH}_{k,k+1}^1$ which in turn implies $\text{MDDH}_{k,\ell}^q$ for all $\ell > k$ and $q \geq 1$ with a tight security reduction. For convenience, we might refer to the $\text{MDDH}_{k,k+1}^1$ assumption by MDDH_k .

3.4 Lattices

Let $n, m \geq 1$ and $p \geq 2$ be integers. Given $\mathbf{A} \in \mathbb{Z}_p^{n \times m}$ and $\mathbf{b} \in \mathbb{Z}_p^n$ such that there exists $\mathbf{k}^* \in \mathbb{Z}^m$ satisfying $\mathbf{A}\mathbf{k}^* = \mathbf{b}$, we define the lattice coset

$$\Lambda_{\mathbf{b}}^\perp(\mathbf{A}) = \{\mathbf{k} \in \mathbb{Z}^m : \mathbf{A}\mathbf{k} = \mathbf{b}\} = \mathbf{k}^* + \Lambda_0^\perp(\mathbf{A})$$

Let Λ' be any lattice coset and $\sigma \geq 0$. We write $\mathcal{D}_{\Lambda', \sigma}$ to denote the discrete Gaussian distribution [MP12] over Λ' with width σ . For $B > 0$, we let $\mathcal{D}_{\mathbb{Z}, \sigma, \leq B}$ be the truncated $\mathcal{D}_{\mathbb{Z}, \sigma}$ distribution which is sampled by first sampling $x \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma}$, then returning x if $|x| \leq B$, and 0 otherwise. We will use the following lemmas.

Lemma 5 (Truncation). *For all $\sigma \geq 1$ and $\varepsilon \geq 2$,*

$$\Pr[|x| > \sigma\sqrt{\varepsilon} : x \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma}] \leq 2^{-\varepsilon},$$

so $\mathcal{D}_{\mathbb{Z}, \sigma, \leq \sigma\sqrt{\varepsilon}}$ is $2^{-\varepsilon}$ -close to $\mathcal{D}_{\mathbb{Z}, \sigma}$.

Lemma 6 (Noise Flooding). *For all $y \in \mathbb{Z}$ and $\sigma \geq 2^{\varepsilon+6}y$, it holds that $(y + \mathcal{D}_{\mathbb{Z}, \sigma})$ is $2^{-\varepsilon}$ -close to $\mathcal{D}_{\mathbb{Z}, \sigma}$.*

We recall the LWE assumption introduced in [Reg05].

Definition 7 (LWE Assumption). Let $n = n(\lambda), m = m(\lambda) \leq \text{poly}(\lambda)$, $p = p(\lambda)$, $\sigma = \sigma(\lambda)$. The $\text{LWE}_{n, m, p, \sigma}$ assumption states that

$$\{(1^\lambda, \mathbf{A}, \mathbf{s}\mathbf{A} + \mathbf{e})\}_{\lambda \in \mathbb{N}} \approx_c \{(1^\lambda, \mathbf{A}, \mathbf{u})\}_{\lambda \in \mathbb{N}},$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_p^{n \times m}$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^n$, $\mathbf{e} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}, \sigma}^m$ and $\mathbf{u} \xleftarrow{\$} \mathbb{Z}_p^m$.

3.5 Exact Linear Garbling

We recall the definition of a linear secret sharing (LSS) scheme [BL90, Bei96]. Let $m, n \in \mathbb{N}$ and p be a prime number. We denote $\mathbf{e}_1 = (1, 0, \dots, 0) \in \mathbb{Z}_p^m$ the first unit-vector in $\mathbb{Z}_p^m$. A LSS scheme over $\mathbb{Z}_p$ for a function $f: \{0, 1\}^n \rightarrow \{0, 1\}$ is specified by a share generating matrix $\mathbf{M} \in \mathbb{Z}_p^{m \times n}$ which satisfies the following condition for all inputs $\mathbf{x} \in \{0, 1\}^n$:⁷

$$f(\mathbf{x}) = 0 \iff \mathbf{e}_1^\top \in \text{span}\{\mathbf{M}[i] : \mathbf{x}[i] = 1\}. \quad (1)$$

Lewko and Waters [LW11] presented an LSS scheme for all monotone access structures. We will use this fact in the following way.

Lemma 8 (Exact Linear Garbling for NC¹). *There exist four efficient algorithms (Gen, Share, Rec, Sim) with the following properties.*

- **Syntax.** On input $f \in \text{NC}^1$, $f: \{0, 1\}^n \rightarrow \{0, 1\}$, Gen(f) outputs $\mathbf{M} \in \mathbb{Z}_p^{m \times 2n}$ and

$$\text{Share}(f, \mathbf{s}; \mathbf{t}) = (\mathbf{s}, \mathbf{t}) \cdot (\mathbf{M} \otimes \mathbf{I}_k)$$

$$\text{Sim}(f; \mathbf{t}) = (\mathbf{0}, \mathbf{t}) \cdot (\mathbf{M} \otimes \mathbf{I}_k),$$

where $\mathbf{s} \in \mathbb{Z}_p^k$, $\mathbf{t} \in \mathbb{Z}_p^{k(m-1)}$ and m is linear in the size of f .

- **Correctness.** For all $\mathbf{x} \in \{0, 1\}^n$ with $f(\mathbf{x}) = 0$, Rec($f, \mathbf{x}$) outputs a vector $\mathbf{d}_{f, \mathbf{x}} \in \mathbb{Z}_p^n$ such that for all $\mathbf{r}, \mathbf{s} \in \mathbb{Z}_p^k$ and $\mathbf{t} \in \mathbb{Z}_p^{k(m-1)}$,

$$\mathbf{p}_{f, \mathbf{x}, \mathbf{r}, \mathbf{s}, \mathbf{t}} \cdot \mathbf{d}_{f, \mathbf{x}}^\top = \mathbf{s} \cdot \mathbf{r}^\top,$$

where we parse $(\mathbf{s}_1^0, \dots, \mathbf{s}_n^0, \mathbf{s}_1^1, \dots, \mathbf{s}_n^1) \leftarrow \text{Share}(f, \mathbf{s})$ ⁸ such that $\mathbf{s}_i^\beta \in \mathbb{Z}_p^k$ for $i \in [n]$ and $\beta \in \{0, 1\}$, and $\mathbf{p}_{f, \mathbf{x}, \mathbf{r}, \mathbf{s}, \mathbf{t}} = (\mathbf{s}_1^{\mathbf{x}[1]}, \dots, \mathbf{s}_n^{\mathbf{x}[n]}) \cdot (\mathbf{I}_n \otimes \mathbf{r}^\top)$. If $f(\mathbf{x}) = 1$, then Rec($f, \mathbf{x}$) outputs $\perp$.

⁷For notational convenience, we consider $\mathbf{x} \in \{0, 1\}^n$ with coordinates being indexed by elements in $[n]$. But this is w.l.o.g., and we may use arbitrary index sets $\mathcal{S}$.

⁸For notational convenience, we may also parse the output of Share($f, \mathbf{s}$) as a set $\{\mathbf{s}_i^\beta\}_{i \in [n], \beta \in \{0, 1\}}$.

- **Security.** For all $f \in \text{NC}^1$, $\mathbf{s} \in \mathbb{Z}_p^k$ and $\mathbf{x} \in \{0, 1\}^n$, if $f(\mathbf{x}) = 1$ then

$$\begin{aligned} & \{(\mathbf{s}_1^{x[1]}, \dots, \mathbf{s}_n^{x[n]}) : (\mathbf{s}_1^0, \dots, \mathbf{s}_n^0, \mathbf{s}_1^1, \dots, \mathbf{s}_n^1) \leftarrow \text{Share}(f, \mathbf{s}; \mathbf{t})\} \\ & \approx_s \{(\mathbf{s}_1^{x[1]}, \dots, \mathbf{s}_n^{x[n]}) : (\mathbf{s}_1^0, \dots, \mathbf{s}_n^0, \mathbf{s}_1^1, \dots, \mathbf{s}_n^1) \leftarrow \text{Sim}(f, \mathbf{t})\} , \end{aligned}$$

where the randomness is taken over $\mathbf{t} \xleftarrow{\$} \mathbb{Z}_p^{k(m-1)}$.

Proof. Given a policy $f: \{0, 1\}^n \rightarrow \{0, 1\}$ computable by an NC^1 circuit, we construct the corresponding monotone policy $f_{\text{mon}}: \{0, 1\}^{2n} \rightarrow \{0, 1\}$ as follows: first, we view f as a Boolean formula consisting of (fan-in 1) $\neg$ gates and (fan-in 2) $\wedge$ and $\vee$ gates. Then, using De Morgan laws, we push the $\neg$ gates to the leaves such that all internal nodes consist only of $\wedge$ and $\vee$ gates, while leaves are labeled by either attributes or their negations. In other words, we can transform each $f \in \text{NC}^1$ over $\{0, 1\}^n$ into a function $f_{\text{mon}}: \{0, 1\}^{2n} \rightarrow \{0, 1\}$ described by a monotone Boolean formula over the same attributes and their negations, i.e., we have $f(\mathbf{x}) = f_{\text{mon}}(\mathbf{x}_{\text{mon}})$ for all $\mathbf{x} \in \{0, 1\}^n$ and $\mathbf{x}_{\text{mon}} = (\mathbf{x}, 1^k - \mathbf{x}) \in \{0, 1\}^{2n}$.

Correctness. Using an LSS scheme for monotone formulas, the implementation of Gen and Rec is immediate.

- Given $f \in \text{NC}^1$, the algorithm $\text{Gen}(f)$ constructs f_{mon} and outputs a share generating matrix $\mathbf{M} \in \mathbb{Z}_p^{m \times 2n}$ for f_{mon} .
- Given $f \in \text{NC}^1$ and $x \in \{0, 1\}^n$ such that $f(\mathbf{x}) = 0$, $\text{Rec}(f, x)$ outputs a coefficient vector $\mathbf{d}_{f, \mathbf{x}} \in \mathbb{Z}_p^n$ such that $\sum_{i \in [n]} \mathbf{d}_{f, \mathbf{x}}[i] \cdot \mathbf{M}[i + n \cdot \mathbf{x}[i]] = \mathbf{e}_1^\top$. The existence of such a vector is guaranteed by Equality (1).

Then we have

$$\begin{aligned} \mathbf{p}_{f, \mathbf{x}, \mathbf{r}, \mathbf{s}, \mathbf{t}} \cdot \mathbf{d}_{f, \mathbf{x}}^\top &= (\mathbf{s}_1^{x[1]}, \dots, \mathbf{s}_n^{x[n]}) \cdot (\mathbf{I}_n \otimes \mathbf{r}^\top) \cdot \mathbf{d}_{f, \mathbf{x}} \\ &= (\mathbf{s}, \mathbf{t}) (\mathbf{I}_m \otimes \mathbf{r}^\top) \cdot \sum_{i \in [n]} \mathbf{d}_{f, \mathbf{x}} \cdot \mathbf{M}[i + n \cdot \mathbf{x}[i]] \\ &= (\mathbf{s}, \mathbf{t}) (\mathbf{I}_m \otimes \mathbf{r}^\top) \cdot \mathbf{e}_1^\top = \mathbf{s} \cdot \mathbf{r}^\top . \end{aligned}$$

Security. Consider $f \in \text{NC}^1$ and $\mathbf{x} \in \{0, 1\}^n$ such that $f(\mathbf{x}) = 1$. Let $M_{\mathbf{x}} = \text{span}\{\mathbf{M}[i + n \cdot \mathbf{x}[i]]\}_{i \in [n]}$ be the vector space spanned by the columns of $\mathbf{M}$ corresponding to the shares $\{\mathbf{s}_i^{x[i]}\}_{i \in [n]}$. Furthermore, let $M_{\mathbf{x}}^\perp$ denote the orthogonal complement of $M_{\mathbf{x}}$ and $A_{\mathbf{x}}^\perp$ the affine subspace of $M_{\mathbf{x}}^\perp$ containing all vectors whose first coordinate is 1. Since $f(\mathbf{x}) = 1$, Equality (1) gives that $\mathbf{e}_1 \notin M_{\mathbf{x}}$ which implies that $A_{\mathbf{x}}^\perp$ is nonempty, i.e., there exists $(1, \mathbf{u}) \in A_{\mathbf{x}}^\perp$. For every $\mathbf{s} \in \mathbb{Z}_p^k$ and random $\mathbf{t} \in \mathbb{Z}_p^{k(m-1)}$, we have

$$\begin{aligned} & \{(\mathbf{s}, \mathbf{t}) \cdot (\mathbf{M}[i + n \cdot \mathbf{x}[i]] \otimes \mathbf{I}_k)\}_{i \in [n]} \\ & \equiv \{(\mathbf{s}, \mathbf{t} + \mathbf{u} \otimes \mathbf{s}) \cdot (\mathbf{M}[i + n \cdot \mathbf{x}[i]] \otimes \mathbf{I}_k)\}_{i \in [n]} \\ & = \{((\mathbf{0}, \mathbf{t}) + (1, \mathbf{u}) \otimes \mathbf{s}) \cdot (\mathbf{M}[i + n \cdot \mathbf{x}[i]] \otimes \mathbf{I}_k)\}_{i \in [n]} \\ & = \{(\mathbf{0}, \mathbf{t}) \cdot (\mathbf{M}[i + n \cdot \mathbf{x}[i]] \otimes \mathbf{I}_k) + ((1, \mathbf{u}) \cdot (\mathbf{M}[i + n \cdot \mathbf{x}[i]])) \otimes \mathbf{s}\}_{i \in [n]} \\ & = \{(\mathbf{0}, \mathbf{t}) \cdot (\mathbf{M}[i + n \cdot \mathbf{x}[i]] \otimes \mathbf{I}_k)\}_{i \in [n]} , \end{aligned}$$

where the first distribution corresponds to the output of Share whereas the last distribution corresponds to the output of Sim. The first equivalence follows from the fact that $\mathbf{t} \xleftarrow{\$} \mathbb{Z}_p^{k(m-1)}$ perfectly masks $\mathbf{u} \otimes \mathbf{s}$, and the last equality since $(1, \mathbf{u}) \in M_{\mathbf{x}}^\perp$ and $\mathbf{M}[i + n \cdot \mathbf{x}[i]] \in M_{\mathbf{x}}$. $\square$

3.6 Noisy Linear Garbling

We recall the definition of noisy linear garblings from [HLL24] in the variant for binary inputs.

Definition 9 (Noisy Linear Garbling). Let $\{p_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of prime numbers and function families, respectively, where each $f \in \mathcal{F}_\lambda: \{0, 1\}^n \rightarrow \{0, 1\}$. A noisy linear garbling for $\mathcal{F}$ consists of the following three efficient algorithms.

Setup(1^λ) $\rightarrow$ PP: On input the security parameter 1^λ , this algorithm outputs some public parameters PP.

GenF(PP, f) $\rightarrow$ (1^{n_f} , $\mathbf{w}_{\text{out}}$, $\{\mathbf{W}_i^0, \mathbf{W}_i^\times\}_{i \in [n]}$, $\mathbf{T}, \mathbf{R}$): On input public parameters PP and a function $f \in \mathcal{F}_\lambda$, this algorithm outputs a deterministic dimension 1^{n_f} of garbling randomness, a secret vector $\mathbf{w}_{\text{out}}$, label functions $\{\mathbf{W}_{i,j}^0, \mathbf{W}_{i,j}^\times\}_{i \in [n]}$, a garbled table $\mathbf{T}$, and some reusable information $\mathbf{R}$. Here, $\mathbf{w}_{\text{out}}$, $\{\mathbf{W}_{i,j}^\times\}_{i \in [n]}$, and $\mathbf{T}$ are n_f -row matrices over $\mathbb{Z}$ and their shapes, including n_f , are fully determined by PP and f .

Eval(PP, $f, \mathbf{R}, \mathbf{x}, \{\mathbf{w}_i\}_{i \in [n]}, \mathbf{t}$) $\rightarrow w_{\text{out}}$: On input the public parameters, the function f , reusable information $\mathbf{R}$, input $\mathbf{x} \in \{0, 1\}^n$, a set of randomized labels $\{\mathbf{w}_i\}_{i \in [n]}$, and the randomized garbled table, this algorithm outputs an approximation of the randomized secret w_{out} if $f(\mathbf{x}) = 0$. Here, w_{out} , $\{\mathbf{w}_i\}_{i \in [n]}$ and $\mathbf{t}$ are over $\mathbb{Z}_{p_\lambda}$.

Correctness. Let $B_{\text{in}} = B_{\text{in}}(\lambda)$, $B_{\text{out}} = B_{\text{out}}(\lambda) \in \mathbb{N}$. A noisy linear garbling is $(B_{\text{in}}, B_{\text{out}})$ -correct if for all $\lambda \in \mathbb{N}$, functions $f \in \mathcal{F}_\lambda$, inputs $\mathbf{x} \in \{0, 1\}^n$ such that $f(\mathbf{x}) = 0$, and error vectors $e_{\text{out}}, \mathbf{e}_1, \dots, \mathbf{e}_n, \mathbf{e}_t \in [-B_{\text{in}}; B_{\text{in}}]^\star$ (of suitable dimensions), it holds that

$$\Pr \left[e \in [-B_{\text{out}}; B_{\text{out}}] \mid \begin{array}{l} \mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^{n_f}; \text{PP} \leftarrow \text{Setup}(1^\lambda) \\ (1^{n_f}, \mathbf{w}_{\text{out}}, \{\mathbf{W}_{i,j}^0, \mathbf{W}_{i,j}^\times\}_{i \in [n]}, \mathbf{T}, \mathbf{R}) \leftarrow \text{GenF}(\text{PP}, f) \\ w_{\text{out}} \leftarrow \text{Eval}(\text{PP}, f, \mathbf{R}, \mathbf{x}, \{\mathbf{s}(\mathbf{W}_i^0 + \mathbf{x}[i]\mathbf{W}_i^\times) + \mathbf{e}_i\}_{i \in [n]}, \mathbf{s}\mathbf{T} + \mathbf{e}_t) \\ e := w_{\text{out}} - (\mathbf{s}\mathbf{w}_{\text{out}}^\top + e_{\text{out}}) \end{array} \right] = 1 .$$

Shortness. Let $B_{\text{short}} = B_{\text{short}}(\lambda) \in \mathbb{N}$. A noisy linear garbling scheme is B_{short} -short if for all $\lambda \in \mathbb{N}$, functions $f \in \mathcal{F}_\lambda$ and inputs $\mathbf{x} \in \{0, 1\}^n$ such that $f(\mathbf{x}) = 1$, it holds that

$$\Pr \left[\begin{array}{l} \|\mathbf{w}_{\text{out}}\|, \|\mathbf{T}\| \leq B_{\text{short}}, \\ \|\mathbf{W}_i^0 + \mathbf{x}[i]\mathbf{W}_i^\times\| \leq B_{\text{short}} \text{ for all } i \in [n] \end{array} \mid \begin{array}{l} \text{PP} \leftarrow \text{Setup}(1^\lambda) \\ (1^{n_f}, \mathbf{w}_{\text{out}}, \{\mathbf{W}_{i,j}^0, \mathbf{W}_{i,j}^\times\}_{i \in [n]}, \mathbf{T}, \mathbf{R}) \leftarrow \text{GenF}(\text{PP}, f) \end{array} \right] = 1 .$$

Security. Security requires that the randomized secret, labels and garbled table are pseudorandom if $f(\mathbf{x}) = 1$.

Definition 10 (Security). A noisy linear garbling scheme NLG is GenNoise-secure, where GenNoise is an efficient algorithm with suitable input/output formats, if $\text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-real}}(1^\lambda) \approx_c \text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-sim}}(1^\lambda)$ for all PPT adversaries $\mathcal{A}$, where $\text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-real}}$ and $\text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-sim}}$ proceeds as follows.

- **Setup.** Launch $\mathcal{A}(1^\lambda)$, sample r_{Setup} , run $\text{PP} \leftarrow \text{Setup}(1^\lambda; r_{\text{Setup}})$ and send r_{Setup} to $\mathcal{A}$.
- **Challenge.** Upon $\mathcal{A}$ submitting a function $f \in \mathcal{F}_\lambda$ and an input $\mathbf{x} \in \{0, 1\}^n$, sample r_{GenF} and run

$$(1^{n_f}, \mathbf{w}_{\text{out}}, \{\mathbf{W}_i^0, \mathbf{W}_i^\times\}_{i \in [n]}, \mathbf{T}, \mathbf{R}) \leftarrow \text{GenF}(\text{PP}, f; r_{\text{GenF}}) .$$

Sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^{n_f}$, run

$$(e_{\text{out}}, \mathbf{e}_1, \dots, \mathbf{e}_n, \mathbf{e}_t) \leftarrow \text{GenNoise}(\text{PP}, f, \mathbf{w}_{\text{out}}, \{\mathbf{W}_i^0, \mathbf{W}_i^\times\}_{i \in [n]}, \mathbf{T}, \mathbf{R}, \mathbf{x}) ,$$

and compute

$$\begin{array}{lll} \text{in } \text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-real}} : & w_{\text{out}} \leftarrow \mathbf{s}\mathbf{w}_{\text{out}}^\top, & \mathbf{t} \leftarrow \mathbf{s}\mathbf{T} + \mathbf{e}_t, & \{\mathbf{w}_i \leftarrow \mathbf{s}(\mathbf{W}_i^0 + \mathbf{x}[i]\mathbf{W}_i^\times) + \mathbf{e}_i\}_{i \in [n]}, \\ \text{in } \text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-sim}} : & w_{\text{out}} \xleftarrow{\$} \mathbb{Z}_p, & \mathbf{t} \xleftarrow{\$} \mathbb{Z}_p^\star, & \{\mathbf{w}_i \xleftarrow{\$} \mathbb{Z}_p^\star\}_{i \in [n]}, \end{array}$$

where $\mathbf{t}$ and $\{\mathbf{w}_i\}_{i \in [n]}$ in $\text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-sim}}$ are of the same dimensions as their counterparts in $\text{Exp}_{\text{NLG}, \mathcal{A}}^{\text{nlg-real}}$. Return $(r_{\text{GenF}}, w_{\text{out}}, \{\mathbf{w}_i\}_{i \in [n]}, \mathbf{t})$ to $\mathcal{A}$.

- **Guess.** The adversary outputs a guess $\alpha \in \{\text{real}, \text{sim}\}$. The outcome of the experiment is α if $f(\mathbf{x}) = 1$. Otherwise, the outcome is set to 0.

3.7 Multi-Client Attribute-Based Encryption

We recall the definition of MC-ABE. Recent papers [Ayy22, FFMV23, ARYY23, PS24] consider a varying syntax for MI-ABE and MC-ABE. Our definition follows [ARYY23, PS24].

Public-Key Setting. We first discuss the public-key case.

Definition 11 (Public-Key Syntax). Let $n = n(\lambda)$ be a polynomial and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$, $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$, $P = \{P_\lambda\}_{\lambda \in \mathbb{N}}$, $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$, and $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of message spaces, label spaces, predicates, ciphertext attribute spaces and key attribute spaces, respectively, where $P_\lambda: \mathcal{X}_\lambda^{[0;n-1]} \times \mathcal{Y}_\lambda \rightarrow S_\lambda$ and S_λ is a strict superset of $\{0\}$.

An MC-ABE scheme for $\mathcal{M}$, $\mathcal{L}$, and P consists of five efficient algorithms:

$\text{Setup}(1^\lambda) \rightarrow (\text{MPK}, \text{MSK}, \{\text{SK}_i\}_{i \in [n-1]})$: On input the security parameter 1^λ , this algorithm outputs a pair of master public key MPK and master secret key MSK as well as a set of secret keys $\{\text{SK}_i\}_{i \in [n-1]}$.

$\text{Enc}(\text{MPK}, L, x_0, \mu) \rightarrow \text{CT}_L$: On input the master public key MPK, a label $L \in \mathcal{L}_\lambda$, an attribute $x_0 \in \mathcal{X}_\lambda$ and a message $\mu \in \mathcal{M}_\lambda$, this algorithm outputs a ciphertext CT_L . We assume that CT_L implicitly includes x_0 .

$\text{AKeyGen}(\text{SK}_i, L, x_i) \rightarrow \text{DK}_{L,i}$: On input a secret key SK_i for some $i \in [n-1]$, a label $L \in \mathcal{L}_\lambda$ and an attribute $x_i \in \mathcal{X}_\lambda$, this algorithm outputs a decryption key $\text{DK}_{L,i}$. We assume that $\text{DK}_{L,i}$ implicitly includes x_i .

$\text{PKeyGen}(\text{MSK}, y) \rightarrow \text{DK}_y$: On input the master secret key MSK and an attribute $y \in \mathcal{Y}_\lambda$, this algorithm outputs a decryption key DK_y . We assume that DK_y implicitly includes a description of y .

$\text{Dec}(\text{DK}_y, \{\text{DK}_{L,i}\}_{i \in [n-1]}, \text{CT}_L) \rightarrow \mu' \vee \perp$: On input a decryption key DK_y for a policy $y \in \mathcal{Y}_\lambda$, a set of attribute decryption keys $\{\text{DK}_{L,i}\}_{i \in [n-1]}$ generated with respect to some label $L \in \mathcal{L}_\lambda$ and a ciphertext CT_L created with respect to the same label L , this algorithm outputs an element $\mu' \in \mathcal{M}_\lambda$ or $\perp$.

An MC-ABE scheme is *correct* if for every $\lambda, n \in \mathbb{N}$, label $L \in \mathcal{L}_\lambda$, message $\mu \in \mathcal{M}_\lambda$ and attributes $x_0, \dots, x_{n-1} \in \mathcal{X}_\lambda$, $y \in \mathcal{Y}_\lambda$ such that $P_\lambda(x_0, \dots, x_{n-1}, y) = 1$, it holds that

$$\Pr \left[\begin{array}{l} (\text{MPK}, \text{MSK}, \{\text{SK}_i\}_{i \in [n-1]}) \leftarrow \text{Setup}(1^\lambda) \\ \text{CT}_L \leftarrow \text{Enc}(\text{MPK}, L, x_0, \mu) \\ \forall i \in [n-1]: \text{DK}_{L,i} \leftarrow \text{AKeyGen}(\text{SK}_i, L, x_i) \\ \text{DK}_y \leftarrow \text{PKeyGen}(\text{MSK}, y) \\ \mu' := \text{Dec}(\text{DK}_y, \{\text{DK}_{L,i}\}_{i \in [n-1]}, \text{CT}_L) \end{array} \right] = 1$$

We define IND-CPA security for MC-ABE in the public-key setting.

Definition 12 (Public-Key Security for MC-ABE). Let $\text{xxx} \in \{\text{adap}, \text{sel}, \text{stat}\}$. An MC-ABE scheme ABE is xxx -secure against Q collusions if $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-0}}(1^\lambda) \approx_c \text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-1}}(1^\lambda)$ for all PPT adversaries $\mathcal{A}$, where $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b}$ for $b \in \{0, 1\}$ proceeds as follows.

- **Setup.** Launch $\mathcal{A}(1^\lambda)$ to receive a set $\mathcal{C} \subseteq [n-1]$ of corrupted slots, run $(\text{MPK}, \text{MSK}, \{\text{SK}_i\}_{i \in [n-1]}) \leftarrow \text{Setup}(1^\lambda)$ and send MPK to $\mathcal{A}$. Initialize an empty set $\mathcal{Q}_{\text{akey}} = \emptyset$ and an empty multiset $\mathcal{Q}_{\text{pkey}} = \emptyset$.
- **Query I.** Repeat the following for arbitrarily many rounds determined by $\mathcal{A}$. In each round, $\mathcal{A}$ has two options.
 - $\text{QAKeyGen}(i, L, x_i)$: upon $\mathcal{A}$ submitting a slot index $i \in [n-1]$, a label $L \in \mathcal{L}_\lambda$ and an attribute $x_i \in \mathcal{X}_\lambda$, run $\text{DK}_i \leftarrow \text{AKeyGen}(\text{SK}_i, L, x_i)$ and return DK_i to $\mathcal{A}$. Add (i, L, x_i) to $\mathcal{Q}_{\text{akey}}$.
 - $\text{QPKeyGen}(y)$: upon $\mathcal{A}$ submitting an attribute $y \in \mathcal{Y}_\lambda$, run $\text{DK}_y \leftarrow \text{PKeyGen}(\text{MSK}, y)$ and return DK_y to $\mathcal{A}$. Add y to $\mathcal{Q}_{\text{pkey}}$.

- **Challenge.** Upon $\mathcal{A}$ submitting a label $L \in \mathcal{L}_\lambda$, a challenge attribute $x_0^* \in \mathcal{X}_\lambda$ and messages $\mu^0, \mu^1 \in \mathcal{M}_\lambda$, run $\text{CT}_L \leftarrow \text{Enc}(\text{MPK}, L, x_0^*, \mu^b)$ and return CT_L to $\mathcal{A}$.
- **Query II.** Same as Query I.
- **Guess.** The adversary outputs a bit $b' \in \{0, 1\}$. The outcome of the experiment is b' if $\mathcal{A}$ satisfies the following admissibility condition. Otherwise, the outcome is set to 0. The adversary $\mathcal{A}$ is admissible if the following conditions are satisfied.
 - $|\mathcal{Q}_{\text{pkey}}| \leq Q$,
 - For all $y \in \mathcal{Q}_{\text{pkey}}$ and $x_1, \dots, x_{n-1} \in \mathcal{X}_\lambda$ such that $(i, L, x_i) \in \mathcal{Q}_{\text{akey}}$ for all $i \in [n-1] \setminus \mathcal{C}$, it holds that $P_\lambda(x_0, \dots, x_{n-1}, y) \neq 0$.
 - if $\text{xxx} = \text{sel}$, then the adversary submits all queries to QakeyGen and the challenge query in one shot. Subsequently, the adversary can query QPkeyGen.
 - If $\text{xxx} = \text{stat}$, then the adversary submits all queries in one shot.

If we do not explicitly specify Q , then the adversary can submit any number of queries to QPkeyGen and we say that ABE is secure against *unbounded* collusions.

Secret-Key Setting. Some of our constructions are presented in the secret-key setting. In this case, the setup algorithm outputs a secret key SK_0 that is used for encryption instead of a master public key MPK. The security definition is the same as in the public-key setting except for the following changes.

- The Setup phase does not provide the adversary with MPK and initializes an additional set $\mathcal{Q}_{\text{enc}}$.
- The phases Query I, Challenge and Query II are replaced by a single phase Query during which the adversary has access to the following three oracles.
 - $\text{QEnc}(L, x_0, \mu^0, \mu^1)$: upon $\mathcal{A}$ submitting a label $L \in \mathcal{L}_\lambda$, an attribute $x_0 \in \mathcal{X}_\lambda$ and messages $\mu^0, \mu^1 \in \mathcal{M}_\lambda$, run $\text{CT}_L \leftarrow \text{Enc}(\text{SK}_0, L, x_0, \mu^b)$ and return CT_L to $\mathcal{A}$. Add (L, x_0, μ^0, μ^1) to $\mathcal{Q}_{\text{enc}}$.
 - $\text{QakeyGen}(i, L, x_i)$: same as in the public-key case.
 - $\text{QPkeyGen}(y)$: same as in the public-key case.
- The second item of the admissibility condition changes to: for all $y \in \mathcal{Q}_{\text{pkey}}$, $L \in \mathcal{L}_\lambda$, $(L, x_0, \mu^0, \mu^1) \in \mathcal{Q}_{\text{enc}}$ and $x_1, \dots, x_{n-1} \in \mathcal{X}_\lambda$ such that $(i, L, x_i) \in \mathcal{Q}_{\text{akey}}$ for all $i \in [n-1] \setminus \mathcal{C}$, it holds that $P_\lambda(x_0, \dots, x_{n-1}, y) \neq 0$ or $\mu^0 = \mu^1$.

Note that public-key security implies secret-key security. For this reason, we preferably present our constructions in the public-key setting and only resort to secret-key security if necessary.

Transformation to MC-PE. In contrast to MC-ABE, MC-PE security is stronger in the secret-key setting (see e.g. [Ayy22, FFMV23, PS24] for details). The work [PS24] presents a compiler that turns any constant-arity, public-key MC-ABE into a constant-arity, secret-key MC-PE. Inspecting their compiler, we observe that the security proof does not rely on the fact that the employed MC-ABE scheme satisfies public-key security. We can therefore use their compiler to also convert secret-key MC-ABE into secret-key MC-PE.

3.8 Attribute-Based Inner-Product Functional Encryption

We recall the syntax of attribute-based IPFE (AB-IPFE) and define security in the classical [ABDP15, ALS16, ACGU20, LLW21] and evasive [HLL24] setting. In this work, we consider evasive IPFE only in the variant *without* structured noises in which case we can adapt the syntax to match that of classical IPFE schemes.

Definition 13 (Syntax of AB-IPFE). Let $\{p_\lambda\}_{\lambda \in \mathbb{N}}$, $\{P_\lambda\}_{\lambda \in \mathbb{N}}$, $\{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of prime numbers, predicates, ciphertext attribute spaces and key attribute spaces, respectively. An AB-IPFE scheme consists of four efficient algorithms.

$\text{Setup}(1^\lambda, 1^N) \rightarrow (\text{MPK}, \text{MSK})$: On input the security parameter 1^λ and the dimension of the vectors 1^N , this algorithm outputs a pair of a master public and a master secret key (MPK, MSK) .

$\text{Enc}(\text{MPK}, x, \mathbf{u}) \rightarrow \text{CT}$: On input a master public key MPK , a ciphertext attribute $x \in \mathcal{X}_\lambda$ and a vector $\mathbf{u} \in \mathbb{Z}_{p_\lambda}^N$, this algorithm outputs a ciphertext CT for $\mathbf{u}$ with respect to x . We assume that CT implicitly contains x .

$\text{KeyGen}(\text{MSK}, y, \mathbf{v}) \rightarrow \text{DK}$: On input a master secret key MSK , a key attribute $y \in \mathcal{Y}_\lambda$ and a vector $\mathbf{v} \in \mathbb{Z}_{p_\lambda}^N$, this algorithm outputs a decryption key DK for $\mathbf{v}$ with respect to y . We assume that DK implicitly contains y .

$\text{Dec}(\text{DK}, \text{CT}) \rightarrow d \vee \perp$: On input a decryption key DK and a ciphertext CT , this algorithm outputs an element $d \in \mathbb{Z}_{p_\lambda}$ or the special symbol $\perp$ indicating failure of decryption.

We say that iFE is a *secret-key* AB-IPFE scheme if Setup outputs only a master secret key MSK which is passed to both Enc and KeyGen . Furthermore, we say that iFE is *based on a pairing group* $\mathbb{G} = \{\mathbb{G}_\lambda = (\mathbb{G}_{\lambda,1}, \mathbb{G}_{\lambda,2}, \mathbb{G}_{\lambda,t}, g_{\lambda,1}, g_{\lambda,2}, g_{\lambda,t}, e_\lambda, p_\lambda)\}_{\lambda \in \mathbb{N}}$ if Enc takes vectors $\llbracket \mathbf{u} \rrbracket_1$ encoded in $\mathbb{G}_{\lambda,1}$, KeyGen takes vectors $\llbracket \mathbf{v} \rrbracket_2$ encoded in $\mathbb{G}_{\lambda,2}$ and Dec outputs elements $\llbracket d \rrbracket_t \in \mathbb{G}_{\lambda,t}$.

Correctness. Let $B = B(\lambda, p_\lambda, N)$ be a function with values in $\mathbb{N}$. An AB-IPFE scheme is B -correct if for all $\lambda, N \in \mathbb{N}$, all attributes $x \in \mathcal{X}_\lambda$ and $y \in \mathcal{Y}_\lambda$ such that $P_\lambda(x, y) = 0$, and all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_{p_\lambda}^N$, it holds that

$$\Pr \left[\begin{array}{l} \text{Dec}(\text{DK}, \text{CT}) = \mathbf{u} \cdot \mathbf{v}^\top + e \text{ for some } e \in [-B; B] \\ \text{CT} \leftarrow \text{Enc}(\text{MSK}, x, \mathbf{u}) \\ \text{DK} \leftarrow \text{KeyGen}(\text{MSK}, y, \mathbf{v}) \end{array} \middle| \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^N) \end{array} \right] = 1 .$$

If we do not explicitly specify B , then it defaults to the constant zero function, i.e., perfect correctness.

Batch Notations. It is convenient to consider Enc , KeyGen and Dec in batches. Consider $x \in \mathcal{X}_\lambda$, $\mathbf{y} \in \mathcal{Y}_\lambda$ and matrices

$$\mathbf{U} = \begin{bmatrix} \text{---} & \mathbf{u}_1 & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{u}_I & \text{---} \end{bmatrix} \quad \mathbf{V} = \begin{bmatrix} \text{---} & \mathbf{v}_1 & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{v}_J & \text{---} \end{bmatrix} .$$

We write

$$\text{CT} \leftarrow \text{Enc}(\text{MPK}, x, \mathbf{U}) \quad \text{and} \quad \text{DK} \leftarrow \text{KeyGen}(\text{MSK}, y, \mathbf{V})$$

for $\text{CT} = \{\text{CT}_i\}_{i \in [I]}$ and $\text{DK} = \{\text{DK}_j\}_{j \in [J]}$, where

$$\begin{aligned} \text{CT}_i &\leftarrow \text{Enc}(\text{MPK}, x, \mathbf{u}_i) && \text{for all } i \in [I] , \\ \text{DK}_j &\leftarrow \text{KeyGen}(\text{MSK}, y, \mathbf{v}_j) && \text{for all } j \in [J] , \end{aligned}$$

and $\text{Dec}(\text{DK}, \text{CT})$ refers to the matrix

$$\begin{bmatrix} \text{Dec}(\text{DK}_1, \text{CT}_1) & \cdots & \text{Dec}(\text{DK}_J, \text{CT}_1) \\ \vdots & \ddots & \vdots \\ \text{Dec}(\text{DK}_1, \text{CT}_I) & \cdots & \text{Dec}(\text{DK}_J, \text{CT}_I) \end{bmatrix}$$

which is (approximately) $\mathbf{U} \cdot \mathbf{V}^\top$ if $P_\lambda(x, y) = 0$.

Concrete Predicates. We consider AB-IPFE for three concrete predicates.

- **Identity-Based IPFE (IB-IPFE).** Let $\mathcal{I} = \{\mathcal{I}_\lambda\}_{\lambda \in \mathbb{N}}$ be a sequence of identity spaces. For each $\lambda \in \mathbb{N}$, we let $\mathcal{X}_\lambda = \mathcal{Y}_\lambda = \mathcal{I}_\lambda$ and $P_\lambda(x, y) = 0$ if $x = y$, and 1 otherwise.
- **Arithmetic Branching Programs.** Let $\{p_\lambda\}_{\lambda \in \mathbb{N}}$ be a sequence of prime numbers. For each $\lambda \in \mathbb{N}$, we let $\mathcal{Y} = \mathcal{F}_n^{\text{abp}}$, $\mathcal{X} = \mathbb{Z}_{p_\lambda}^n$ and $P_\lambda(x, y) = y(x)$.
- **Bounded-Depth Circuits.** For each $\lambda \in \mathbb{N}$, we let $\mathcal{Y}_\lambda = \mathcal{F}_{n,d}^{\text{circ}}$, $\mathcal{X}_\lambda = \{0, 1\}^n$ and $P_\lambda(x, y) = y(x)$.

Classical Security. We define IND-CPA security.

Definition 14 (Classical Security of AB-IPFE). Let $\text{xxx} \in \{\text{adp}, \text{stat}\}$. An AB-IPFE scheme iFE is secure against Q collusions if $\text{Exp}_{\text{iFE}, \mathcal{A}}^{\text{ab-ipfe-0}}(1^\lambda) \approx_c \text{Exp}_{\text{iFE}, \mathcal{A}}^{\text{ab-ipfe-1}}(1^\lambda)$ for all PPT adversaries $\mathcal{A}$, where $\text{Exp}_{\text{iFE}, \mathcal{A}}^{\text{ab-ipfe-}b}$ for $b \in \{0, 1\}$ proceeds as follows.

- **Setup.** Launch $\mathcal{A}(1^\lambda)$, run $(\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^N)$ and send MPK to $\mathcal{A}$. Initialize an empty multi-set $\mathcal{Q}_{\text{key}} = \emptyset$.
- **Query I.** Repeat the following for arbitrarily many rounds determined by $\mathcal{A}$. In each round, $\mathcal{A}$ submits a query $\text{QKeyGen}(y, \mathbf{v})$, where $y \in \mathcal{Y}_\lambda$ and $\mathbf{v} \in \mathbb{Z}_{p_\lambda}^N$. Upon this query, run $\text{DK} \leftarrow \text{KeyGen}(\text{MSK}, y, \mathbf{v})$ and return DK to $\mathcal{A}$. Add $(y, \mathbf{v})$ to $\mathcal{Q}_{\text{key}}$.
- **Challenge.** Upon $\mathcal{A}$ submitting a challenge attribute x^* and two vectors $\mathbf{u}^0, \mathbf{u}^1 \in \mathbb{Z}_{p_\lambda}^N$, compute $\text{CT} \leftarrow \text{Enc}(\text{MPK}, x^*, \mathbf{u}^b)$ and return CT to $\mathcal{A}$.
- **Query II.** Same as Query I.
- **Guess.** The adversary outputs a bit $b' \in \{0, 1\}$. The outcome of the experiment is b' if $\mathcal{A}$ satisfies the following admissibility condition. Otherwise, the outcome is set to 0. The adversary $\mathcal{A}$ is admissible if
 - $|\mathcal{Q}_{\text{key}}| \leq Q$,
 - for all $(y, \mathbf{v}) \in \mathcal{Q}_{\text{key}}$, if $P_\lambda(x^*, y) = 0$ then $\mathbf{u}^0 \mathbf{v}^\top = \mathbf{u}^1 \mathbf{v}^\top$, and
 - if $\text{xxx} = \text{stat}$, then $\mathcal{A}$ submits all queries to QKeyGen and the challenge query in one shot.

If we do not explicitly specify Q , then the adversary can submit any number of queries to QKeyGen and we say that iFE is secure against *unbounded* collusions.

Assuming the hardness of LWE, there exists an AB-IPFE scheme for bounded-depth circuits that is secure against $\text{poly}(\lambda)$ collusions [LLW21].

Function-Hiding Security. In the secret-key, pairing-based setting, we can define the stronger notion of function-hiding security which differs from classical security (Definition 14) in the following way.

- The Setup phase does not provide the adversary with MPK and initializes a second set $\mathcal{Q}_{\text{enc}} = \emptyset$.
- The phases Query I, Challenge and Query II are replaced with a single phase Query during which the adversary has access to the following two oracles.
 - $\text{QEnc}(x, [\mathbf{u}^0]_1, [\mathbf{u}^1]_1)$: upon $\mathcal{A}$ submitting an attribute $x \in \mathcal{X}_\lambda$ and two vectors $[\mathbf{u}^0]_1, [\mathbf{u}^1]_1 \in \mathbb{G}_{\lambda,1}^N$, run $\text{CT} \leftarrow \text{Enc}(\text{MSK}, x, [\mathbf{u}^b]_1)$ and return CT to $\mathcal{A}$. Add $(x, [\mathbf{u}^0]_1, [\mathbf{u}^1]_1)$ to $\mathcal{Q}_{\text{enc}}$.
 - $\text{QKeyGen}(y, [\mathbf{v}^0]_2, [\mathbf{v}^1]_2)$: upon $\mathcal{A}$ submitting an attribute $y \in \mathcal{Y}_\lambda$ and two vectors $[\mathbf{v}^0]_2, [\mathbf{v}^1]_2 \in \mathbb{G}_{\lambda,2}^N$, run $\text{DK} \leftarrow \text{KeyGen}(\text{MSK}, y, [\mathbf{v}^b]_2)$ and return DK to $\mathcal{A}$. Add $(y, [\mathbf{v}^0]_2, [\mathbf{v}^1]_2)$ to $\mathcal{Q}_{\text{key}}$.
- The second item of the admissibility condition changes to: for all tuples $(x, [\mathbf{u}^0]_1, [\mathbf{u}^1]_1) \in \mathcal{Q}_{\text{enc}}$ and $(y, [\mathbf{v}^0]_2, [\mathbf{v}^1]_2) \in \mathcal{Q}_{\text{key}}$, if $P_\lambda(x, y) = 0$ then $[\mathbf{u}^0]_1 ([\mathbf{v}^0]_2)^\top = [\mathbf{u}^1]_1 ([\mathbf{v}^1]_2)^\top$.

Assuming MDDH on a pairing group $\mathbb{G}$, there exists an AB-IPFE scheme for ABP predicates based on $\mathbb{G}$ that is function-hiding secure against unbounded collusions (special case of [ATY23]).

Restricted Evasive Security for IB-IPFE. For the special case of IB-IPFE we define very selective evasive security against restricted samplers following [HLL24].

Definition 15 (Restricted Evasive Security of IB-IPFE). Let $\mathcal{S} = (\mathcal{S}_V, \mathcal{S}_U)$ be two algorithms with the following syntax.

- Given randomness r_{pub} , $\mathcal{S}_V(1^\lambda; r_{\text{pub}})$ outputs

$$1^N, \text{ID} \subseteq \mathcal{I}, \{(1^{I_{\text{id}}}, 1^{J_{\text{id}}}, \mathbf{V}_{\text{id}} \in \mathbb{Z}_p^{J_{\text{id}} \times N})\}_{\text{id} \in \text{ID}}, \sigma_{\text{pre}} \geq 0.$$

- Given r_{pub} and additional randomness r_{pri} , $\mathcal{S}_U(1^\lambda, r_{\text{pub}}; r_{\text{pri}})$ proceeds in the following two steps.

1. Use a fixed deterministic algorithm $\mathcal{S}_{A0}$ to compute

$$(1^{n'}, \{\mathbf{A}_{\text{id},0,i}\}_{\text{id} \in \text{ID}, i \in [I_{\text{id}}]}) \leftarrow \mathcal{S}_{A0}(1^\lambda, r_{\text{pub}}),$$

where $\mathbf{A}_{\text{id},0,i} \in \mathbb{Z}_p^{n' \times N}$ for all $\text{id} \in \text{ID}$, $i \in [I_{\text{id}}]$, and the values of ID and $\{I_{\text{id}}\}_{\text{id} \in \text{ID}}$ agree with the output of $\mathcal{S}_V(1^\lambda; r_{\text{pub}})$.

2. Use r_{pri} to sample $\mathbf{d}_0 \xleftarrow{\$} \mathbb{Z}_p^{n'}$ and output $\{\mathbf{U}_{\text{id}}\}_{\text{id} \in \text{ID}}$, where

$$\mathbf{U}_{\text{id}} = \begin{bmatrix} \text{---} & \mathbf{u}_{\text{id},1} & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{u}_{\text{id},I_{\text{id}}} & \text{---} \end{bmatrix} \in \mathbb{Z}_p^{I_{\text{id}} \times N},$$

and $\mathbf{u}_{\text{id},i} = \mathbf{d}_0 \mathbf{A}_{\text{id},0,i}$ for all $\text{id} \in \text{ID}$ and $i \in [I_{\text{id}}]$.

Suppose $(\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^N)$ and, for all $\text{id} \in \text{ID}$,

$$\begin{aligned} \mathbf{E}_{\text{id}} &\leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_{\text{pre}}}^{I_{\text{id}} \times J_{\text{id}}}, & \text{CT}_{\text{id}}^* &\leftarrow \text{Enc}(\text{MPK}, \text{id}, \mathbf{U}_{\text{id}}), \\ \Delta_{\text{pre}, \text{id}} &\xleftarrow{\$} \mathbb{Z}_p^{I_{\text{id}} \times J_{\text{id}}}, & \text{CT}_{\text{id}}^\$ &\leftarrow \text{Enc}(\text{MPK}, \text{id}, \mathbf{U}_{\$, \text{id}}), \\ \mathbf{U}_{\$, \text{id}} &\xleftarrow{\$} \mathbb{Z}_p^{I_{\text{id}} \times N}, & \text{DK}_{\text{id}} &\leftarrow \text{KeyGen}(\text{MSK}, \text{id}, \mathbf{V}_{\text{id}}). \end{aligned}$$

$\mathcal{S}$ has *pseudorandom noisy inner products*, denoted $\text{IPFEsec}_{\text{pre}}^{\mathcal{S}}$, if

$$\left\{ \left(1^\lambda, r_{\text{pub}}, \{\mathbf{U}_{\text{id}} \mathbf{V}_{\text{id}}^\top + \mathbf{E}_{\text{id}}\}_{\text{id} \in \text{ID}} \right) \right\}_{\lambda \in \mathbb{N}} \approx_c \left\{ \left(1^\lambda, r_{\text{pub}}, \{\Delta_{\text{id}}\}_{\text{id} \in \text{ID}} \right) \right\}_{\lambda \in \mathbb{N}}.$$

The IB-IPFE scheme iFE is $\mathcal{S}$ -secure, denoted $\text{IPFEsec}_{\text{post}}^{\mathcal{S}}$, if

$$\left\{ \left(1^\lambda, r_{\text{pub}}, \text{MPK}, \{\text{CT}_{\text{id}}^*, \text{DK}_{\text{id}}\}_{\text{id} \in \text{ID}} \right) \right\}_{\lambda \in \mathbb{N}} \approx_c \left\{ \left(1^\lambda, r_{\text{pub}}, \text{MPK}, \{\text{CT}_{\text{id}}^\$, \text{DK}_{\text{id}}\}_{\text{id} \in \text{ID}} \right) \right\}_{\lambda \in \mathbb{N}}.$$

The scheme is $\sigma_{\text{pre},0}$ -secure if it is $\mathcal{S}$ -secure for all efficient $\mathcal{S}$ such that $\sigma_{\text{pre}} \leq \sigma_{\text{pre},0}$ and $\text{IPFEsec}_{\text{pre}}^{\mathcal{S}}$ holds.

Assuming evasive LWE, there exists an IB-IPFE scheme satisfying very selective evasive security against restricted samplers [HLL24].

3.9 Slotted Identity-Based Inner-Product Functional Encryption

We recall the definition of slotted IPFE from [LL20] in the identity-based setting.

Definition 16 (Slotted IB-IPFE). Let $\mathbb{G} = \{\mathbb{G}_\lambda = (\mathbb{G}_{\lambda,1}, \mathbb{G}_{\lambda,2}, \mathbb{G}_{\lambda,t}, g_{\lambda,1}, g_{\lambda,2}, g_{\lambda,t}, e_\lambda, p_\lambda)\}_{\lambda \in \mathbb{N}}$ be a sequence of pairing groups and $\{\mathcal{I}_\lambda\}_{\lambda \in \mathbb{N}}$ a sequence of identity spaces. A slotted IPFE scheme based on $\mathbb{G}$ consists of five efficient algorithms:

$\text{Setup}(1^\lambda, \mathfrak{s}_{\text{pub}}, \mathfrak{s}_{\text{pri}}) \rightarrow (\text{MPK}, \text{MSK})$: On input the security parameter and two disjoint index sets, the public slot $\mathfrak{s}_{\text{pub}}$ and the private slot $\mathfrak{s}_{\text{pri}}$, this algorithm outputs a pair of a master public and a master secret key (MPK, MSK). We denote the whole index set by $\mathfrak{s} := \mathfrak{s}_{\text{pub}} \cup \mathfrak{s}_{\text{pri}}$.

$\text{Enc}(\text{MSK}, i, \llbracket \mathbf{u} \rrbracket_1) \rightarrow \text{CT}$: On input a master secret key MSK, an identity $i \in \mathcal{I}_\lambda$ and an encoding of a vector $\mathbf{u} \in \mathbb{Z}_{p_\lambda}^{\mathfrak{s}}$ in $\mathbb{G}_{\lambda,1}$, this algorithm outputs a ciphertext CT for $\mathbf{u}$.

$\text{KeyGen}(\text{MSK}, i', \llbracket \mathbf{v} \rrbracket_2) \rightarrow \text{DK}$: On input a master secret key MSK, an identity $i' \in \mathcal{I}_\lambda$ and an encoding of a vector $\mathbf{v} \in \mathbb{Z}_{p_\lambda}^{\mathfrak{s}}$ in $\mathbb{G}_{\lambda,2}$, this algorithm outputs a decryption key DK for $\mathbf{v}$.

$\text{Dec}(\text{DK}, \text{CT}) \rightarrow \llbracket d \rrbracket_t \vee \perp$: On input a decryption key DK and a ciphertext CT, this algorithm outputs an element $\llbracket d \rrbracket_t \in \mathbb{G}_{\lambda,t}$ or the special symbol $\perp$ indicating failure of decryption.

$\text{SlotEnc}(\text{MPK}, i, \llbracket \mathbf{u}_{\text{pub}} \rrbracket_1) \rightarrow \text{CT}$: On input a master public key MPK, an identity $i \in \mathcal{I}_\lambda$ and an encoding of a message vector $\mathbf{u}_{\text{pub}} \in \mathbb{Z}_{p_\lambda}^{\mathfrak{s}_{\text{pub}}}$ in $\mathbb{G}_{\lambda,1}$, this algorithm outputs a ciphertext for the vector $\mathbf{u} = \mathbf{u}_{\text{pub}} |^{\mathfrak{s}} \in \mathbb{Z}_{p_\lambda}^{\mathfrak{s}}$.

Correctness. A slotted IB-IPFE scheme satisfies *decryption correctness* if for all $\lambda \in \mathbb{N}$, all disjoint index sets $\mathfrak{s}_{\text{pub}}, \mathfrak{s}_{\text{pri}}$, all identities $i \in \mathcal{I}_\lambda$ and all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_{p_\lambda}^{\mathfrak{s}}$, it holds that

$$\Pr \left[\text{Dec}(\text{DK}, \text{CT}) = \llbracket \mathbf{u}\mathbf{v}^\top \rrbracket_t \mid \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, \mathfrak{s}_{\text{pub}}, \mathfrak{s}_{\text{pri}}) \\ \text{CT} \leftarrow \text{Enc}(\text{MSK}, i, \llbracket \mathbf{u} \rrbracket_1) \\ \text{DK} \leftarrow \text{KeyGen}(\text{MSK}, i, \llbracket \mathbf{v} \rrbracket_2) \end{array} \right] = 1 .$$

Furthermore, we say that a slotted IB-IPFE scheme satisfies *slot-mode correctness* if for all $\lambda \in \mathbb{N}$, all disjoint index sets $\mathfrak{s}_{\text{pub}}, \mathfrak{s}_{\text{pri}}$, all identities $i \in \mathcal{I}_\lambda$ and $\mathbf{u}_{\text{pub}} \in \mathbb{Z}_{p_\lambda}^{\mathfrak{s}_{\text{pub}}}$, the following distributions $\mathcal{D}_0, \mathcal{D}_1$ are identical:

$$\begin{aligned} \mathcal{D}_0 &= \left\{ (\text{MPK}, \text{MSK}, \text{CT}) \mid \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, \mathfrak{s}_{\text{pub}}, \mathfrak{s}_{\text{pri}}) \\ \text{CT} \leftarrow \text{Enc}(\text{MSK}, i, \llbracket \mathbf{u}_{\text{pub}} |^{\mathfrak{s}} \rrbracket_1) \end{array} \right\} , \\ \mathcal{D}_1 &= \left\{ (\text{MPK}, \text{MSK}, \text{CT}) \mid \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, \mathfrak{s}_{\text{pub}}, \mathfrak{s}_{\text{pri}}) \\ \text{CT} \leftarrow \text{SlotEnc}(\text{MPK}, i, \llbracket \mathbf{u}_{\text{pub}} \rrbracket_1) \end{array} \right\} , \end{aligned}$$

where the probability is taken over the random coins of the algorithms.

Security. We define adaptive function-hiding IND-CPA security.

Definition 17 (Function-Hiding Security). A slotted IB-IPFE scheme iFE satisfies function-hiding security if $\text{Exp}_{0,\mathcal{A}}^{\text{sl-ipfe-}b}(1^\lambda) \approx_c \text{Exp}_{1,\mathcal{A}}^{\text{sl-ipfe-}b}(1^\lambda)$ for all PPT adversaries $\mathcal{A}$, where $\text{Exp}_{\text{iFE},\mathcal{A}}^{\text{sl-ipfe-}b}$ for $b \in \{0, 1\}$ proceeds as follows.

- **Setup.** Launch $\mathcal{A}(1^\lambda)$, generate $(\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda)$ and send MPK to $\mathcal{A}$. Initialize two empty sets $\mathcal{Q}_{\text{enc}}, \mathcal{Q}_{\text{key}} = \emptyset$.
- **Query.** Repeat the following for arbitrarily many rounds determined by $\mathcal{A}$. In each round, $\mathcal{A}$ has two options.
 - $\text{QEnc}(i, \llbracket \mathbf{u}^0 \rrbracket_1, \llbracket \mathbf{u}^1 \rrbracket_1)$: upon $\mathcal{A}$ submitting an attribute $i \in \mathcal{I}_\lambda$ and two vectors $\llbracket \mathbf{u}^0 \rrbracket_1, \llbracket \mathbf{u}^1 \rrbracket_1 \in \mathbb{G}_{\lambda,1}^{\mathfrak{s}}$, run $\text{CT} \leftarrow \text{Enc}(\text{MSK}, i, \llbracket \mathbf{u}^b \rrbracket_1)$ and return CT to $\mathcal{A}$. Add $(i, \llbracket \mathbf{u}^0 \rrbracket_1, \llbracket \mathbf{u}^1 \rrbracket_1)$ to $\mathcal{Q}_{\text{enc}}$.
 - $\text{QKeyGen}(i', \llbracket \mathbf{v}^0 \rrbracket_2, \llbracket \mathbf{v}^1 \rrbracket_2)$: upon $\mathcal{A}$ submitting an attribute $i' \in \mathcal{I}_\lambda$ and two vectors $\llbracket \mathbf{v}^0 \rrbracket_2, \llbracket \mathbf{v}^1 \rrbracket_2 \in \mathbb{G}_{\lambda,2}^{\mathfrak{s}}$, run $\text{DK} \leftarrow \text{KeyGen}(\text{MSK}, i', \llbracket \mathbf{v}^b \rrbracket_2)$ and return DK to $\mathcal{A}$. Add $(i', \llbracket \mathbf{v}^0 \rrbracket_2, \llbracket \mathbf{v}^1 \rrbracket_2)$ to $\mathcal{Q}_{\text{key}}$.
- **Guess.** The adversary outputs a bit $b' \in \{0, 1\}$. The outcome of the experiment is b' if $\mathcal{A}$ satisfies the following admissibility condition. Otherwise, the outcome is set to 0. The adversary $\mathcal{A}$ is admissible if for all $(i, \llbracket \mathbf{u}^0 \rrbracket_1, \llbracket \mathbf{u}^1 \rrbracket_1) \in \mathcal{Q}_{\text{enc}}$ and $(i', \llbracket \mathbf{v}^0 \rrbracket_2, \llbracket \mathbf{v}^1 \rrbracket_2) \in \mathcal{Q}_{\text{key}}$ it holds that $\llbracket \mathbf{v}_0 |_{\mathfrak{s}_{\text{pub}}} \rrbracket_2 = \llbracket \mathbf{v}_1 |_{\mathfrak{s}_{\text{pub}}} \rrbracket_2$ and, if $i = i'$, then $\llbracket \mathbf{u}^0 \rrbracket_1 \llbracket (\mathbf{v}^0)^\top \rrbracket_2 = \llbracket \mathbf{u}^1 \rrbracket_1 \llbracket (\mathbf{v}^1)^\top \rrbracket_2$.

There exists a (non-identity-based) slotted IPFE scheme based on $\mathbb{G}$ which can be proven secure under the MDDH assumption on $\mathbb{G}$. The construction is based on a sequence of works [ALS16, Wee17, LV16, Lin17] and has been described explicitly in [LL20]. In this work, we use slotted IB-IPFE only for identity spaces of polynomial size. Slotted IB-IPFE for such small identity spaces can be obtained naively by running multiple slotted IPFE instance in parallel, one for each identity.

3.10 Attribute-Based Encryption

We recall the definition of Attribute-Based Encryption (ABE).

Definition 18 (Attribute-Based Encryption). Let $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$, $P = \{P_\lambda\}_{\lambda \in \mathbb{N}}$, $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of message spaces, predicates, ciphertext attribute spaces and key attribute spaces, respectively. An ABE scheme for $\mathcal{M}$, P , $\mathcal{X}$ and $\mathcal{Y}$ consists of four efficient algorithms:

Setup(1^λ) $\rightarrow$ (MPK, MSK): On input the security parameter, this algorithm outputs a pair of a master public key MPK and a master secret key MSK.

Enc(MPK, x, μ) $\rightarrow$ CT: On input a master public key MPK, a ciphertext attribute $x \in \mathcal{X}_\lambda$ and a message $\mu \in \mathcal{M}_\lambda$, this algorithm outputs a ciphertext CT for μ created with respect to x .

KeyGen(MSK, y) $\rightarrow$ DK: On input a master secret key MSK and a key attribute $y \in \mathcal{Y}_\lambda$, this algorithm outputs a decryption key DK for y .

Dec(DK, CT) $\rightarrow \mu' \vee \perp$: On input a decryption key DK and a ciphertext CT, this algorithm outputs an element $\mu' \in \mathcal{M}_\lambda$ or $\perp$.

Correctness. An ABE scheme is said to be correct if for all $\lambda \in \mathbb{N}$, all attributes $x \in \mathcal{X}_\lambda$ and $y \in \mathcal{Y}_\lambda$ such that $P_\lambda(x, y) = 0$, and all messages $\mu \in \mathcal{M}_\lambda$, it holds that

$$\Pr \left[\text{Dec}(\text{DK}, \text{CT}) = \mu \mid \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda) \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, x, \mu) \\ \text{DK} \leftarrow \text{KeyGen}(\text{MSK}, y) \end{array} \right] = 1 ,$$

where the probability is taken over the random coins of the algorithms.

Security. We define adaptive IND-CPA security.

Definition 19 (Security). An ABE scheme ABE is secure if $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{abe-0}}(1^\lambda) \approx_c \text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{abe-1}}(1^\lambda)$ for all PPT adversaries $\mathcal{A}$, where $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{abe-}b}$ for $b \in \{0, 1\}$ proceeds as follows.

- **Setup.** Launch $\mathcal{A}(1^\lambda)$, run $(\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda)$ and send MPK to $\mathcal{A}$. Initialize an empty set $\mathcal{Q} = \emptyset$.
- **Query I.** Repeat the following for arbitrarily many rounds determined by $\mathcal{A}$. In each round, $\mathcal{A}$ submits a query $\text{QKeyGen}(y)$ where $y \in \mathcal{Y}_\lambda$. Upon this query, run $\text{DK} \leftarrow \text{KeyGen}(\text{MSK}, y)$ and return DK to $\mathcal{A}$. Add y to $\mathcal{Q}$.
- **Challenge.** Upon $\mathcal{A}$ submitting a challenge attribute x^* and two messages $\mu^0, \mu^1 \in \mathcal{M}_\lambda$, run $\text{CT} \leftarrow \text{Enc}(\text{MPK}, x^*, \mu^b)$ and return CT to $\mathcal{A}$.
- **Query II.** Same as Query I.
- **Guess.** The adversary outputs a bit $b' \in \{0, 1\}$. The outcome of the experiment is b' if $P_\lambda(x^*, y) \neq 0$ for all $y \in \mathcal{Q}$. Otherwise, the outcome is set to 0.

We mostly use ABE for the equality predicate, i.e., Identity-Based Encryption (IBE). There exist various IBE schemes in the group-based setting, e.g., [Wat09, CLL⁺13, JR17], and in the lattice-based setting, e.g., [GPV08, ABB10]. We also rely on a CP-ABE for bounded-depth circuits in which case $\mathcal{X}_\lambda = \mathcal{F}_{n,d}^{\text{circ}}$, $\mathcal{Y}_\lambda = \mathbb{Z}_{p_\lambda}$ (for a prime p_λ) and $P_\lambda(x, y) = x(y)$ for all $\lambda \in \mathbb{N}$. Assuming the hardness of LWE, there exists a CP-ABE for bounded-depth circuits in the *secret-key* setting [AY20].

3.11 Other Preliminaries

We recall the following lemma from [LLW21].

Lemma 20 ([LLW21], Lemma 5.4). *Let $\kappa = Q\tau\epsilon^2$ and $\tau = \Theta(\epsilon)$. There exists an efficient sampling algorithm, denoted $\text{SamplerSet}(\kappa, Q, \tau)$, with the following properties:*

1. *The sampler always outputs a set $\Delta \subset [\kappa]$ with cardinality τ .*
2. *For independent samples $\Delta_1, \dots, \Delta_Q$ from $\text{SamplerSet}(\kappa, Q, \tau)$, the sets are cover-free with probability $(1 - 2^{-\Omega(\epsilon)})$, i.e., for all $i \in [Q]$,*

$$\Pr[\Delta_i \setminus (\bigcup_{j \in [Q] \setminus \{i\}} \Delta_j) \neq \emptyset] \geq 1 - 2Q \cdot 2^{-\Omega(\epsilon)}.$$

4 MC-ABE for Succinctly Enumerable Function Classes

Our constructions in this section capture all function classes that satisfy a specific property which we call succinct enumerability.

Succinct Enumerability. We define the following notations:

- Let $X_0 = \{0\}$, $X_1 = \{1\}$ and $X_\star = \{0, 1\}$. For a vector $\mathbf{v} \in \{0, 1, \star\}^k$, we let

$$X_{\mathbf{v}} = X_{\mathbf{v}[1]} \times \dots \times X_{\mathbf{v}[k]}.$$

- For concatenated vectors $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in \{0, 1, \star\}^{[n-1] \times [k]}$, we write

$$X_{\mathbf{v}} = X_{\mathbf{v}_1} \times \dots \times X_{\mathbf{v}_{n-1}} \quad \text{and} \quad J_{\mathbf{v}} = \{(i, j) \in [n-1] \times [k] : \mathbf{v}_i[j] \neq \star\}.$$

Definition 21 (Succinct Enumerability). Consider a policy class $\mathcal{F}$ containing policies $f: \{0, 1\}^{[0; n-1] \times [k]} \rightarrow \{0, 1\}$. $\mathcal{F}$ is called succinctly enumerable if there exists a set $V \subseteq \{0, 1, \star\}^{[n-1] \times [k]}$ of polynomial size such that for all $f \in \mathcal{F}$ there exists a set $J_f \subseteq [n-1] \times [k]$ satisfying the following two conditions.

- **Correctness.** For all $(\mathbf{x}_0, \mathbf{x}_{\geq 1}) \in \{0, 1\}^{\{0\} \times [k]} \times \{0, 1\}^{[n-1] \times [k]}$, if $f(\mathbf{x}_0, \mathbf{x}_{\geq 1}) = 0$ then there exists $\mathbf{v} \in V$ such that $J_{\mathbf{v}} = J_f$ and $\mathbf{x}_{\geq 1} \in X_{\mathbf{v}}$.
- **Security.** For all $\mathbf{x}_0 \in \{0, 1\}^{\{0\} \times [k]}$ and $\mathbf{v} \in V$, if $J_{\mathbf{v}} = J_f$ then the function f is constant on the set $\{(\mathbf{x}_0, \mathbf{x}_{\geq 1}) : \mathbf{x}_{\geq 1} \in X_{\mathbf{v}}\}$.

We may consider the following examples of succinctly enumerable classes $\mathcal{F}$.

- **Constant Locality.** Let $k, n = \text{poly}(\lambda)$ and $d = O(1)$ be some fixed upper bound such that each function $f \in \mathcal{F}$ depends on at most d out of the $k(n-1)$ bits in $\mathbf{x}_{\geq 1}$. For this class of functions, we define

$$V = \{\mathbf{v} \in \{0, 1, \star\}^{[n-1] \times [k]} : |J_{\mathbf{v}}| = d\}$$

and, for $f \in \mathcal{F}$, we can pick any set $J_f \subseteq [n-1] \times [k]$ of size d which includes the (at most d) bits that the policy depends on. Correctness and security follow immediately from the fact that the inputs in $([n-1] \times [k]) \setminus J_f$ are irrelevant for the output of f . Therefore, it suffices to observe that

$$|V| \leq \binom{kn}{d} \cdot 2^d = \text{poly}(\lambda) .$$

Note that this class includes all NC^0 policies which is one of the classes considered in [PS24]. However, we do *not* limit the bit dependency for $\mathbf{x}_0$ here, hence this class also includes policies that cannot be computed by an NC^0 circuit.

- **Polynomial Accepting Sets.** Let $k, n = \text{poly}(\lambda)$ and $V \subseteq \{0, 1\}^{[n-1] \times k}$ a set of polynomial size satisfying the following implication for each $(\mathbf{x}_0, \mathbf{x}_{\geq 1}) \in \{0, 1\}^{\{0\} \times [k]} \times \{0, 1\}^{[n-1] \times [k]}$ and $f \in \mathcal{F}$: if $f(\mathbf{x}_0, \mathbf{x}_{\geq 1}) = 0$ then $\mathbf{x}_{\geq 1} \in V$. When choosing $J_f = [n-1] \times [k]$ for each $f \in \mathcal{F}$, then correctness and security are immediate as the definition of V does not involve any $\star$ -coordinates, and $|V| = \text{poly}(\lambda)$ by definition.

While this function class may seem rather limited, we observe that we do not limit the total number of accepting inputs but only the number of accepting $\mathbf{x}_{\geq 1}$ *per* $\mathbf{x}_0$. This is more general than the classes of $O(1)$ -threshold and $(kn - O(1))$ -threshold policies considered in [PS24] whose total number of accepting inputs is polynomial.

- **Logarithmic-Length Inputs.** Let k, n such that $kn = O(\log \lambda)$, i.e., the total length of the input is logarithmic in λ . For this parameter choice, every class of functions is succinctly enumerable as we can define $V = \{0, 1\}^{[n-1] \times [k]}$ and $J_f = [n-1] \times [k]$ for each $f \in \mathcal{F}$. Correctness and security are immediate as the definition of V does not involve any $\star$ -coordinates, and $|V| = 2^{k(n-1)} = \text{poly}(\lambda)$ due to the small input length.

This includes all NC^1 policies with logarithmic-length inputs as considered in [PS24] but equally applies to all short-input bounded-depth circuits as we show in Section 4.3.

Complete Policies. Let $k, n \in \mathbb{N}$. We define $c: \{0, 1\}^{[0; n-1] \times \{0\}} \rightarrow \{0, 1\}$ as

$$c((x_{i,0})_{i \in [0; n-1]}) = \bigwedge_{i \in [0; n-1]} x_{i,0} .$$

Given a policy $f: \{0, 1\}^{[0; n-1] \times [k]} \rightarrow \{0, 1\}$, we call $(c \wedge f): \{0, 1\}^{[0; n-1] \times [0; k]} \rightarrow \{0, 1\}$ the *complete* policy defined as

$$(c \wedge f)((x_{i,j})_{i,j}) = c((x_{i,0})_{i \in [0; n-1]}) \wedge f((x_{i,j})_{(i,j) \in [0; n-1] \times [k]}) .$$

To deal with complete policies, we adapt our previous notations as follows:

- For a vector $\mathbf{v} \in \{0, 1\}^{\star [k]}$, we write $\bar{\mathbf{v}} = (1, \mathbf{v}) \in \{0, 1\}^{[0; k]}$.
- For $\mathbf{v} \in V$, we extend the notation of $J_{\mathbf{v}} = \{(i, j) \in [n-1] \times [k] : \mathbf{v}_i[j] \neq \star\}$ to

$$\bar{J}_{\mathbf{v}} = J_{\mathbf{v}} \cup (\{0\} \times [k]) \cup ([0; n-1] \times \{0\}) \subseteq ([0; n-1] \times [0; k]) .$$

Intuitively, for a given vector $\mathbf{x}_0 \in \{0, 1\}^{\{0\} \times [k]}$, the set $\bar{J}_{\mathbf{v}}$ contains the indices of the non- $\star$ coordinates for the extension $(\bar{\mathbf{x}}_0, \bar{\mathbf{v}}_1, \dots, \bar{\mathbf{v}}_{n-1}) \in \bar{V}_{\mathbf{x}_0}$ of a vector $(\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V$.

- Similarly, for a function f with corresponding set J_f (as per Definition 21), we extend the notation of $J_f \subseteq [n-1] \times [k]$ to

$$\bar{J}_f = J_f \cup (\{0\} \times [k]) \cup ([0; n-1] \times \{0\}) \subseteq ([0; n-1] \times [0; k]) .$$

4.1 Construction from Slotted IPFE

Let $\mathcal{F}$ be a succinctly enumerable class of functions $f: \{0,1\}^{[0;n-1] \times [k]} \rightarrow \{0,1\}$. We present our construction of MC-ABE for the predicate $P_{\mathcal{F}}: \mathcal{X}^n \times \mathcal{Y} \rightarrow \{0,1\}$, where $\mathcal{X} = \{0,1\}^{[k]}$, $\mathcal{Y} = \mathcal{F}$ and $P_{\mathcal{F}}((\mathbf{x}_0, \dots, \mathbf{x}_{n-1}), f) = f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1})$. In this section, we give the construction using a slotted IPFE as building block. This allows basing security on MDDH $_k$ on pairings. In Section 4.3, we give a modified version of the construction that makes use of evasive IPFE, thus providing an instantiation from (public-coin) evasive LWE.

Construction 22 (MC-ABE from Slotted IPFE). Let $\mathcal{F} \subseteq \text{NC}^1$ be a succinctly enumerable class of functions. Our construction uses the following ingredients:

- an identity-based slotted IPFE scheme $\text{iFE} = (\text{iSetup}, \text{iEnc}, \text{iKeyGen}, \text{iDec})$ based on a pairing group $\mathbb{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t, g_1, g_2, g_t, e, p)$ with identity space $([0; n-1] \times [0; k]) \cup \{\text{pad}\}$,
- an identity-based encryption scheme $\text{idFE} = (\text{idSetup}, \text{idEnc}, \text{idKeyGen}, \text{idDec})$ with identity space $\mathcal{L} \times \{0,1,\star\}^{[k]}$ for $\mathcal{L} = \{0,1\}^{\text{poly}(\lambda)}$ and message space being the ciphertext space of iFE, and
- an (exact) linear garbling scheme $(\text{Gen}, \text{Share}, \text{Rec}, \text{Sim})$ for NC^1 .

The MC-ABE scheme ABE for the predicate $P_{\mathcal{F}}$ with message space $\mathcal{M} = \mathbb{G}_t$ and label space $\mathcal{L}$:

$\text{Setup}(1^\lambda)$ takes as input the security parameter 1^λ and generates

$$\begin{aligned} \text{iMPK}, \text{iMSK} &\leftarrow \text{iSetup}(1^\lambda, [2\kappa], \{2\kappa+1\}) \\ \{(\text{idMPK}_i, \text{idMSK}_i) &\leftarrow \text{idSetup}(1^\lambda)\}_{i \in [n-1]} . \end{aligned}$$

Then it outputs $(\text{MPK}, \text{MSK}, \{\text{SK}_i\}_{i \in [n-1]})$ as follows:

$$\text{MPK} = (\text{iMPK}, \{\text{idMPK}_i\}_{i \in [n-1]}), \text{MSK} = \text{iMSK}, \{\text{SK}_i = \text{idMSK}_i\}_{i \in [n-1]}.$$

$\text{Enc}(\text{MPK}, L, \mathbf{x}_0, \llbracket \mu \rrbracket_t)$ takes $\text{MPK} = (\text{iMPK}, \{\text{idMPK}_i\}_{i \in [n-1]})$, a label $L \in \mathcal{L}$, an attribute $\mathbf{x}_0 \in \{0,1\}^k$ and a message $\llbracket \mu \rrbracket_t \in \mathbb{G}_t$ as input. The algorithm picks $\llbracket \sigma \rrbracket_1 \xleftarrow{\$} \mathbb{G}_1$ and $\llbracket \mathbf{r}^{\mathbf{v}} \rrbracket_1 \xleftarrow{\$} \mathbb{G}_1^k$ for each $\mathbf{v} \in V$, sets $\mathbf{v}_0 = \mathbf{x}_0$ and

$$\begin{aligned} \{\mathbf{p}_{\text{pad}}^{\mathbf{v}} &= (\mathbf{r}^{\mathbf{v}}, \sigma, \mathbf{0}_{\kappa-1})\}_{\mathbf{v} \in V} \\ \{\mathbf{p}_{i,j}^{\mathbf{v}} &= (0^{\bar{\mathbf{v}}_i[j] \cdot \kappa}, \mathbf{r}^{\mathbf{v}}, 0^{(1-\bar{\mathbf{v}}_i[j]) \cdot \kappa})\}_{\substack{\mathbf{v}=(\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V \\ (i,j) \in \bar{J}_{\mathbf{v}}}} , \end{aligned}$$

computes $\llbracket d \rrbracket_t = \llbracket \mu + \sigma \rrbracket_t$ and generates

$$\begin{aligned} \{\text{iCT}_{\text{pad}}^{\mathbf{v}} &\leftarrow \text{iSlotEnc}(\text{iMPK}, \text{pad}, \llbracket \mathbf{p}_{\text{pad}}^{\mathbf{v}} \rrbracket_1)\}_{\mathbf{v} \in V} \\ \{\text{iCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{iSlotEnc}(\text{iMPK}, (i, j), \llbracket \mathbf{p}_{i,j}^{\mathbf{v}} \rrbracket_1)\}_{(i,j) \in \bar{J}_{\mathbf{v}}}^{\mathbf{v} \in V} \\ \{\text{idCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{idEnc}(\text{idMPK}_i, (L, \mathbf{v}_i), \text{iCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_{\mathbf{v}}, i>0}^{\mathbf{v}=(\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V} . \end{aligned}$$

Finally, the algorithm outputs the ciphertext

$$\text{CT}_L = (\llbracket d \rrbracket_t, \{\text{iCT}_{\text{pad}}^{\mathbf{v}}\}_{\mathbf{v} \in V}, \{\text{iCT}_{0,j}^{\mathbf{v}}\}_{(0,j) \in \bar{J}_{\mathbf{v}}}^{\mathbf{v} \in V}, \{\text{idCT}_{i,j}^{\mathbf{v}}\}_{(i,j) \in \bar{J}_{\mathbf{v}}, i>0}^{\mathbf{v} \in V}) .$$

$\text{AKeyGen}(\text{SK}_i, L, \mathbf{x}_i)$ takes as input $\text{SK}_i = \text{idMSK}_i$ for some $i \in [n-1]$, a label $L \in \mathcal{L}$ and an attribute $\mathbf{x}_i \in \{0,1\}^k$, defines

$$V_{\mathbf{x}_i} = \{\mathbf{v}_i : (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V, \mathbf{x}_i \in X_{\mathbf{v}_i}\} ,$$

computes

$$\{\text{idDK}_i^{\mathbf{v}_i} \leftarrow \text{idKeyGen}(\text{idMSK}_i, (L, \mathbf{v}_i))\}_{\mathbf{v}_i \in V_{\mathbf{x}_i}}$$

and outputs $\text{DK}_{L,i} = \{\text{idDK}_i^{\mathbf{v}_i}\}_{\mathbf{v}_i \in V_{\mathbf{x}_i}}$.

$\text{PKeyGen}(\text{MSK}, f)$ takes as input $\text{MSK} = \text{iMSK}$ and a policy $f \in \mathcal{F}$. We let $c \wedge f$ be the complete policy viewed as a function $\{0, 1\}^{\bar{J}_f} \rightarrow \{0, 1\}$.⁹ The algorithm samples $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^k$, generates shares $\{\mathbf{s}_{i,j}^\beta\}_{(i,j) \in \bar{J}_f}^{\beta \in \{0,1\}} \leftarrow \text{Share}(c \wedge f, \mathbf{s})$, defines

$$\mathbf{q}_{\text{pad}} = (\mathbf{s}, 1, \mathbf{0}_{\kappa-1}, 0), \quad \{\mathbf{q}_{i,j} = (\mathbf{s}_{i,j}^0, \mathbf{s}_{i,j}^1, 0)\}_{(i,j) \in \bar{J}_v},$$

and computes

$$\begin{aligned} \text{idK}_{\text{pad}} &\leftarrow \text{iKeyGen}(\text{iMSK}, \text{pad}, \llbracket \mathbf{q}_{\text{pad}} \rrbracket_2) \\ \{\text{idK}_{i,j} &\leftarrow \text{iKeyGen}(\text{iMSK}, (i, j), \llbracket \mathbf{q}_{i,j} \rrbracket_2)\}_{(i,j) \in \bar{J}_f}. \end{aligned}$$

Finally, it outputs $\text{DK}_f = (\text{idK}_{\text{pad}}, \{\text{idK}_{i,j}\}_{(i,j) \in \bar{J}_f})$.

$\text{Dec}(\text{DK}_f, \{\text{DK}_{L,i}\}_{i \in [n-1]}, \text{CT}_L)$ takes as input a decryption key DK_f for a policy $f \in \mathcal{F}$, a set of decryption keys $\{\text{DK}_{L,i}\}_{i \in [n-1]}$ created with respect to attributes $\mathbf{x}_1, \dots, \mathbf{x}_{n-1} \in \{0, 1\}^k$ and a label $L \in \mathcal{L}$, and a ciphertext CT_L created with respect to an attribute $\mathbf{x}_0 \in \{0, 1\}^k$ and the same label L . If $f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) = 1$, return $\perp$. Otherwise, parse $\text{DK}_f, \{\text{DK}_{L,i}\}_{i \in [n-1]}$ and CT_L as above. The algorithm picks $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V$ such that $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in X_{\mathbf{v}}$ and $J_{\mathbf{v}} = J_f$. Then it recovers d_{pad} and $\mathbf{p}_{f,\mathbf{x}} = (\mathbf{p}_{f,\mathbf{x}}[i, j])_{(i,j) \in \bar{J}_v}$ by decrypting

$$\begin{aligned} \{\text{idCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{idDec}(\text{idDK}_i^{\mathbf{v}}, \text{idCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_v, i > 0} \\ \{\llbracket \mathbf{p}_{f,\mathbf{x}}[i, j] \rrbracket_t &\leftarrow \text{iDec}(\text{idK}_{i,j}, \text{idCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_v} \\ \llbracket d_{\text{pad}} \rrbracket_t &\leftarrow \text{iDec}(\text{idK}_{\text{pad}}, \text{idCT}_{\text{pad}}^{\mathbf{v}}). \end{aligned}$$

Furthermore, it computes $\mathbf{d}_{f,\mathbf{x}} = (\mathbf{d}_{f,\mathbf{x}}[i, j])_{(i,j) \in \bar{J}_v} \leftarrow \text{Rec}(c \wedge f, \mathbf{x})$, where $(c \wedge f)$ is again viewed as a function with domain $\bar{J}_v$. Finally, the algorithm outputs $\llbracket \mu' \rrbracket_t = \llbracket d \rrbracket_t + \llbracket \mathbf{p}_{f,\mathbf{x}} \rrbracket_t \cdot \mathbf{d}_{f,\mathbf{x}}^\top - \llbracket d_{\text{pad}} \rrbracket_t$.

Correctness. Consider $f \in \mathcal{F}$ and $(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) \in \{0, 1\}^{[0;n-1] \times [k]}$ such that $f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) = 0$. The correctness property of succinctly enumerable function classes guarantees that there exists $\mathbf{v} \in V$ such that $J_{\mathbf{v}} = J_f$ and $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in X_{\mathbf{v}}$. This implies that all idFE ciphertexts $\{\text{idCT}_{i,j}^{\mathbf{v}}\}_{(i,j) \in \bar{J}_v, i > 0}$ and secret keys $\{\text{idDK}_i^{\mathbf{v}}\}_{(i,j) \in \bar{J}_v, i > 0}$ used in the first step of the decryption algorithm are contained in CT_L and $\{\text{DK}_{L,i}\}_{i \in [n-1]}$, respectively. From the correctness of idFE and iFE it follows that $d_{\text{pad}} = \sigma + \mathbf{s} \cdot \mathbf{r}^\top$ and $\mathbf{p}_{f,\mathbf{x}}[i, j] = \mathbf{s}_{i,j}^{\mathbf{x}_i[j]} \cdot \mathbf{r}^\top$ for each $(i, j) \in \bar{J}_v$. Thus, we have

$$\mathbf{p}_{f,\mathbf{x}} \cdot \mathbf{d}_{f,\mathbf{x}}^\top = (\mathbf{s}_{i,j}^{\mathbf{x}_i[j]})_{(i,j) \in \bar{J}_v} \cdot (\mathbf{I}_m \otimes \mathbf{r}^\top) \cdot \mathbf{d}_{f,\mathbf{x}}^\top = \mathbf{s} \cdot \mathbf{r}^\top,$$

where the last equality follows from the correctness of the linear garbling scheme (Lemma 8). Plugging everything together gives

$$\llbracket \mu' \rrbracket_t = \llbracket d \rrbracket_t + \llbracket \mathbf{p}_{f,\mathbf{x}} \rrbracket_t \cdot \mathbf{d}_{f,\mathbf{x}}^\top - \llbracket d_{\text{pad}} \rrbracket_t = \llbracket (\mu + \sigma) + \mathbf{s} \cdot \mathbf{r}^\top - (\sigma + \mathbf{s} \cdot \mathbf{r}^\top) \rrbracket_t = \llbracket \mu \rrbracket_t.$$

Security. We prove selective security.

Proposition 23. *If the MDDH_k assumption holds in $\mathbb{G}_2$, iFE is slot-mode correct and function-hiding and idFE is secure, then Construction 22 is sel-secure.*

When instantiating iFE with the identity-based version of slotted IPFE from [LL20] and idFE with the IBE from [HJP18], we obtain the following theorem.

Theorem 24. *Assuming MDDH_k on pairings, there exists a sel-secure MC-ABE for each succinctly enumerable policy class $\mathcal{F} \subseteq \text{NC}^1$.*

⁹Note that this is well-defined since the definition of succinct enumerability implies that inputs in $([0; n-1] \times [0; k]) \setminus \bar{J}_f$ are irrelevant for the output of f .

4.2 Proof of Security

Proof of Proposition 23. Let $\text{QEnc}(L, \mathbf{x}_0, \llbracket \mu^0 \rrbracket_t, \llbracket \mu^1 \rrbracket_t)$ be the (single) encryption query. We let $q = |V|$ and parse $V = \{\mathbf{v}_\ell\}_{\ell \in [q]}$, where $\mathbf{v}_\ell = (\mathbf{v}_{\ell,1}, \dots, \mathbf{v}_{\ell,n-1})$ for each $\ell \in [q]$. For notational convenience, we also set $\mathbf{v}_{\ell,0} = \mathbf{x}_0$. For the reply to the encryption query, the challenger samples random elements $\llbracket \sigma \rrbracket_1 \xleftarrow{\$} \mathbb{G}_1$ and $\llbracket \mathbf{r}^{\mathbf{v}_\ell} \rrbracket_1 \xleftarrow{\$} \mathbb{G}_1^X$ for each $\ell \in [q]$, sets

$$\{\mathbf{p}_{\text{pad}}^{\mathbf{v}_\ell} = (\mathbf{r}^{\mathbf{v}_\ell}, \sigma, \mathbf{0}_{\kappa-1})\}_{\ell \in [q]}, \quad \{\mathbf{p}_{i,j}^{\mathbf{v}_\ell} = (0^{\bar{\mathbf{v}}_{\ell,i}[j] \cdot \kappa}, \mathbf{r}^{\mathbf{v}_\ell}, 0^{(1-\bar{\mathbf{v}}_{\ell,i}[j]) \cdot \kappa})\}_{\ell \in [q], (i,j) \in \bar{J}_{\mathbf{v}_\ell}},$$

computes $\llbracket d \rrbracket_t = \llbracket \mu + \sigma \rrbracket_t$ and generates

$$\begin{aligned} \{\text{iCT}_{\text{pad}}^{\mathbf{v}_\ell} &\leftarrow \text{iSlotEnc}(\text{iMPK}, \text{pad}, \llbracket \mathbf{p}_{\text{pad}}^{\mathbf{v}_\ell} \rrbracket_1)\}_{\ell \in [q]} \\ \{\text{iCT}_{i,j}^{\mathbf{v}_\ell} &\leftarrow \text{iSlotEnc}(\text{iMPK}, (i, j), \llbracket \mathbf{p}_{i,j}^{\mathbf{v}_\ell} \rrbracket_1)\}_{\ell \in [q], (i,j) \in \bar{J}_{\mathbf{v}_\ell}} \\ \{\text{idCT}_{i,j}^{\mathbf{v}_\ell,i} &\leftarrow \text{idEnc}(\text{idMPK}_i, (L, \mathbf{v}_{\ell,i}), \text{iCT}_{i,j}^{\mathbf{v}_\ell})\}_{\ell \in [q], (i,j) \in \bar{J}_{\mathbf{v}_\ell}, i > 0}. \end{aligned}$$

Then it sends the following challenge ciphertext CT_L to $\mathcal{A}$:

$$\text{CT}_L = (\llbracket d \rrbracket_t, \{\text{iCT}_{\text{pad}}^{\mathbf{v}_\ell}\}_{\ell \in [q]}, \{\text{iCT}_{0,j}^{\mathbf{v}_\ell}\}_{\ell \in [q], (0,j) \in \bar{J}_{\mathbf{v}_\ell}}, \{\text{idCT}_{i,j}^{\mathbf{v}_\ell,i}\}_{\ell \in [q], (i,j) \in \bar{J}_{\mathbf{v}_\ell}, i > 0}).$$

We consider a sequence of hybrid games G_ℓ^b , for $\ell \in [0; q]$ and $b \in \{0, 1\}$, where G_ℓ^b is the same as $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b}$ except that, for each $\ell' \in [q]$, the challenger computes

$$\mathbf{p}_{\text{pad}}^{\mathbf{v}_{\ell'}} = \begin{cases} (\mathbf{r}^{\mathbf{v}_{\ell'}}, \mathbf{0}, \mathbf{0}_{\kappa-1}) & \text{if } \ell' \leq \ell \\ (\mathbf{r}^{\mathbf{v}_{\ell'}}, \sigma, \mathbf{0}_{\kappa-1}) & \text{if } \ell' > \ell \end{cases}$$

We note that $G_0^b = \text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b}$ and $G_q^0 \equiv G_q^1$. For the latter, we observe that in the hybrids G_q^0 and G_q^1 the random mask $\llbracket \sigma \rrbracket_t \xleftarrow{\$} \mathbb{G}_t$ appears only in the term $\llbracket d \rrbracket_t := \llbracket \sigma + \mu^b \rrbracket_t$, which implies that the distributions $\llbracket \sigma + \mu^0 \rrbracket_t$ and $\llbracket \sigma + \mu^1 \rrbracket_t$ are identical. Below we prove the following claim for all $\ell \in [q]$ and $b \in \{0, 1\}$.

Claim 25. *We have $G_{\ell-1}^b \approx_c G_\ell^b$ under the slot-mode correctness and function-hiding security of iFE, the security of idFE and the MDDH_k assumption in $\mathbb{G}_2$.*

This concludes the proof of the proposition. $\square$

We next prove the claim.

Proof of Claim 25. We consider a sequence of hybrids $\widehat{G}_0^0, \dots, \widehat{G}_7^0, \widehat{G}_7^1, \dots, \widehat{G}_0^1$, where $\widehat{G}_0^0 = G_{\ell-1}^b$ and $\widehat{G}_0^1 = G_\ell^b$. We argue $\widehat{G}_{\alpha-1}^{b'} \approx_c \widehat{G}_\alpha^{b'}$, for $\alpha \in [7]$ and $b' \in \{0, 1\}$, as well as $G_7^0 \equiv G_7^1$. Then the claim follows via a hybrid argument. Modifications between consecutive games are highlighted using **boxes**.

Game $\widehat{G}_0^{b'}$: This is $G_{\ell-1}^b$.

Game $\widehat{G}_1^{b'}$: This game is the same as $\widehat{G}_0^{b'}$, except that the challenger computes

$$\begin{aligned} \text{iCT}_{\text{pad}}^{\mathbf{v}_\ell} &\leftarrow \boxed{\text{iEnc}(\text{iMSK}, \text{pad}, \llbracket \tilde{\mathbf{p}}_{\text{pad}}^{\mathbf{v}_\ell} \rrbracket_1)} \\ \{\text{iCT}_{i,j}^{\mathbf{v}_\ell} &\leftarrow \boxed{\text{iEnc}(\text{iMSK}, (i, j), \llbracket \tilde{\mathbf{p}}_{i,j}^{\mathbf{v}_\ell} \rrbracket_1)}\}_{(i,j) \in \bar{J}_{\mathbf{v}_\ell}}, \end{aligned}$$

where $\sigma^0 = \sigma, \sigma^1 = 0$ and the encrypted vectors have an additional 0-coordinate taking the private input of Enc into account:

$$\begin{aligned} \tilde{\mathbf{p}}_{\text{pad}}^{\mathbf{v}_\ell} &= (\mathbf{r}^{\mathbf{v}_\ell}, \sigma^{b'}, \mathbf{0}_{\kappa-1}, \mathbf{0}) \\ \tilde{\mathbf{p}}_{i,j}^{\mathbf{v}_\ell} &= (0^{\bar{\mathbf{v}}_{\ell,i}[j] \cdot \kappa}, \mathbf{r}^{\mathbf{v}_\ell}, 0^{(1-\bar{\mathbf{v}}_{\ell,i}[j]) \cdot \kappa}, \mathbf{0}). \end{aligned}$$

We have $\widehat{G}_0^{b'} \equiv \widehat{G}_1^{b'}$ which follows from the slot-mode correctness of iFE.

Game $\widehat{G}_2^{b'}$: We define the set $K_\ell \subseteq [0; n-1] \times [0; k]$ containing the indices of all ciphertexts $\text{iCT}_{i,j}^{\mathbf{v}_\ell}$ known to the adversary:

$$K_\ell = \{(i, j) \in \bar{J}_{\mathbf{v}_\ell} : [i \in \mathcal{C} \cup \{0\}] \vee [\exists(i, L, \mathbf{x}_i) \in \mathcal{Q}_{\text{akey}} \text{ s.t. } \mathbf{x}_i \in X_{\mathbf{v}_\ell, i}]\}.$$

This game is the same as $\widehat{G}_1^{b'}$ except that the challenger sets $\tilde{\mathbf{p}}_{i,j}^{\mathbf{v}_\ell} = (\mathbf{0}_{2\kappa}, 0)$ if $(i, j) \notin K_\ell$. We have $\widehat{G}_1^{b'} \approx_c \widehat{G}_2^{b'}$ from the security of idFE.

Game $\widehat{G}_3^{b'}$: Next we deal with the ciphertexts $\text{iCT}_{i,j}^{\mathbf{v}_\ell}$ such that $(i, j) \in K_\ell$. To reply to the encryption query, the challenger now sets

$$\begin{aligned} \tilde{\mathbf{p}}_{\text{pad}}^{\mathbf{v}_\ell} &= (\mathbf{0}_\kappa, 0, \mathbf{0}_{\kappa-1}, \mathbf{1}) \\ \tilde{\mathbf{p}}_{i,j}^{\mathbf{v}_\ell} &= \begin{cases} (\mathbf{0}_\kappa, \mathbf{0}_\kappa, \mathbf{1}) & \text{if } (i, j) \in K_\ell \\ (\mathbf{0}_\kappa, \mathbf{0}_\kappa, 0) & \text{if } (i, j) \notin K_\ell. \end{cases} \end{aligned}$$

When receiving a query $\text{QPKeyGen}(f)$, the challenger samples a fresh secret $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^\kappa$, generates shares $\{\mathbf{s}_{i,j}^\beta\}_{\substack{\beta \in \{0,1\} \\ (i,j) \in \bar{J}_f}} \leftarrow \text{Share}(c \wedge f, \mathbf{s})$ and defines

$$\begin{aligned} \mathbf{q}_{\text{pad}} &= (\mathbf{s}, \mathbf{1}, \mathbf{0}_{\kappa-1}, \mathbf{s} \cdot (\mathbf{r}^{\mathbf{v}_\ell})^\top + \sigma^{b'}) \\ \mathbf{q}_{i,j} &= \begin{cases} (\mathbf{s}_{i,j}^0, \mathbf{s}_{i,j}^1, \mathbf{s}_{i,j}^{\bar{\mathbf{v}}_{\ell,i}[j]} \cdot (\mathbf{r}^{\mathbf{v}_\ell})^\top) & \text{if } (i, j) \in \bar{J}_f \cap K_\ell \\ (\mathbf{s}_{i,j}^0, \mathbf{s}_{i,j}^1, 0) & \text{if } (i, j) \in \bar{J}_f \setminus K_\ell, \end{cases} \end{aligned}$$

where $\mathbf{r}^{\mathbf{v}_\ell} \xleftarrow{\$} \mathbb{Z}_p^\kappa$ is sampled at the outset of the game and used in all queries to QPKeyGen . As the public parts of $\mathbf{q}_{\text{pad}}$, $\{\mathbf{q}_{i,j}\}_{i,j}$ and the inner products $\tilde{\mathbf{p}}_{\text{pad}}^{\mathbf{v}_\ell} \mathbf{q}_{\text{pad}}^\top$, $\{\tilde{\mathbf{p}}_{i,j}^{\mathbf{v}_\ell} \mathbf{q}_{i,j}^\top\}_{i,j}$ do not change, it follows $\widehat{G}_2^{b'} \approx_c \widehat{G}_3^{b'}$ from the function-hiding security of iFE.

Game $\widehat{G}_4^{b'}$: This game is the same as $\widehat{G}_3^{b'}$ except that, upon receiving a query $\text{QPKeyGen}(f)$, the challenger samples $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^\kappa$, $\mathbf{t} \xleftarrow{\$} \mathbb{Z}_p^{\kappa(m-1)}$, defines

$$\hat{\mathbf{s}} = \mathbf{s} \cdot (\mathbf{r}^{\mathbf{v}_\ell})^\top \in \mathbb{Z}_p, \quad \hat{\mathbf{t}} = \mathbf{t} \cdot (\mathbf{I}_{m-1} \otimes (\mathbf{r}^{\mathbf{v}_\ell})^\top) \in \mathbb{Z}_p^{m-1},$$

and computes the vectors $\mathbf{q}_{\text{pad}}$ and $\{\mathbf{q}_{i,j}\}_{i,j}$ as follows:

$$\begin{aligned} \{\mathbf{s}_{i,j}^\beta\}_{\substack{\beta \in \{0,1\} \\ (i,j) \in \bar{J}_f}} &\leftarrow \text{Share}(c \wedge f, \mathbf{s}; \mathbf{t}), \quad \{\hat{\mathbf{s}}_{i,j}^\beta\}_{\substack{\beta \in \{0,1\} \\ (i,j) \in \bar{J}_f}} \leftarrow \text{Share}(c \wedge f, \hat{\mathbf{s}}; \hat{\mathbf{t}}), \\ \mathbf{q}_{\text{pad}} &= (\mathbf{s}, \mathbf{1}, \mathbf{0}_{\kappa-1}, \hat{\mathbf{s}} + \sigma^{b'}), \\ \mathbf{q}_{i,j} &= \begin{cases} (\mathbf{s}_{i,j}^0, \mathbf{s}_{i,j}^1, \hat{\mathbf{s}}_{i,j}^{\bar{\mathbf{v}}_{\ell,i}[j]}) & \text{if } (i, j) \in \bar{J}_f \cap K_\ell \\ (\mathbf{s}_{i,j}^0, \mathbf{s}_{i,j}^1, 0) & \text{if } (i, j) \in \bar{J}_f \setminus K_\ell. \end{cases} \end{aligned}$$

We have $\widehat{G}_3^{b'} \equiv \widehat{G}_4^{b'}$ as the modifications are purely syntactical.

Game $\widehat{G}_5^{b'}$: This game is the same as $\widehat{G}_4^{b'}$ except that, upon receiving a query $\text{QPKeyGen}(f)$, the challenger samples $\hat{\mathbf{s}} \xleftarrow{\$} \mathbb{Z}_p$ and $\hat{\mathbf{t}} \xleftarrow{\$} \mathbb{Z}_p^{m-1}$ at random. We have that $\widehat{G}_4^{b'} \approx_c \widehat{G}_5^{b'}$ under the MDDH_κ assumption in $\mathbb{G}_2$. For the simulation of the games, we recall the fact that the Share algorithm of the garbling scheme can also take $(\llbracket \hat{\mathbf{s}} \rrbracket_2, \llbracket \hat{\mathbf{t}} \rrbracket_2)$ encoded in $\mathbb{G}_2$, run the sharing algorithm in the exponent and output $\{\llbracket \hat{\mathbf{s}}_{i,j}^\beta \rrbracket_2\}_{\substack{\beta \in \{0,1\} \\ (i,j) \in \bar{J}_f}}$.

Game $\widehat{G}_6^{b'}$: This game is the same as $\widehat{G}_5^{b'}$ except that, upon receiving a query $\text{QPKeyGen}(f)$, the challenger computes

$$\{\hat{\mathbf{s}}_{i,j}^\beta\}_{\substack{\beta \in \{0,1\} \\ (i,j) \in \bar{J}_f}} \leftarrow \text{Sim}(c \wedge f; \hat{\mathbf{t}}).$$

It follows from the adversary's admissibility condition and the security property of succinctly enumerable function classes that the adversary does not hold a subset of $\{\hat{\mathbf{s}}_{i,j}^\beta\}_{i,j}$ which enables the reconstruction of $\hat{\mathbf{s}}$. Therefore, we can conclude from the security of the garbling scheme that $\widehat{G}_5^{b'} \approx_s \widehat{G}_6^{b'}$.

Game $\widehat{G}_7^{b'}$: This game is the same as $\widehat{G}_6^{b'}$ except that, when receiving a query $\text{QPKeyGen}(f)$, the challenger sets

$$\mathbf{q}_{\text{pad}} = (\mathbf{s}, 1, \mathbf{0}_{\kappa-1}, \widehat{\mathbf{s}} + \sigma^{b'}) .$$

We note that the random element $\widehat{\mathbf{s}} \xleftarrow{\$} \mathbb{Z}_p$ appears only in $\mathbf{q}_{\text{pad}}$, so it perfectly masks $\sigma^{b'}$ and we obtain that $\widehat{G}_6^{b'} \equiv \widehat{G}_7^{b'}$. Furthermore, we can observe that the challenger's output in G_7^β is independent of the bit b' which implies that $\widehat{G}_7^0 \equiv \widehat{G}_7^1$.

□

4.3 Construction from Evasive IPFE

We present our construction of MC-ABE based on evasive IPFE.

Construction 26 (MC-ABE from Evasive IPFE). Let $d, k, n \in \mathbb{N}$ and $\mathcal{F} \subseteq \mathcal{F}_{kn,d}^{\text{circ}}$ be a succinctly enumerable class of functions. Our construction uses the following ingredients:

- an evasive identity-based IPFE scheme $\text{iFE} = (\text{iSetup}, \text{iEnc}, \text{iKeyGen}, \text{iDec})$ with identity space $([0; n-1] \times [0; k]) \cup \{0\}$,
- an identity-based encryption scheme $\text{idFE} = (\text{idSetup}, \text{idEnc}, \text{idKeyGen}, \text{idDec})$ with identity space $\mathcal{L} \times \{0, 1, \star\}^{[k]}$ for $\mathcal{L} = \{0, 1\}^{\text{poly}(\lambda)}$ and message space being the ciphertext space of iFE, and
- a noisy linear garbling scheme $\text{NLG} = (\text{gSetup}, \text{gGenF}, \text{gEval})$ supporting $\mathcal{F}_{kn,d}^{\text{circ}}$.

The MC-ABE scheme ABE for the predicate $P_{\mathcal{F}}$ with message space $\mathcal{M} = \{0, 1\}$ and label space $\mathcal{L}$ works as follows:

$\text{Setup}(1^\lambda)$ takes as input the security parameter 1^λ and generates

$$(\text{iMPK}, \text{iMSK}) \leftarrow \text{iSetup}(1^\lambda, 1^{2n_r}), \quad \{(\text{idMPK}_i, \text{idMSK}_i) \leftarrow \text{idSetup}(1^\lambda)\}_{i \in [n-1]}, \quad \text{gPP} \leftarrow \text{gSetup}(1^\lambda).$$

Then it outputs $(\text{MPK}, \text{MSK}, \{\text{SK}_i\}_{i \in [n-1]})$ as follows:

$$\text{MPK} = (\text{gPP}, \text{iMPK}, \{\text{idMPK}_i\}_{i \in [n-1]}), \quad \{\text{SK}_i = \text{idMSK}_i\}_{i \in [n-1]}, \quad \text{MSK} = \text{iMSK}.$$

$\text{Enc}(\text{MPK}, L, \mathbf{x}_0, \mu)$ takes $\text{MPK} = (\text{iMPK}, \{\text{idMPK}_i\}_{i \in [n-1]})$, a label $L \in \mathcal{L}$, an attribute $\mathbf{x}_0 \in \{0, 1\}^k$ and a message $\mu \in \{0, 1\}$ as input. The algorithm picks a sufficiently large message encoding scalar $r_{\text{msg}} \xleftarrow{\$} \mathbb{Z}_p \setminus [-2 \cdot 2^{-\varepsilon} p; 2 \cdot 2^{-\varepsilon} p]$ and secret vectors $\mathbf{r}^{\mathbf{v}} \xleftarrow{\$} \mathbb{Z}_p^{n_r}$ for each $\mathbf{v} \in V$, sets $\mathbf{v}_0 = \mathbf{x}_0$ and

$$\{\mathbf{p}_{\text{msg}}^{\mathbf{v}} = (\mathbf{r}^{\mathbf{v}}, \mu r_{\text{msg}}, \mathbf{0}_{n_r-1})\}_{\mathbf{v} \in V}, \quad \{\mathbf{p}_{i,j}^{\mathbf{v}} = (\mathbf{r}^{\mathbf{v}}, \bar{\mathbf{v}}_i[j] \cdot \mathbf{r}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_{\mathbf{v}}^{\mathbf{v}} = \{(\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V\}}.$$

Then it generates iFE and idFE ciphertexts

$$\begin{aligned} \{\text{iCT}_{\text{msg}}^{\mathbf{v}} &\leftarrow \text{iEnc}(\text{iMPK}, 0, \mathbf{p}_{\text{msg}}^{\mathbf{v}})\}_{\mathbf{v} \in V}, \\ \{\text{iCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{iEnc}(\text{iMPK}, (i, j), \mathbf{p}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_{\mathbf{v}}^{\mathbf{v}}}, \\ \{\text{idCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{idEnc}(\text{idMPK}_i, (L, \mathbf{v}_i), \text{iCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_{\mathbf{v}}^{\mathbf{v}}, i > 0}^{\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V}, \end{aligned}$$

and outputs the ciphertext

$$\text{CT}_L = (\{\text{iCT}_{\text{msg}}^{\mathbf{v}}\}_{\mathbf{v} \in V}, \{\text{iCT}_{0,j}^{\mathbf{v}}\}_{(0,j) \in \bar{J}_{\mathbf{v}}^{\mathbf{v}}}, \{\text{idCT}_{i,j}^{\mathbf{v}}\}_{(i,j) \in \bar{J}_{\mathbf{v}}^{\mathbf{v}}, i > 0}^{\mathbf{v} \in V}).$$

$\text{AKeyGen}(\text{SK}_i, L, \mathbf{x}_i)$ takes as input $\text{SK}_i = \text{idMSK}_i$ for some $i \in [n-1]$, a label $L \in \mathcal{L}$ and an attribute $\mathbf{x}_i \in \{0, 1\}^k$, computes

$$\{\text{idDK}_i^{\mathbf{v}_i} \leftarrow \text{idKeyGen}(\text{idMSK}_i, (L, \mathbf{v}_i))\}_{\mathbf{v}_i \in V_{\mathbf{x}_i}}$$

and outputs $\text{DK}_{L,i} = \{\text{idDK}_i^{\mathbf{v}_i}\}_{\mathbf{v}_i \in V_{\mathbf{x}_i}}$.

$\text{PKeyGen}(\text{MSK}, f)$ takes as input $\text{MSK} = \text{iMSK}$ and a policy $f \in \mathcal{F}$, generates a noisy linear garbling of $c \wedge f$ by

$$(1^{n_f}, \mathbf{w}_{\text{out}}, \{\mathbf{W}_{i,j}^0, \mathbf{W}_{i,j}^\times\}_{(i,j) \in \bar{J}_f}, \mathbf{T}, \mathbf{R}) \leftarrow \text{gGenF}(\text{gPP}, c \wedge f),$$

where $c \wedge f$ denotes the complete policy viewed as a function $\{0, 1\}^{\bar{J}_f} \rightarrow \{0, 1\}$, and samples a random public matrix $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_p^{n_r \times n_f}$. Then the algorithm computes

$$\begin{aligned} \mathbf{q}_{\text{msg}} &= (\mathbf{B}\mathbf{w}_{\text{out}}^\top)^\top, 1, \mathbf{0}_{n_r-1}), \quad \{\mathbf{Q}_{i,j} = (\mathbf{B}\mathbf{W}_{i,j}^0)^\top, (\mathbf{B}\mathbf{W}_{i,j}^\times)^\top\}_{(i,j) \in \bar{J}_f}, \\ \mathbf{Q}_t &= (\mathbf{B}\mathbf{T})^\top, \mathbf{0}_{\star \times n_r}), \end{aligned}$$

and generates iFE decryption keys

$$\begin{aligned} \text{idK}_{\text{msg}} &\leftarrow \text{iKeyGen}(\text{iMSK}, 0, \mathbf{q}_{\text{msg}}), \\ \text{idK}_t &\leftarrow \text{iKeyGen}(\text{iMSK}, 0, \mathbf{Q}_t), \\ \{\text{idK}_{i,j} &\leftarrow \text{iKeyGen}(\text{iMSK}, (i, j), \mathbf{Q}_{i,j})\}_{(i,j) \in \bar{J}_f}. \end{aligned}$$

Finally, it outputs

$$\text{DK}_f = (\text{idK}_{\text{msg}}, \text{idK}_t, \{\text{idK}_{i,j}\}_{(i,j) \in \bar{J}_f}).$$

$\text{Dec}(\text{DK}_f, \{\text{DK}_{L,i}\}_{i \in [n-1]}, \text{CT}_L)$ takes as input a decryption key DK_f for a policy $f \in \mathcal{F}$, a set of decryption keys $\{\text{DK}_{L,i}\}_{i \in [n-1]}$ created with respect to attributes $\mathbf{x}_1, \dots, \mathbf{x}_{n-1} \in \{0, 1\}^k$ and a label $L \in \mathcal{L}$, and a ciphertext CT_L created with respect to an attribute $\mathbf{x}_0 \in \{0, 1\}^k$ and the same label L . If $f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) = 1$, return $\perp$. Otherwise, parse DK_f , $\{\text{DK}_{L,i}\}_{i \in [n-1]}$ and CT_L as above. The algorithm picks $\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_{n-1}) \in V$ such that $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in X_{\mathbf{v}}$ and $J_{\mathbf{v}} = J_f$. Subsequently, it recovers w_{msg} , $\mathbf{t}$ and $\{\mathbf{w}_{i,j}\}_{(i,j) \in \bar{J}_{\mathbf{v}}}$ by decrypting

$$\begin{aligned} w_{\text{msg}} &\leftarrow \text{idDec}(\text{idK}_{\text{msg}}, \text{idCT}_{\text{msg}}^{\mathbf{v}}), & \{\text{idCT}_{i,j}^{\mathbf{v}} &\leftarrow \text{idDec}(\text{idDK}_i^{\mathbf{v}}, \text{idCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_{\mathbf{v}}, i > 0}, \\ \mathbf{t} &\leftarrow \text{idDec}(\text{idK}_t, \text{idCT}_t^{\mathbf{v}}), & \{\mathbf{w}_{i,j} &\leftarrow \text{idDec}(\text{idK}_{i,j}, \text{idCT}_{i,j}^{\mathbf{v}})\}_{(i,j) \in \bar{J}_{\mathbf{v}}}. \end{aligned}$$

It evaluates the garbling

$$w_{\text{out}} \leftarrow \text{gEval}(\text{gPP}, f, \mathbf{R}, \{\mathbf{x}_i[j]\}_{i,j \in \bar{J}_{\mathbf{v}}}, \{\mathbf{w}_{i,j}\}_{i,j \in \bar{J}_{\mathbf{v}}}, \mathbf{t}),$$

and outputs the decryption result

$$\mu' \leftarrow \begin{cases} 0 & \text{if } w_{\text{msg}} - w_{\text{out}} \in [-2^{-\varepsilon} p; 2^{-\varepsilon} p], \\ 1 & \text{otherwise.} \end{cases}$$

Correctness. The construction is correct for a suitable choice of parameters.

Proposition 27. *Construction 26 is correct if*

- idFE is correct,
- NLG is $(B_{\text{in}}, B_{\text{out}})$ -correct for $B_{\text{out}} \leq 2^{-\varepsilon} p$, and
- iFE is B_{ipfe} -correct for some $B_{\text{ipfe}} \leq B_{\text{in}}$.

Proof. Let $f \in \mathcal{F}$ and $(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) \in \{0, 1\}^{[0;n-1] \times [k]}$ such that $f(\mathbf{x}_0, \dots, \mathbf{x}_{n-1}) = 0$. The correctness property of succinctly enumerable function classes guarantees the existence of a vector $\mathbf{v} \in V$ such that $J_{\mathbf{v}} = J_f$ and $(\mathbf{x}_1, \dots, \mathbf{x}_{n-1}) \in X_{\mathbf{v}}$. This implies that all idFE ciphertexts $\{\text{idCT}_{i,j}^{\mathbf{v}}\}_{(i,j) \in \bar{J}_{\mathbf{v}}, i > 0}$ and secret keys $\{\text{idDK}_{i,j}\}_{(i,j) \in \bar{J}_{\mathbf{v}}, i > 0}$

used in the first step of decryption are contained in CT_L and $\{\text{DK}_{L,i}\}_{i \in [n-1]}$, respectively. From the correctness of idFE and iFE, it follows that

$$\begin{aligned} w_{\text{msg}} &= \mathbf{p}_{\text{msg}}^{\mathbf{v}} \mathbf{q}_{\text{msg}}^{\top} + e_{\text{out}} = \mathbf{r}^{\mathbf{v}} \mathbf{B} \mathbf{w}_{\text{out}}^{\top} + \mu r_{\text{msg}} + e_{\text{out}}, \\ \mathbf{t} &= \mathbf{p}_{\text{msg}}^{\mathbf{v}} \mathbf{Q}_{\mathbf{t}}^{\top} + \mathbf{e}_{\mathbf{t}} = \mathbf{r}^{\mathbf{v}} \mathbf{B} \mathbf{T} + \mathbf{e}_{\mathbf{t}}, \\ \mathbf{w}_{i,j} &= \mathbf{p}_{i,j}^{\mathbf{v}} \mathbf{Q}_{i,j}^{\top} + \mathbf{e}_{i,j} = \mathbf{r}^{\mathbf{v}} \mathbf{B} (\mathbf{W}_{i,j}^0 + \bar{\mathbf{v}}_i[j] \mathbf{W}_{i,j}^{\times}) + \mathbf{e}_{i,j} \quad \text{for each } (i, j) \in \bar{J}_{\mathbf{v}}, \end{aligned}$$

where $\mathbf{r}^{\mathbf{v}} \mathbf{B}$ randomizes the garbling, and the errors from iFE decryption are bounded by $B_{\text{ipfe}} \leq B_{\text{in}}$. By the $(B_{\text{in}}, B_{\text{out}})$ -correctness of garbling, we have

$$w_{\text{out}} - \mathbf{r}^{\mathbf{v}} \mathbf{B} \mathbf{w}_{\text{out}}^{\top} + e_{\text{out}} \in [-B_{\text{out}}; B_{\text{out}}] \subseteq [-2^{-\varepsilon} p; 2^{-\varepsilon} p].$$

If $\mu = 0$, it always holds that $w_{\text{msg}} - w_{\text{out}} \in [-2^{-\varepsilon} p; 2^{-\varepsilon} p]$, which is never the case if $\mu = 1$ by the choice of r_{msg} . This concludes that Construction 26 is correct. $\square$

Security. We prove very selective security.

Proposition 28. *Construction 26 is stat-secure if*

- iFE is restricted- σ_{ipfe} -secure (as per Definition 15),
- NLG is B_{short} -short (as per Definition 9) and GenNoise-secure (as per Definition 10) such that the output of GenNoise is in $[-B_{\text{nlg}}; B_{\text{nlg}}]$ and $2^{\varepsilon+6} B_{\text{nlg}} \leq \sigma_{\text{ipfe}}$, and
- $\text{LWE}_{n_r, \text{poly}(\lambda), p, \sigma_r}$ (as per Definition 7) holds for some $\sigma_r \leq 2^{-\varepsilon-6} / (\sqrt{\varepsilon} \cdot B_{\text{short}})$.

When instantiating iFE and NLG with the evasive IFPE and noisy linear garbling from [HLL24], and idFE with the IBE from [ABB10], we obtain the following theorem.

Theorem 29. *Assuming the hardness of LWE with subexponential modulus-to-noise ratio and (public-coin) evasive LWE with public-coin samplers (in the variant of [HLL24]), there exists a very selectively secure MC-ABE for each succinctly enumerable policy classes $\mathcal{F} \subseteq \mathcal{F}_{\text{kn},d}^{\text{circ}}$.*

4.4 Proof of Security

Proof of Proposition 28. We consider a sequence of hybrid games $G_0^0, \dots, G_3^0, G_3^1, \dots, G_0^1$, where $G_0^b = \text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b}$ for $b \in \{0, 1\}$. We argue $G_{\alpha-1}^b \approx_c G_{\alpha}^b$, for $\alpha \in [3]$, and $G_3^0 \equiv G_3^1$. Then the claim follows via a hybrid argument.

Game G_0^b : This is $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b}$. We let $q_v = |V|$ and parse $V = \{\mathbf{v}_{\ell}\}_{\ell \in [q_v]}$, where $\mathbf{v}_{\ell} = (\mathbf{v}_{\ell,1}, \dots, \mathbf{v}_{\ell,n-1})$ for each $\ell \in [q_v]$. For notational convenience, we also set $\mathbf{v}_{\ell,0} = \mathbf{x}_0$. For the reply to the challenge encryption query $\text{QEnc}(L, \mathbf{x}_0, \mu^0, \mu^1)$, the challenger samples a sufficiently large message encoding scalar $r_{\text{msg}} \xleftarrow{\$} \mathbb{Z}_p \setminus [-2 \cdot 2^{-\varepsilon} p; 2 \cdot 2^{-\varepsilon} p]$ and secret vectors $\mathbf{r}^{\mathbf{v}_{\ell}} \xleftarrow{\$} \mathbb{Z}_p^{n_r}$ for each $\ell \in [q_v]$. Then it sets

$$\{\mathbf{p}_{\text{msg}}^{\mathbf{v}_{\ell}} = (\mathbf{r}^{\mathbf{v}_{\ell}}, \mu^b r_{\text{msg}}, \mathbf{0}_{n_r-1})\}_{\ell \in [q_v]}, \quad \{\mathbf{p}_{i,j}^{\mathbf{v}_{\ell}} = (\mathbf{r}^{\mathbf{v}_{\ell}}, \bar{\mathbf{v}}_{\ell,i}[j] \cdot \mathbf{r}^{\mathbf{v}_{\ell}})\}_{\ell \in [q_v], (i,j) \in \bar{J}_{\mathbf{v}_{\ell}}},$$

and generates

$$\begin{aligned} \{\text{idCT}_{\text{msg}}^{\mathbf{v}_{\ell}} &\leftarrow \text{idEnc}(\text{idMPK}, (0, \mathbf{v}_{\ell}), \mathbf{p}_{\text{msg}}^{\mathbf{v}_{\ell}})\}_{\ell \in [q_v]}, \\ \{\text{idCT}_{i,j}^{\mathbf{v}_{\ell}} &\leftarrow \text{idEnc}(\text{idMPK}, (i, j, \mathbf{v}_{\ell}), \mathbf{p}_{i,j}^{\mathbf{v}_{\ell}})\}_{\ell \in [q_v], (i,j) \in \bar{J}_{\mathbf{v}_{\ell}}}, \\ \{\text{idCT}_{i,j}^{\mathbf{v}_{\ell}} &\leftarrow \text{idEnc}(\text{idMPK}_i, (L, \mathbf{v}_{\ell,i}), \text{idCT}_{i,j}^{\mathbf{v}_{\ell}})\}_{\ell \in [q_v], (i,j) \in \bar{J}_{\mathbf{v}_{\ell}}, i > 0}. \end{aligned}$$

Finally, it sends the following challenge ciphertext CT_L to $\mathcal{A}$:

$$\text{CT}_L = (\{\text{idCT}_{\text{msg}}^{\mathbf{v}_{\ell}}\}_{\ell \in [q_v]}, \{\text{idCT}_{0,j}^{\mathbf{v}_{\ell}}\}_{\ell \in [q_v], (0,j) \in \bar{J}_{\mathbf{v}_{\ell}}}, \{\text{idCT}_{i,j}^{\mathbf{v}_{\ell,i}}\}_{\ell \in [q_v], (i,j) \in \bar{J}_{\mathbf{v}_{\ell}}, i > 0}).$$

Game G_1^b : We recall the definition of the set $K_\ell \subseteq [0; n-1] \times [0; k]$ containing the indices of all ciphertexts $\text{iCT}_{i,j}^{\mathbf{v}_\ell}$ known to the adversary:

$$K_\ell = \{(i, j) \in \bar{J}_{\mathbf{v}_\ell} : [i \in \mathcal{C} \cup \{0\}] \vee [\exists(i, L, \mathbf{x}_i) \in \mathcal{Q}_{\text{akey}} \text{ s.t. } \mathbf{x}_i \in X_{\mathbf{v}_\ell, i}]\}.$$

This game is the same as G_0^b except that, for all $\ell \in [q_v]$ and $(i, j) \in \bar{J}_{\mathbf{v}_\ell}$, the challenger sets $\text{iCT}_{i,j}^{\mathbf{v}_\ell} = \perp$ if $(i, j) \notin K_\ell$. We have $G_0^b \approx_c G_1^b$ from the security of idFE.

Game G_2^b : This game is the same as G_1^b except that the challenger samples the message encoding scalar $r_{\text{msg}} \xleftarrow{\$} \mathbb{Z}_p$. Clearly $G_1^b \approx_s G_2^b$.

Game G_3^b : This game is the same as G_2^b except that the challenger replaces vectors inside iFE ciphertexts by random. Clearly, $G_3^0 \equiv G_3^1$. The rest of the proof is devoted to showing $G_2^b \approx_c G_3^b$.

We want to rely on the restricted- σ_{ipfe} -security of iFE. To do so, we consider the following evasive IPFE sampler $\mathcal{S}^b = (\mathcal{S}_V, \mathcal{S}_U)$ for an efficient and admissible adversary $\mathcal{A}$.

- $\mathcal{S}_V(r_{\text{pub}})$ parses $r_{\text{pub}} = (r_{\mathcal{A}}, r_{\text{gSetup}}, r_{\text{gGenF}})$ and runs $\mathcal{A}(1^\lambda; r_{\mathcal{A}})$ to obtain the challenge $(L, \mathbf{x}_0, \mu^0, \mu^1)$, a set of attribute key queries $\mathcal{Q}_{\text{akey}} = \{(i, L, \mathbf{x}_i)\}$ and a set of policy key queries $\mathcal{Q}_{\text{pkey}} = \{f^{\ell'}\}_{\ell' \in [q_f]}$.

The algorithm sets up the noisy linear garbling by running

$$\text{gPP} \leftarrow \text{gSetup}(1^\lambda; r_{\text{gSetup}}),$$

and deterministically picks a suitable n_r . It runs

$$(1^{n_{f^{\ell'}}}, \mathbf{w}_{\text{out}}^{\ell'}, \{\mathbf{W}_{i,j}^{\ell',0}, \mathbf{W}_{i,j}^{\ell',\times}\}_{(i,j) \in \bar{J}_f}, \mathbf{T}^{\ell'}, \mathbf{R}^{\ell'}) \leftarrow \text{gGenF}(\text{gPP}, c \wedge f^{\ell'})$$

and samples $\mathbf{B}^{\ell'} \xleftarrow{\$} \mathbb{Z}_p^{n_r \times n_{f^{\ell'}}}$ for all $\ell' \in [q_f]$. Suppose $\mathbf{W}_{i,j}^{\ell',0} \in \mathbb{Z}_p^{n_{f^{\ell'}} \times m_{i,j}^{\ell'}}$ and $\mathbf{T}^{\ell'} \in \mathbb{Z}_p^{n_{f^{\ell'}} \times m_t^{\ell'}}$. Then the algorithm sets

$$\begin{aligned} N &= 2n_r, & \text{ID} &= \bigcup_{\ell \in [q_v]} \bar{J}_{\mathbf{v}_\ell} \cup \{0\}, \\ (\text{id}, I_{\text{id}}) &= \begin{cases} ([q_v], |\text{id}|) & \text{if id} = 0 \\ (\{\ell \in [q_v] : \text{id} \in \bar{J}_{\mathbf{v}_\ell}\}, |\text{id}|) & \text{if id} \in \text{ID} \setminus \{0\}, \end{cases} \\ (\text{jid}, J_{\text{id}}) &= \begin{cases} ([q_f], \sum_{\ell' \in \text{id}} (m_t^{\ell'} + 1)) & \text{if id} = 0 \\ (\{\ell' \in [q_f] : \text{id} \in \bar{J}_{f^{\ell'}}\}, \sum_{\ell' \in \text{id}} m_t^{\ell'}) & \text{if id} \in \text{ID} \setminus \{0\}, \end{cases} \\ \mathbf{V}_0 &= \begin{bmatrix} \vdots \\ \text{---} \mathbf{q}_{\text{msg}}^{\ell'} \text{---} \\ \text{---} \mathbf{Q}_t^{\ell'} \text{---} \\ \vdots \end{bmatrix}_{\ell' \in \text{jid}_0} & \text{where} & \begin{aligned} \mathbf{q}_{\text{msg}}^{\ell'} &= ((\mathbf{B}^{\ell'} (\mathbf{w}_{\text{out}}^{\ell'})^\top)^\top, 1, \mathbf{0}_{n_r-1}), \\ \mathbf{Q}_t^{\ell'} &= ((\mathbf{B}^{\ell'} \mathbf{T}^{\ell'})^\top, \mathbf{0}_{\star \times n_r}), \end{aligned} \\ \mathbf{V}_{\text{id}} &= \begin{bmatrix} \vdots \\ \text{---} \mathbf{Q}_{\text{id}}^{\ell'} \text{---} \\ \vdots \end{bmatrix}_{\ell' \in \text{jid}} & \text{where} & \begin{aligned} \text{id} &\in \text{ID} \setminus \{0\}, \\ \mathbf{Q}_{\text{id}}^{\ell'} &= ((\mathbf{B}^{\ell'} \mathbf{W}_{i,j}^{\ell',0})^\top, (\mathbf{B}^{\ell'} \mathbf{W}_{i,j}^{\ell',\times})^\top), \end{aligned} \end{aligned}$$

and outputs $(1^N, \text{ID}, \{1^{I_{\text{id}}}, 1^{J_{\text{id}}}, \mathbf{V}_{\text{id}}\}_{\text{id} \in \text{ID}})$.

- $\mathcal{S}_U^b(r_{\text{pub}}; r_{\text{pri}})$ starts by running the deterministic subroutine $\mathcal{S}_{A0}^b(r_{\text{pub}})$ which does the following. First, it reruns $\mathcal{A}(1^\lambda; r_{\mathcal{A}})$ to obtain the challenge query $(L, \mathbf{x}_0, \mu^0, \mu^1)$. Then the algorithm then picks matrices $\{\mathbf{A}_{\text{id},0,\ell}\}_{\text{id} \in \text{ID}, \ell \in \mathbf{i}_{\text{id}}}$ such that

$$\begin{aligned} \mathbf{d}_0 \mathbf{A}_{0,0,\ell} &= \mathbf{p}_{\text{msg}}^{\mathbf{v}_\ell} = (\mathbf{r}^{\mathbf{v}_\ell}, \mu r_{\text{msg}}, \mathbf{0}_{n_r-1}) \\ \mathbf{d}_{i,j} \mathbf{A}_{(i,j),0,\ell} &= \mathbf{p}_{i,j}^{\mathbf{v}_\ell} = (\mathbf{r}^{\mathbf{v}_\ell}, \bar{\mathbf{v}}_{\ell,i}[j] \cdot \mathbf{r}^{\mathbf{v}_\ell}) \end{aligned}$$

where $\mathbf{d}_0 = (r_{\text{msg}}, \mathbf{r}^{\mathbf{v}_1}, \dots, \mathbf{r}^{\mathbf{v}_{q_v}})$ and $n' = n_r q_v + 1$, and outputs the tuple $(1^{n'}, \{\mathbf{A}_{\text{id},0,\ell}\}_{\text{id} \in \text{ID}, \ell \in \mathbf{i}_{\text{id}}})$. After completing $\mathcal{S}_{A0}^b$, the algorithm $\mathcal{S}_U^b$ sample $\mathbf{d}_0 \xleftarrow{\$} \mathbb{Z}_p^{n'}$ using r_{pri} and outputs

$$\mathbf{U}_{\text{id}} = \left[\begin{array}{c} \vdots \\ \text{--- } \mathbf{d}_0 \mathbf{A}_{\text{id},0,\ell} \text{ ---} \\ \vdots \end{array} \right]_{\ell \in \mathbf{i}_{\text{id}}} \quad \text{for all id} \in \text{ID}.$$

We observe that G_2^b and G_3^b correspond to $\text{IPFEsec}_{\text{pre}}^{\mathcal{S}_U^b}$ and $\text{IPFEsec}_{\text{post}}^{\mathcal{S}_U^b}$, respectively. It therefore follows $G_2^b \approx_c G_3^b$ from the restricted- σ_{ipfe} -security of iFE and the following claim for $b \in \{0, 1\}$.

Claim 30. $\mathcal{S}^b$ has pseudorandom noisy inner products, i.e., $\text{IPFEsec}_{\text{pre}}^{\mathcal{S}_U^b}$ holds.

This completes the proof of the proposition. $\square$

Proof of Claim 30. We consider a sequence of hybrids $\widehat{G}_0, \dots, \widehat{G}_5$, where $\widehat{G}_0$ and $\widehat{G}_5$ are the left and right distribution of $\text{IPFEsec}_{\text{pre}}^{\mathcal{S}_U^b}$, respectively. We argue $\widehat{G}_{\alpha-1} \approx_c \widehat{G}_\alpha$, for $\alpha \in [5]$. Then the claim follows via a hybrid argument. Modifications between consecutive games are highlighted using **boxes**.

Game $\widehat{G}_0$: This is the L.H.S. distribution of $\text{IPFEsec}_{\text{pre}}^{\mathcal{S}_U^b}$ (Definition 15). We recall that $r_{\text{pub}} = (r_{\mathcal{A}}, r_{\text{gSetup}}, r_{\text{gGenF}})$ and the inner products are of the form

$$\begin{aligned} w_{\text{msg}}^{\ell, \ell'} &= \mathbf{p}_{\text{msg}}^{\mathbf{v}_\ell} (\mathbf{q}_{\text{msg}}^{\ell'})^\top + e_{\text{out}}^{\ell, \ell'} \\ &= \mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} (\mathbf{w}_{\text{out}}^{\ell'})^\top + \mu r_{\text{msg}} + e_{\text{out}}^{\ell, \ell'}, & \text{for } \ell \in \mathbf{i}_0, \ell' \in \mathbf{j}_0 \\ \mathbf{t}^{\ell, \ell'} &= \mathbf{p}_{\text{msg}}^{\mathbf{v}_\ell} (\mathbf{Q}_{\text{t}}^{\ell'})^\top + \mathbf{e}_{\text{t}}^{\ell, \ell'} \\ &= \mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} \mathbf{T}^{\ell'} + \mathbf{e}_{\text{t}}^{\ell, \ell'}, & \text{for } \ell \in \mathbf{i}_0, \ell' \in \mathbf{j}_0 \\ \mathbf{w}_{i,j}^{\ell, \ell'} &= \mathbf{p}_{i,j}^{\mathbf{v}_\ell} (\mathbf{Q}_{i,j}^{\ell'})^\top + \mathbf{e}_{i,j}^{\ell, \ell'} \\ &= \mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} (\mathbf{W}_{i,j}^{\ell',0} + \bar{\mathbf{v}}_{\ell,i}[j] \mathbf{W}_{i,j}^{\ell',\times}) + \mathbf{e}_{i,j}^{\ell, \ell'}, & \text{for } \ell \in \mathbf{i}_{i,j}, \ell' \in \mathbf{j}_{i,j} \end{aligned}$$

where $(i, j) \in \text{ID} \setminus \{0\}$ and the entries of $\mathbf{e}$'s are independent $\mathcal{D}_{\mathbb{Z}, \sigma_{\text{ipfe}}}$.

Game $\widehat{G}_1$: This is the same as $\widehat{G}_0$ except that there are additional small noises attached to $\{\mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'}\}_{\ell \in [q_v]}$. For all $\ell \in [q_v]$, $\ell' \in [q_f]$, sample $\mathbf{e}_{\text{r}}^{\ell, \ell'} \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_r, \leq \sigma_r \sqrt{\varepsilon}}^{n_{f\ell'}}$ and set the inner products to

$$\begin{aligned} w_{\text{msg}}^{\ell, \ell'} &= (\mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} + \boxed{\mathbf{e}_{\text{r}}^{\ell, \ell'}}) (\mathbf{w}_{\text{out}}^{\ell'})^\top + \mu r_{\text{msg}} + e_{\text{out}}^{\ell, \ell'}, \\ \mathbf{t}^{\ell, \ell'} &= (\mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} + \boxed{\mathbf{e}_{\text{r}}^{\ell, \ell'}}) \mathbf{T}^{\ell'} + \mathbf{e}_{\text{t}}^{\ell, \ell'}, \\ \mathbf{w}_{i,j}^{\ell, \ell'} &= (\mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} + \boxed{\mathbf{e}_{\text{r}}^{\ell, \ell'}}) (\mathbf{W}_{i,j}^{\ell',0} + \bar{\mathbf{v}}_{\ell,i}[j] \mathbf{W}_{i,j}^{\ell',\times}) + \mathbf{e}_{i,j}^{\ell, \ell'}. \end{aligned}$$

The introduced errors are bounded by

$$B_{\text{short}} \cdot \sigma_r \sqrt{\varepsilon} \leq 2^{-\varepsilon-6} \sigma_{\text{ipfe}},$$

so they are flooded (Lemma 6) by $\{e_{\text{out}}^{\ell, \ell'}, \mathbf{e}_{\text{t}}^{\ell, \ell'}, \mathbf{e}_{i,j}^{\ell, \ell'}\}_{i,j}^{\ell, \ell'}$ which implies that $\widehat{G}_0 \approx_s \widehat{G}_1$.

Game $\widehat{G}_2$: This is the same as $\widehat{G}_1$ except that the $\mathbf{e}_r^{\ell, \ell'}$'s are no longer truncated, i.e., $\mathbf{e}_{r, \ell} \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_r}^{n_{f^{\ell'}}$. We have $\widehat{G}_1 \approx_s \widehat{G}_2$ by Lemma 5.

Game $\widehat{G}_3$: This is the same as $\widehat{G}_2$ except that the garblings use uniform randomness. For all $\ell \in [q_v]$, $\ell' \in [q_f]$, sample $\mathbf{s}^{\ell, \ell'} \xleftarrow{\$} \mathbb{Z}_p^{n_{f^{\ell'}}$ and set the inner products to

$$\begin{aligned} w_{\text{msg}}^{\ell, \ell'} &= \mathbf{s}^{\ell, \ell'} (\mathbf{w}_{\text{out}}^{\ell'})^\top + \mu r_{\text{msg}} + e_{\text{out}}^{\ell, \ell'}, \\ \mathbf{t}^{\ell, \ell'} &= \mathbf{s}^{\ell, \ell'} \mathbf{T}^{\ell'} + \mathbf{e}_t^{\ell, \ell'}, \\ \mathbf{w}_{i, j}^{\ell, \ell'} &= \mathbf{s}^{\ell, \ell'} (\mathbf{W}_{i, j}^{\ell', 0} + \bar{\mathbf{v}}_{\ell, i}[j] \mathbf{W}_{i, j}^{\ell', \times}) + \mathbf{e}_{i, j}^{\ell, \ell'}, \end{aligned}$$

By $[q_v]$ hybrids of applying $\text{LWE}_{n_r, \text{poly}(\lambda), p, \sigma_r}$ over $\{\mathbf{v}_\ell\}_{\ell \in [q_v]}$ with public matrix $(\mathbf{B}^1, \dots, \mathbf{B}^{q_f})$, we have

$$\{\mathbf{r}^{\mathbf{v}_\ell} \mathbf{B}^{\ell'} + \mathbf{e}_r^{\ell, \ell'}\}^{\ell, \ell'} \approx_c \{\mathbf{s}^{\mathbf{v}_\ell}\}^{\ell, \ell'},$$

which implies $\widehat{G}_2 \approx_s \widehat{G}_3$.

Game $\widehat{G}_4$: This is the same as $\widehat{G}_3$ except that there are additional noises generated by GenNoise attached to the garblings. For all $\ell \in [q_v]$, $\ell' \in [q_f]$, run

$$\begin{pmatrix} \bar{\mathbf{e}}_{\text{out}}^{\ell, \ell'}, \bar{\mathbf{e}}_t^{\ell, \ell'} \\ (\bar{\mathbf{e}}_{i, j}^{\ell, \ell'})_{(i, j) \in \bar{J}_{f^{\ell'}}} \end{pmatrix} \leftarrow \text{GenNoise} \left(\begin{pmatrix} \text{gPP}, f^{\ell'}, \mathbf{w}_{\text{out}}^{\ell'}, \{\mathbf{W}_{i, j}^{\ell', 0}, \mathbf{W}_{i, j}^{\ell', \times}\}_{(i, j) \in \bar{J}_{f^{\ell'}}} \\ \mathbf{T}^{\ell'}, \mathbf{R}^{\ell'}, (\mathbf{v}_{\ell, i}[j])_{(i, j) \in \bar{J}_{f^{\ell'}}} \end{pmatrix} \right),$$

where GenNoise can interpret $\star$ -coordinates of $\mathbf{v}^\ell$ as 0 since these coordinates are irrelevant for $f^{\ell'}$. Then set the inner products to

$$\begin{aligned} w_{\text{msg}}^{\ell, \ell'} &= \mathbf{s}^{\ell, \ell'} (\mathbf{w}_{\text{out}}^{\ell'})^\top + \bar{\mathbf{e}}_{\text{out}}^{\ell, \ell'} + \mu r_{\text{msg}} + e_{\text{out}}^{\ell, \ell'}, \\ \mathbf{t}^{\ell, \ell'} &= \mathbf{s}^{\ell, \ell'} \mathbf{T}^{\ell'} + \bar{\mathbf{e}}_t^{\ell, \ell'} + \mathbf{e}_t^{\ell, \ell'}, \\ \mathbf{w}_{i, j}^{\ell, \ell'} &= \mathbf{s}^{\ell, \ell'} (\mathbf{W}_{i, j}^{\ell', 0} + \bar{\mathbf{e}}_{i, j}^{\ell, \ell'} + \bar{\mathbf{v}}_{\ell, i}[j] \mathbf{W}_{i, j}^{\ell', \times}) + \mathbf{e}_{i, j}^{\ell, \ell'}. \end{aligned}$$

The introduced $\bar{e}, \bar{\mathbf{e}}$'s are bounded by $B_{\text{NLG}} \leq 2^{-\varepsilon-6} \sigma_{\text{ipfe}}$, so they are flooded (Lemma 6) by their counterparts $e, \mathbf{e}$ which implies $\widehat{G}_3 \approx_s \widehat{G}_4$.

Game $\widehat{G}_5$: This is the same as $\widehat{G}_4$ except that all inner products are replaced by random, i.e., the right distribution of $\text{IPFSec}_{\text{pre}}^{S^b}$. We note that in $\widehat{G}_4$ all inner products are well-randomized garblings (by $\mathbf{s}, \bar{\mathbf{e}}, \bar{\mathbf{e}}$) plus independent noises ($e, \mathbf{e}$'s). Therefore, by applying GenNoise-security of NLG in a sequence of hybrids over all $(\ell, \ell') \in [q_v] \times [q_f]$, we conclude that $\widehat{G}_4 \approx_c \widehat{G}_5$. □

5 MC-ABE for Conjunction Predicates

In this section, we present our construction of secret-key MC-ABE for conjunctions of a predicate P .

Definition 31 (Conjunctions of Predicates). Let $\{P_\lambda\}_{\lambda \in \mathbb{N}}$, $\{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of predicates, ciphertext attribute spaces and key attribute spaces, respectively. For $n \in \mathbb{N}$, we denote $\{P_{\lambda, n}\}_{\lambda \in \mathbb{N}}$ the sequence of conjunction predicates, where $P_{\lambda, n}: (\mathcal{X}_\lambda \cup \{\star\})^{[0; n-1]} \times \mathcal{Y}_\lambda^{[0; n-1]} \rightarrow \{0, 1\}$ is defined as

$$P_{\lambda, n}((x_0, \dots, x_{n-1}), (y_0, \dots, y_{n-1})) = P_\lambda^\star(x_0, y_0) \wedge \dots \wedge P_\lambda^\star(x_{n-1}, y_{n-1}),$$

where $P_\lambda^\star(x, y) = P_\lambda(x, y)$ if $x \in \mathcal{X}_\lambda$ and $P_\lambda^\star(\star, y) = 0$ for each $y \in \mathcal{Y}_\lambda$.

5.1 Construction

Construction 32 (MC-ABE for Conjunction Predicates). Our construction uses the following ingredients:

- an AB-IPFE scheme $\text{iFE} = (\text{iSetup}, \text{iKeyGen}, \text{iEnc}, \text{iDec})$ for a predicate $P^*: (\mathcal{X} \cup \{\star\}) \times \mathcal{Y} \rightarrow \{0, 1\}$ which provides

either $\boxed{\text{function-hiding security based on a pairing group } \mathbb{G}}$,
or $\boxed{\text{standard security against bounded collusions}}$,

- a family of PRFs $\{\text{PRF}_K\}_{K \in \mathcal{K}}$ where $\text{PRF}_K: (\mathcal{L} \times [\kappa]) \rightarrow \mathbb{Z}_p$ for each key $K \in \mathcal{K}$, and
- a hash function $\boxed{H: \mathbb{G}_t \rightarrow \mathcal{M}}$ $\boxed{H: \mathbb{Z}_p \rightarrow \mathcal{M}}$ for $\mathcal{M} = \{0, 1\}^{\text{poly}(\lambda)}$.

If iFE provides function-hiding security, we let κ be the parameter of the MDDH_κ assumption which we want to rely on during the security proof. If iFE provides standard security against Q key queries, then we choose the parameters κ and τ according to Lemma 20. For notational convenience, we assume that iFE is a secret-key AB-IPFE scheme. But this is clearly without loss of generality; if iFE is a public-key scheme, we may simply concatenate iMPK and iMSK and pass this concatenation to both iEnc and iKeyGen.

The MC-ABE scheme ABE for conjunctions of P with message space $\mathcal{M}$ and label space $\mathcal{L}$ works as follows:

$\text{Setup}(1^\lambda)$ takes as input 1^λ and samples $K_i \xleftarrow{\$} \mathcal{K}$ for $i \in [n-1]$, $i < j$. Then the algorithm runs $\text{iMSK}_i \leftarrow \text{iSetup}(1^\lambda, \boxed{1^{\kappa+2}} \boxed{1^{\kappa+1}})$ for $i \in [0; n-1]$ and outputs $\text{SK}_0 = (\text{iMSK}_0, \{K_i\}_{i \in [n-1]})$, $\{\text{SK}_i = (\text{iMSK}_i, K_i)\}_{i \in [n-1]}$ and $\text{MSK} = \{\text{iMSK}_i\}_{i \in [0; n-1]}$.

$\text{Enc}(\text{SK}_0, L, x_0, \mu)$ takes SK_0 , a label $L \in \mathcal{L}$, an attribute $x_0 \in \mathcal{X}$ and a message $\mu \in \mathcal{M}$ as input and sets $\mathbf{t}_{L,0} \in \mathbb{Z}_p^\kappa$ component-wise as

$$\mathbf{t}_{L,0}[v] = - \sum_{i \in [n-1]} \text{PRF}_{K_i}(L, v) \quad \text{for } v \in [\kappa],$$

samples $r \xleftarrow{\$} \mathbb{Z}_p$, runs

$$\begin{array}{|l} \mathbf{c} = \mu \oplus H(\llbracket r \rrbracket_t) \\ \text{iCT}_{L,0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, \llbracket (\mathbf{t}_{L,0}, r, 0) \rrbracket_1) \end{array} \quad \begin{array}{|l} \mathbf{c} = \mu \oplus H(r) \\ \text{iCT}_{L,0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, (\mathbf{t}_{L,0}, r)) \end{array}$$

and outputs the ciphertext $\text{CT}_L = (\text{iCT}_{L,0}, \mathbf{c})$.

$\text{AKeyGen}(\text{SK}_i, L, x_i)$ takes as input SK_i for some $i \in [n-1]$, a label $L \in \mathcal{L}$ and an attribute $x_i \in \mathcal{X}$, and defines $\mathbf{t}_{L,i} \in \mathbb{Z}_p^\kappa$ component-wise as $\mathbf{t}_{L,i}[v] = \text{PRF}_{K_i}(L, v)$ for $v \in [\kappa]$. Then the algorithm generates an iFE ciphertext

$$\boxed{\text{iCT}_{L,i} \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, \llbracket (\mathbf{t}_{L,i}, 0, 0) \rrbracket_1)} \quad \boxed{\text{iCT}_{L,i} \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, (\mathbf{t}_{L,i}, 0))},$$

and outputs $\text{DK}_{L,i} = \text{iCT}_{L,i}$.

$\text{PKeyGen}(\text{MSK}, \mathbf{y})$ takes as input MSK and attributes $\mathbf{y} \in \mathcal{Y}^{[0; n-1]}$, and samples

$$\boxed{\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^\kappa} \quad \boxed{\Delta \leftarrow \text{SamplerSet}(\kappa, Q, \tau), \quad \mathbf{e}_\Delta = \sum_{v \in \Delta} \mathbf{e}_v},$$

where $\mathbf{e}_v$ denotes the v -th unit vector in $\mathbb{Z}_p^\kappa$. Then the algorithm generates

$$\boxed{\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}_i, \mathbf{y}[i], \llbracket (\mathbf{s}, 1, 0) \rrbracket_2)} \quad \boxed{\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}_i, \mathbf{y}[i], (\mathbf{e}_\Delta, 1))},$$

for all $i \in [0; n-1]$, and outputs $\text{DK}_\mathbf{y} = \{\text{iDK}_i\}_{i \in [0; n-1]}$.

$\text{Dec}(\text{DK}_\mathbf{y}, \{\text{DK}_{L,i}\}_{i \in [n-1]}, \text{CT}_L)$ takes as input a policy decryption key $\text{DK}_\mathbf{y} = \{\text{iDK}_i\}_{i \in [0; n-1]}$, a set of attribute decryption keys $\{\text{DK}_{L,i} = \text{iCT}_{L,i}\}_{i \in [n-1]}$ and a ciphertext $\text{CT}_L = (\text{iCT}_{L,0}, \mathbf{c})$. For each $i \in [0; n-1]$, the algorithm runs $\theta_i \leftarrow \text{iDec}(\text{iDK}_i, \text{iCT}_{L,i})$ and immediately outputs $\perp$ if $\theta_i = \perp$. Otherwise, it computes $\theta = \sum_{i \in [0; n-1]} \theta_i$ and outputs $\mu = \mathbf{c} \oplus H(\theta)$.

Correctness. This readily follows from the correctness of iFE and the fact that $\sum_{i \in [0; n-1]} \mathbf{t}_{L,i} = \mathbf{0}$ for each $L \in \mathcal{L}$.

Security. The security naturally depends on the security of the employed AB-IPFE scheme.

Proposition 33. *If iFE provides function-hiding sel-security and the MDDH_κ assumption holds in $\mathbb{G}_2$, then the resulting MC-ABE scheme ABE in Construction 32 is sel-secure in the random oracle model. If iFE is stat-secure against Q key queries, then so is ABE relying additionally on random oracles.*

The restriction to stat security if iFE is not function-hiding comes from the fact that we cannot hardcode decryption results in the keys during the security proof but only in the ciphertexts. This essentially forces us to flip the usual proof technique, leading to selective key queries instead of selective encryption queries. Technically, this would allow for adaptive encryption queries if the employed iFE scheme did. But the only LWE-based instantiation of iFE in the literature that we are aware of [LLW21] requires selective encryption queries. Therefore, we prove security only in the static setting. The full proof of Proposition 33 can be found in Sections 5.2 and 5.3.

When instantiating iFE in Construction 32 with either the AB-IPFE implied by [ATY23] or the AB-IPFE of [LLW21], we obtain the following theorem.

Theorem 34. • Assuming MDDH on pairings, there exists a selectively secure MC-ABE for conjunctions of ABPs in the random oracle model.

• Assuming the hardness of LWE with subexponential modulus-to-noise ratio, there exists a bounded-collision statically secure MC-ABE for conjunctions of bounded-depth circuits in the random oracle model.

5.2 Proof of Security – Function-Hiding Case

The proof makes use of the following technical lemma.

Lemma 35. *Let $q = q(\lambda)$ be a polynomial, S a finite set of superpolynomial size and $H: S \rightarrow \{0, 1\}^{\text{poly}(\lambda)}$ a hash function modelled as a random oracle. Then, for $s, r_1, \dots, r_q \xleftarrow{\$} S$ and $\mathbf{y}_1, \dots, \mathbf{y}_q \xleftarrow{\$} \{0, 1\}^{\text{poly}(\lambda)}$, the following distributions are computationally indistinguishable:*

$$\{(s_j, H(r_j))\}_{j \in [q]} \approx_c \{(s_j, \mathbf{y}_j)\}_{j \in [q]},$$

where $s_j := s + r_j$ for all $i \in [q]$.

Proof of Lemma 35. Let $\mathcal{A}$ be a PPT adversary running in time $t = t(\lambda)$. By the superpolynomial order of S , we have $r_j \neq r_{j'}$ for all distinct $j, j' \in [q]$, $j \neq j'$, with overwhelming probability. Therefore, by the definition of a random oracle, the hash values $\{H(r_j)\}_{j \in [q]}$ are all independent and, for a fixed r_j , $H(r_j)$ can be distinguished from a random element $\mathbf{y}_j \xleftarrow{\$} \{0, 1\}^{\text{poly}(\lambda)}$ if and only if r_j is known. We hence need to bound the probability that the adversary learns one of the values in $\{r_1, \dots, r_q\}$. For a single query to the random oracle, the probability to have asked an r_j is bounded by $q/|S|$. This allows the adversary to exclude up to q^2 points using the constraints $\{s + r_j = s_j\}_{j \in [q]}$. Thus, after t queries, the probability to have queried the random oracle on one of the r_j is at most $qt/(|S| - q^2 t)$ which is negligible in λ . $\square$

Proof of Proposition 33 – Function-Hiding Case. Let q_L denote the number of different labels being input to a QEnc query and let $\mathcal{Q}_L = \{L_1, \dots, L_{q_L}\}$ be the set of these labels. The proposition is proven via a sequence of hybrid games $G_0^0, \dots, G_{q_L}^0, G_{q_L}^1, \dots, G_0^1$, where $G_0^b \approx_c \text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-b}}$ for $b \in \{0, 1\}$. Modifications between consecutive games are highlighted using **boxes**. In G_0^b , the challenger behaves as follows. Initially, sample $\hat{\mathbf{t}}_{L,i} \xleftarrow{\$} \mathbb{Z}_p^k$ for each $L \in \mathcal{Q}_L$ and $i \in [n-1] \setminus \mathcal{C}$.

- For each encryption query $\text{QEnc}(L, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$, define $\mathbf{t}_{L,0} \in \mathbb{Z}_p^k$ component-wise as

$$\mathbf{t}_{L,0}[v] = - \sum_{i \in \mathcal{C}} \text{PRF}_{K_i}(L, v) - \sum_{i \in [n-1] \setminus \mathcal{C}} \hat{\mathbf{t}}_{L,i}[v] \quad \text{for } v \in [\kappa],$$

sample $r \xleftarrow{\$} \mathbb{Z}_p$, run $\text{iCT}_{L,0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, \llbracket (\mathbf{t}_{L,0}, r, 0) \rrbracket_1)$ and return $\text{CT}_L = (\text{iCT}_{L,0}, \mathbf{c} = \boldsymbol{\mu}^b \oplus H(\llbracket r \rrbracket_t))$.

- Upon receiving an attribute key query $\text{QAKeyGen}(i, L, x_i)$, define $\mathbf{t}_{L,i} \in \mathbb{Z}_p^\kappa$ component-wise for $v \in [\kappa]$ as

$$\mathbf{t}_{L,i}[v] = \begin{cases} \hat{\mathbf{t}}_{L,i}[v] & \text{if } L \in \mathcal{Q}_L \text{ and } i \in [n-1] \setminus \mathcal{C}, \\ \text{PRF}_{K_i}(L, v) & \text{otherwise,} \end{cases}$$

run $\text{iCT}_{L,i} \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, \llbracket (\mathbf{t}_{L,i}, 0, 0) \rrbracket_1)$ and return $\text{DK}_{L,i} = \text{iCT}_{L,i}$.

- Upon receiving a policy key query $\text{QPKeyGen}(\mathbf{y})$, sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^\kappa$, generate $\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}_i, \mathbf{y}[i], \llbracket (\mathbf{s}, 1, 0) \rrbracket_2)$ for $i \in [0; n-1]$ and return $\text{DK}_{\mathbf{y}} = \{\text{iDK}_i\}_{i \in [0; n-1]}$.

It is not hard to see that $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b} \approx_c G_0^b$ by the security of PRF. The game G_ℓ^b , for $\ell \in [q_L]$ and $b \in \{0, 1\}$, is the same as G_0^b except that for encryption queries $\text{QEnc}(L, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$ such that $L \in \{L_1, \dots, L_\ell\}$, the challenger returns $\text{CT}_L = (\text{iCT}_{L,0}, \mathbf{c} = \boldsymbol{\mu}^0 \oplus H(\llbracket r \rrbracket_t))$. Below we proof the following claim for all $\ell \in [q_L]$ and $b \in \{0, 1\}$

Claim 36. *If iFE is function-hiding and MDDH_κ holds in $\mathbb{G}_2$, then $G_{\ell-1}^b \approx_c G_\ell^b$.*

Furthermore, we observe that $G_{q_L}^0 \equiv G_{q_L}^1$ as the challenger's output is independent of the bit b in these games. This concludes the proof of the proposition. $\square$

Proof of Claim 36. We start with an analysis of the admissibility condition. For each policy query $\text{QPKeyGen}(\mathbf{y})$, we can distinguish the following two cases:

1. all queries $\text{QEnc}(L_\ell, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$ satisfy $\boldsymbol{\mu}^0 = \boldsymbol{\mu}^1$ or $P^*(x_0, \mathbf{y}[0]) \neq 0$, or
2. there exists at least one query $\text{QEnc}(L_\ell, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$ such that $\boldsymbol{\mu}^0 \neq \boldsymbol{\mu}^1$ and $P^*(x_0, \mathbf{y}[0]) = 0$.

In case 2, the admissibility condition implies that there exists at least one index $i_{\mathbf{y}} \in ([n-1] \setminus \mathcal{C})$ such that it holds $P^*(x, \mathbf{y}[i_{\mathbf{y}}]) \neq 0$ for all queries of the form $\text{QAKeyGen}(i_{\mathbf{y}}, L_\ell, x)$.

The proof considers a sequence of hybrids $\widehat{G}_0^0, \dots, \widehat{G}_5^0, \widehat{G}_5^1, \dots, \widehat{G}_0^1$, where $\widehat{G}_0^0 = G_{\ell-1}^b$ and $\widehat{G}_0^1 = G_\ell^b$. We argue $\widehat{G}_{\alpha-1}^{b'} \approx_c \widehat{G}_\alpha^{b'}$, for $\alpha \in [5]$ and $b' \in \{0, 1\}$, as well as $G_5^0 \equiv G_5^1$. Then the claim follows via a hybrid argument.

Game $\widehat{G}_0^{b'}$: This is $G_{\ell-1+b'}^b$.

Game $\widehat{G}_1^{b'}$: This is the same as $\widehat{G}_0^{b'}$ except that we hardcode $\{\mathbf{t}_{L_\ell, i}\}_{i \in [0; n-1]}$ in the policy decryption keys in the following way:

- for the response to an encryption query $\text{QEnc}(L_\ell, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$, the challenger computes

$$\text{iCT}_{L_\ell, 0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, \llbracket (\mathbf{0}, r, \mathbf{1}) \rrbracket_1),$$

- for the response to an attribute key query $\text{QAKeyGen}(i, L_\ell, x_i)$, the challenger computes

$$\text{iCT}_{L_\ell, i} \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, \llbracket (\mathbf{0}, 0, \mathbf{1}) \rrbracket_1),$$

- for the response to a policy key query $\text{QPKeyGen}(\mathbf{y})$, the challenger computes,

$$\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}_i, \mathbf{y}[i], \llbracket (\mathbf{s}, 1, \mathbf{s} \cdot \mathbf{t}_{L_\ell, i}^\top) \rrbracket_2) \quad \text{for } i \in [0; n-1].$$

It is not hard to see that the inner products between vectors encoded in iFE ciphertexts and decryption key do not change. Thus, we have $\widehat{G}_0^{b'} \approx_c \widehat{G}_1^{b'}$ under the function-hiding security of iFE.

Game $\widehat{G}_2^{b'}$: This is the same as $\widehat{G}_1^{b'}$ except that, upon receiving a policy key query $\text{QPKeyGen}(\mathbf{y})$ of type 2, the challenger computes

$$\text{iDK}_{i_{\mathbf{y}}} \leftarrow \text{iKeyGen}(\text{iMSK}_{i_{\mathbf{y}}}, \mathbf{y}[i_{\mathbf{y}}], \llbracket (\mathbf{s}, 1, \mathbf{0}) \rrbracket_2)$$

We have $\widehat{G}_0^{b'} \approx_c \widehat{G}_1^{b'}$ under the security of iFE since $P^*(x, \mathbf{y}[i_{\mathbf{y}}]) \neq 0$ for all attribute key queries of the form $\text{QAKeyGen}(i_{\mathbf{y}}, L_\ell, x)$.

Game $\widehat{G}_3^{b'}$: This is the same as $\widehat{G}_2^{b'}$ except that, upon receiving a policy key query $\text{QPKeyGen}(\mathbf{y})$ of type **2**, the challenger samples $s_\ell \xleftarrow{\$} \mathbb{Z}_p$ and computes

$$\text{iDK}_0 \leftarrow \text{iKeyGen}(\text{iMSK}_0, \mathbf{y}[0], \llbracket (\mathbf{s}, 1, \mathbf{s} \cdot \mathbf{t}_{L_\ell, 0}^\top + s_\ell) \rrbracket_2) .$$

We have $\widehat{G}_2^{b'} \approx_c \widehat{G}_3^{b'}$ since $\mathbf{t}_{L_\ell, 0}$ contains the term $\widehat{\mathbf{t}}_{L_\ell, i_y} \xleftarrow{\$} \mathbb{Z}_p$, and the MDDH_K assumption in $\mathbb{G}_2$ implies that $\llbracket \mathbf{s} \cdot \widehat{\mathbf{t}}_{L_\ell, 0}^\top \rrbracket_2 \approx_c \llbracket \mathbf{s} \cdot \widehat{\mathbf{t}}_{L_\ell, 0}^\top + s_\ell \rrbracket_2$.

Game $\widehat{G}_4^{b'}$: This is the same as $\widehat{G}_3^{b'}$ except that we move the random scalars s_ℓ into the ciphertexts:

- for the response to an encryption query $\text{QEnc}(L_\ell, x_0, \mu^0, \mu^1)$ satisfying $\mu^0 \neq \mu^1$, the challenger computes

$$\text{iCT}_{L_\ell, 0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, \llbracket (\mathbf{0}, r + s_\ell, 1) \rrbracket_1) ,$$

- for the response to a policy key query $\text{QPKeyGen}(\mathbf{y})$, the challenger computes

$$\text{iDK}_0 \leftarrow \text{iKeyGen}(\text{iMSK}_0, \mathbf{y}[0], \llbracket (\mathbf{s}, 1, \mathbf{s} \cdot \mathbf{t}_{L_\ell, 0}^\top + s_\ell) \rrbracket_2) .$$

We have $\widehat{G}_3^{b'} \approx_c \widehat{G}_4^{b'}$ under the security of iFE. To see this, we distinguish two cases for any pair of queries $\text{QPKeyGen}(\mathbf{y})$ and $\text{QEnc}(L_\ell, x_0, \mu^0, \mu^1)$ such that $\mu^0 \neq \mu^1$. Let $\{\text{iDK}_i\}_{i \in [0; n-1]}$ and $\text{iCT}_{L_\ell, 0}$ denote the challenger's responses. If $P^*(x_0, \mathbf{y}[0]) \neq 0$, then the adversary learns nothing at all about the vectors encoded in $\text{iCT}_{L_\ell, 0}$ and iDK_0 . If $P^*(x_0, \mathbf{y}[0]) = 0$, then the adversary can decrypt this combination of ciphertext and decryption key. However, $\text{QPKeyGen}(\mathbf{y})$ is a type **2** query in this case, so the inner products of the vectors encoded in $\text{iCT}_{L_\ell, 0}$ and iDK_0 do not change from $\widehat{G}_3^{b'}$ to $\widehat{G}_4^{b'}$.

Game $\widehat{G}_5^{b'}$: This game is the same as $\widehat{G}_4^{b'}$ except that, for each query $\text{QEnc}(L, x_0, \mu^0, \mu^1)$ such that $\mu^0 \neq \mu^1$, the challenger samples a random element $\mathbf{h} \xleftarrow{\$} \mathcal{M}$ and computes $\mathbf{c} = \mu^b \oplus \mathbf{h}$. To argue $\widehat{G}_4^b \approx_c \widehat{G}_5^b$, we let $\text{QEnc}(L_\ell, x_0^j, \mu^{j,0}, \mu^{j,1})$ denote the j -th encryption query for the label L_ℓ such that $\mu^{j,0} \neq \mu^{j,1}$, and q_ℓ refers to the number of such queries. For the reply to the j -th query, the challenger samples $r^j \xleftarrow{\$} \mathbb{Z}_p$ and computes

$$\text{iCT}_{L_\ell, 0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, \llbracket (\mathbf{0}, r^j + s_\ell, 1) \rrbracket_1) .$$

For each query $\text{QPKeyGen}(\mathbf{y})$, the adversary learns either nothing about r^j if $P^*(x_0^j, \mathbf{y}[0]) \neq 0$, or it learns $r^j + s_\ell$ if $P^*(x_0^j, \mathbf{y}[0]) = 0$. Thus, the adversary's view is captured by $\{(r^j + s_\ell, H(r^j))\}_{j \in [q_\ell]}$. Then $\widehat{G}_4^b \approx_c \widehat{G}_5^b$ follows from an application of Lemma 35 for $s = s_\ell$ and $q = q_\ell$.

Finally, we observe that $\widehat{G}_5^0 \equiv \widehat{G}_5^1$ since for each query $\text{QEnc}(L, x_0, \mu^0, \mu^1)$ such that $\mu^0 \neq \mu^1$, the random element $\mathbf{h} \xleftarrow{\$} \mathcal{M}$ perfectly hides μ^b in $\mathbf{c} = \mu^b \oplus \mathbf{h}$. This concludes the proof. $\square$

5.3 Proof of Security – Non-Function-Hiding Case

Proof of Proposition 33 – Non-Function-Hiding Case. The proposition is proven via the following sequence of hybrid games. Modifications between consecutive games are highlighted using **boxes**.

Game $\widehat{G}_0^b$ for $b \in \{0, 1\}$: This is the experiment $\text{Exp}_{\text{ABE}, \mathcal{A}}^{\text{mc-abe-}b}(1^\lambda)$. In particular, the challenger behaves as follows.

- Upon receiving an encryption query $\text{QEnc}(L, x_0, \mu^0, \mu^1)$, define $\mathbf{t}_{L, 0} \in \mathbb{Z}_p^\kappa$ component-wise as

$$\mathbf{t}_{L, 0}[v] = - \sum_{i \in [n-1]} \text{PRF}_{K_i}(L, v) \quad \text{for } v \in [\kappa] ,$$

sample $r \xleftarrow{\$} \mathbb{Z}_p$, run $\text{iCT}_{L, 0} \leftarrow \text{iEnc}(\text{iMSK}_0, x_0, (\mathbf{t}_{L, 0}, r))$ and return $\text{CT}_L = (\text{iCT}_{L, 0}, \mathbf{c} = \mu^b \oplus H(r))$.

- Upon receiving an attribute key query $\text{QAKeyGen}(i, L, x_i)$, set $\mathbf{t}_{L,i} \in \mathbb{Z}_p^\kappa$ component-wise as $\mathbf{t}_{L,i}[v] = \text{PRF}_{K_i}(L, v)$ for $v \in [\kappa]$, run $\text{iCT}_{L,i} \leftarrow \text{iEnc}(\text{iMSK}_i, x_i, (\mathbf{t}_{L,i}, 0))$ and return $\text{DK}_{L,i} = \text{iCT}_{L,i}$.
- Let q_k denote the number of queries to QPKeyGen . Upon receiving the ℓ -th policy key query $\text{QPKeyGen}(\mathbf{y}^\ell)$ for $\ell \in [q_k]$, sample a subset $\Delta_\ell \leftarrow \text{SamplerSet}(\kappa, Q, \tau)$ and define the vector $\mathbf{e}_{\Delta_\ell} = \sum_{v \in \Delta_\ell} \mathbf{e}_v$. Then generate $\text{iDK}_i^\ell \leftarrow \text{iKeyGen}(\text{iMSK}_i, \mathbf{y}^\ell[i], (\mathbf{e}_{\Delta_\ell}, 1))$ for all $i \in [0; n-1]$ and return $\text{DK}_\mathbf{y}^\ell = \{\text{iDK}_i^\ell\}_{i \in [0; n-1]}$.

Similar to the function-hiding case, a policy key query $\text{QPKeyGen}(\mathbf{y}^\ell)$ can be of two types w.r.t. a label $L \in \mathcal{L}$.

1. all queries $\text{QEnc}(L, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$ satisfy $\boldsymbol{\mu}^0 = \boldsymbol{\mu}^1$ or $P^*(x_0, \mathbf{y}^\ell[0]) \neq 0$, or
2. there exists at least one query $\text{QEnc}(L, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$ such that $\boldsymbol{\mu}^0 \neq \boldsymbol{\mu}^1$ and $P^*(x_0, \mathbf{y}^\ell[0]) = 0$.

In case 2, the admissibility condition implies that there exists at least one index $i_\ell \in ([n-1] \setminus \mathcal{C})$ such that $P^*(x, \mathbf{y}^\ell[i_\ell]) \neq 0$ for all queries of the form $\text{QAKeyGen}(i_\ell, L, x)$.

Game G_1^b for $b \in \{0, 1\}$: This game is the same as game G_0^b except that the challenger picks the subsets $\Delta_\ell \leftarrow \text{SamplerSet}(\kappa, Q, \tau)$ for $\ell \in [q_k]$ in one shot at the outset of the game. For every $\ell \in [q_k]$, the challenger chooses an index

$$\delta_\ell \in \Delta'_\ell := \Delta_\ell \setminus \left(\bigcup_{\ell' \in [q_k] \setminus \{k\}} \Delta_{\ell'} \right)$$

if $\Delta'_\ell \neq \emptyset$, and aborts otherwise. The admissibility condition requires that $q_k \leq Q$. Therefore, it follows from an application of Lemma 20 that $\Delta'_\ell \neq \emptyset$ for all $\ell \in [q_k]$ with overwhelming probability which implies that $G_1^b \approx_s G_2^b$.

Game G_2^b for $b \in \{0, 1\}$: This game is the same as G_1^b except that, upon receiving a query $\text{QEnc}(L, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$, the challenger computes

$$\text{iCT}_{L,0} \leftarrow \text{iEnc}\left(\text{iMSK}_0, x_0, \left(\mathbf{t}_{L,0} + r \cdot \sum_{\ell \in [q_k]} \mathbf{e}_{\delta_\ell}, \mathbf{0}\right)\right).$$

We note that all inner products of vectors embedded in ciphertexts and keys of the 0-th iFE instance do not change from G_1^b to G_2^b . Thus, we have $G_1^b \approx_c G_2^b$ under the security of iFE.

Game G_3^b for $b \in \{0, 1\}$: This game is the same as G_2^b except that, upon receiving a query $\text{QAKeyGen}(L, x_i)$, the challenger computes $\mathbf{t}_{L,i}$ component-wise for each $v \in [\kappa]$ as

$$\mathbf{t}_{L,i}[v] = \begin{cases} \mathbf{0} & \text{if } \begin{aligned} & \bullet v = \delta_\ell \text{ for some } \ell \in [q_k], \\ & \bullet \text{QPKeyGen}(\mathbf{y}^\ell) \text{ is a type 2 query w.r.t. } L, \\ & \bullet i = i_\ell, \end{aligned} \\ \text{PRF}_{K_i}(L, v) & \text{otherwise.} \end{cases}$$

We argue that the adversary cannot notice such a modification in $\mathbf{t}_{L,i}$. First, as $\text{QPKeyGen}(\mathbf{y}^\ell)$ is a type 2 query w.r.t. L , the iFE decryption key iDK_i^ℓ cannot be used to decrypt the ciphertext $\text{iCT}_{L,i}$ encoding $\mathbf{t}_{L,i}$. Second, since $v = \delta_\ell$, all other decryption keys $\text{iDK}_i^{\ell'}$, for $\ell' \neq \ell$, encode a vector $\mathbf{e}_{\Delta_{\ell'}}$ such that $\mathbf{e}_{\Delta_{\ell'}}[v] = 0$ by construction. So the inner product does not change and we have $G_2^b \approx G_3^b$ under the security of iFE.

Game G_4^b for $b \in \{0, 1\}$: Let $\mathcal{Q}_L = \{L_1, \dots, L_{q_L}\}$ be the set of all labels that appear in an encryption query and let $q_L = |\mathcal{Q}_L|$. This game is the same as G_3^b except that the challenger initially samples a scalar $s_L \xleftarrow{\$} \mathbb{Z}_p$ for each $L \in \mathcal{Q}_L$. Upon receiving a query $\text{QEnc}(L, x_0, \boldsymbol{\mu}^0, \boldsymbol{\mu}^1)$ such that $\boldsymbol{\mu}^0 \neq \boldsymbol{\mu}^1$, it now computes

$$\text{iCT}_{L,0} \leftarrow \text{iEnc}\left(\text{iMSK}_0, x_0, \left(\mathbf{t}_{L,0} + r \cdot \sum_{\ell \in [q_k]} \mathbf{e}_{\delta_\ell} + s_L \cdot \sum_{\substack{\ell \in [q_k], \\ P^*(x_0, \mathbf{y}^\ell[0])=0}} \mathbf{e}_{\delta_\ell}, \mathbf{0}\right)\right).$$

We have $G_3^b \approx_c G_4^b$ which can be seen as follows. Let $\text{QEnc}(L, x_0, \mu^0, \mu^1)$ and $\ell \in [q_k]$ such that $\mu^0 \neq \mu^1$ and $P^*(x_0, \mathbf{y}^\ell[0]) = 0$, i.e. $\text{QPKeyGen}(\mathbf{y}^\ell)$ is a type 2 query w.r.t. the label L . After the changes from the previous game, we can observe that the value $\text{PRF}_{K_{i_\ell}}(L, \delta_\ell)$ now only appears inside $\mathbf{t}_{L,0}$. Furthermore, the slot $i_\ell \in [n-1] \setminus \mathcal{C}$ is honest, so the PRF key $K_{0,i}$ is not known to the adversary. This implies that $\{\text{PRF}_{K_{i_\ell}}(L, \delta_\ell)\} \approx_c \{\text{PRF}_{K_{i_\ell}}(L, \delta_\ell) + s_L\}$ by the security of PRF, where the first distribution corresponds to G_3^b and the second distribution corresponds to G_4^b , followed by the same post-processing.

Game $G_{4,\alpha}^b$ for $\alpha \in [0; q_L]$ and $b \in \{0, 1\}$: This game is the same as game G_4^b except that, when receiving a query $\text{QEnc}(L_{\alpha'}, x_0, \mu^0, \mu^1)$ such that $\alpha' \in [\alpha]$ and $\mu^0 \neq \mu^1$, the challenger samples a random element $\mathbf{h} \xleftarrow{\$} \mathcal{M}$ and computes $\mathbf{c} = \mu^b \oplus \mathbf{h}$. Clearly, we have $G_{4,0}^b = G_4^b$ as $[0] = \emptyset$. Furthermore, we can observe that $G_{4,q_L}^0 \equiv G_{4,q_L}^1$ since for each query $\text{QEnc}(L, x_0, \mu^0, \mu^1)$ such that $\mu^0 \neq \mu^1$, the random element $\mathbf{h} \xleftarrow{\$} \mathcal{M}$ perfectly hides μ^b in $\mathbf{c} = \mu^b \oplus \mathbf{h}$. It remains to argue that $G_{4,\alpha-1}^b \approx_c G_{4,\alpha}^b$ for all $\alpha \in [q_L]$. Let $\text{QEnc}(L_\alpha, x_0^j, \mu^{j,0}, \mu^{j,1})$ denote the j -th encryption query for the label L_α such that $\mu^{j,0} \neq \mu^{j,1}$, and q_α refers to the number of such queries. For each $\ell \in [q_k]$, the adversary learns either nothing about r^j if $P^*(x_0^j, \mathbf{y}^\ell[0]) \neq 0$, or it learns $r^j + s_{L_\alpha}$ if $P^*(x_0^j, \mathbf{y}^\ell[0]) = 0$. Thus, the adversary's view is captured by $\{(r^j + s_{L_\alpha}, H(r^j))\}_{j \in [q_\alpha]}$. Then an application of Lemma 35 for $s = s_{L_\alpha}$ and $q = q_\alpha$ implies that $G_{4,\alpha-1}^b \approx_c G_{4,\alpha}^b$ which concludes the proof. $\square$

Acknowledgments

This work was supported in part by the France 2030 ANR Project ANR-22-PECY-003 SecureCompute.

References

- [ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Lattice basis delegation in fixed dimension and shorter-ciphertext hierarchical IBE. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 98–115. Springer, Berlin, Heidelberg, August 2010.
- [ABDP15] Michel Abdalla, Florian Bourse, Angelo De Caro, and David Pointcheval. Simple functional encryption schemes for inner products. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 733–751. Springer, Berlin, Heidelberg, March / April 2015.
- [ABG19] Michel Abdalla, Fabrice Benhamouda, and Romain Gay. From single-input to multi-client inner-product functional encryption. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 552–582. Springer, Cham, December 2019.
- [ABKW19] Michel Abdalla, Fabrice Benhamouda, Markulf Kohlweiss, and Hendrik Waldner. Decentralizing inner-product functional encryption. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 128–157. Springer, Cham, April 2019.
- [ACF⁺18] Michel Abdalla, Dario Catalano, Dario Fiore, Romain Gay, and Bogdan Ursu. Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 597–627. Springer, Cham, August 2018.
- [ACGU20] Michel Abdalla, Dario Catalano, Romain Gay, and Bogdan Ursu. Inner-product functional encryption with fine-grained access control. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 467–497. Springer, Cham, December 2020.
- [Agr17] Shweta Agrawal. Stronger security for reusable garbled circuits, general definitions and attacks. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 3–35. Springer, Cham, August 2017.
- [AGRW17] Michel Abdalla, Romain Gay, Mariana Raykova, and Hoeteck Wee. Multi-input inner-product functional encryption from pairings. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 601–626. Springer, Cham, April / May 2017.
- [AGT21a] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-input quadratic functional encryption from pairings. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 208–238, Virtual Event, August 2021. Springer, Cham.
- [AGT21b] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-party functional encryption. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 224–255. Springer, Cham, November 2021.
- [AGT22] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-input quadratic functional encryption: Stronger security, broader functionality. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 711–740. Springer, Cham, November 2022.
- [AJ15] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 308–326. Springer, Berlin, Heidelberg, August 2015.
- [AKY24] Shweta Agrawal, Simran Kumari, and Shota Yamada. Pseudorandom multi-input functional encryption and applications. Cryptology ePrint Archive, Paper 2024/1720, 2024.

- [ALS16] Shweta Agrawal, Benoît Libert, and Damien Stehlé. Fully secure functional encryption for inner products, from standard assumptions. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 333–362. Springer, Berlin, Heidelberg, August 2016.
- [ARYY23] Shweta Agrawal, Mélissa Rossi, Anshu Yadav, and Shota Yamada. Constant input attribute based (and predicate) encryption from evasive and tensor LWE. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 532–564. Springer, Cham, August 2023.
- [ATY23] Shweta Agrawal, Junichi Tomida, and Anshu Yadav. Attribute-based multi-input FE (and more) for attribute-weighted sums. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 464–497. Springer, Cham, August 2023.
- [AY20] Shweta Agrawal and Shota Yamada. CP-ABE for circuits (and more) in the symmetric key setting. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 117–148. Springer, Cham, November 2020.
- [AYY22] Shweta Agrawal, Anshu Yadav, and Shota Yamada. Multi-input attribute based encryption and predicate encryption. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 590–621. Springer, Cham, August 2022.
- [Bei96] Amos Beimel. *Secure Schemes for Secret Sharing and Key Distribution*. PhD thesis, Technion - Israel Institute of Technology, Haifa, Israel, 1996. <https://www.cs.bgu.ac.il/~beimel/Papers/thesis.pdf>.
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2001.
- [BJK⁺18] Zvika Brakerski, Aayush Jain, Ilan Komargodski, Alain Passelègue, and Daniel Wichs. Non-trivial witness encryption and null-iO from standard assumptions. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 425–441. Springer, Cham, September 2018.
- [BL90] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In Shafi Goldwasser, editor, *CRYPTO’88*, volume 403 of *LNCS*, pages 27–35. Springer, New York, August 1990.
- [BW07] Dan Boneh and Brent Waters. Conjunctive, subset, and range queries on encrypted data. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 535–554. Springer, Berlin, Heidelberg, February 2007.
- [CDG⁺18] Jérémy Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Decentralized multi-client functional encryption for inner product. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 703–732. Springer, Cham, December 2018.
- [CLL⁺13] Jie Chen, Hoon Wei Lim, San Ling, Huaxiong Wang, and Hoeteck Wee. Shorter IBE and signatures via asymmetric pairings. In Michel Abdalla and Tanja Lange, editors, *PAIRING 2012*, volume 7708 of *LNCS*, pages 122–140. Springer, Berlin, Heidelberg, May 2013.
- [CLW25] Valerio Cini, Russell W. F. Lai, and Ivy K. Y. Woo. Lattice-based multi-authority/client attribute-based encryption for circuits. *IACR Communications in Cryptology*, 1(4), 2025.

- [DOT18] Pratish Datta, Tatsuaki Okamoto, and Junichi Tomida. Full-hiding (unbounded) multi-input inner product functional encryption from the k -Linear assumption. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 245–277. Springer, Cham, March 2018.
- [EHK⁺13] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147. Springer, Berlin, Heidelberg, August 2013.
- [FFMV23] Danilo Francati, Daniele Friolo, Giulio Malavolta, and Daniele Venturi. Multi-key and multi-input predicate encryption from learning with errors. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 573–604. Springer, Cham, April 2023.
- [GGG⁺14] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 578–602. Springer, Berlin, Heidelberg, May 2014.
- [GGH⁺13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th FOCS*, pages 40–49. IEEE Computer Society Press, October 2013.
- [GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 89–98. ACM Press, October / November 2006. Available as Cryptology ePrint Archive Report 2006/309.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.
- [HJP18] Dennis Hofheinz, Dingding Jia, and Jiaxin Pan. Identity-based encryption tightly secure under chosen-ciphertext attacks. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 190–220. Springer, Cham, December 2018.
- [HLL24] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. A general framework for lattice-based ABE using evasive inner-product functional encryption. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 433–464. Springer, Cham, May 2024.
- [JR17] Charanjit S. Jutla and Arnab Roy. Shorter quasi-adaptive NIZK proofs for linear subspaces. *Journal of Cryptology*, 30(4):1116–1156, October 2017.
- [Lin17] Huijia Lin. Indistinguishability obfuscation from SXDH on 5-linear maps and locality-5 PRGs. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 599–629. Springer, Cham, August 2017.
- [LL20] Huijia Lin and Ji Luo. Compact adaptively secure ABE from k -Lin: Beyond NC^1 and towards NL. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 247–277. Springer, Cham, May 2020.
- [LLW21] Qiqi Lai, Feng-Hao Liu, and Zhedong Wang. New lattice two-stage sampling technique and its applications to functional encryption - stronger security and smaller ciphertexts. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 498–527. Springer, Cham, October 2021.

- [LT19] Benoît Libert and Radu Titiiu. Multi-client functional encryption for linear functions in the standard model from LWE. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 520–551. Springer, Cham, December 2019.
- [LV16] Huijia Lin and Vinod Vaikuntanathan. Indistinguishability obfuscation from DDH-like assumptions on constant-degree graded encodings. In Irit Dinur, editor, *57th FOCS*, pages 11–20. IEEE Computer Society Press, October 2016.
- [LW11] Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 568–588. Springer, Berlin, Heidelberg, May 2011.
- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, April 2012.
- [NPP22] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. Multi-client functional encryption with fine-grained access control. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 95–125. Springer, Cham, December 2022.
- [NPP24] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. Multi-client functional encryption with public inputs and strong security. Cryptology ePrint Archive, Report 2024/740, 2024.
- [PS24] David Pointcheval and Robert Schädlich. Multi-client attribute-based and predicate encryption from standard assumptions. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part III*, volume 15366 of *LNCS*, pages 31–64. Springer, Cham, December 2024.
- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.
- [SBC⁺07] Elaine Shi, John Bethencourt, Hubert T.-H. Chan, Dawn Xiaodong Song, and Adrian Perrig. Multi-dimensional range query over encrypted data. In *2007 IEEE Symposium on Security and Privacy*, pages 350–364. IEEE Computer Society Press, May 2007.
- [Sch25] Robert Schädlich. Multi-client attribute-based and predicate encryption, revisited. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography - PKC 2025*, Cham, 2025. Springer Nature Switzerland.
- [SV23] Elaine Shi and Nikhil Vanjani. Multi-client inner product encryption: Function-hiding instantiations without random oracles. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 622–651. Springer, Cham, May 2023.
- [SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 457–473. Springer, Berlin, Heidelberg, May 2005.
- [Tom19] Junichi Tomida. Tightly secure inner product functional encryption: Multi-input and function-hiding constructions. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 459–488. Springer, Cham, December 2019.
- [TÜ24] Erkan Tairi and Akin Ünal. Lower bounds for lattice-based compact functional encryption. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 249–279. Springer, Cham, May 2024.
- [Üna20] Akin Ünal. Impossibility results for lattice-based functional encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 169–199. Springer, Cham, May 2020.

- [Wat09] Brent Waters. Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 619–636. Springer, Berlin, Heidelberg, August 2009.
- [Wee17] Hoeteck Wee. Attribute-hiding predicate encryption in bilinear groups, revisited. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 206–233. Springer, Cham, November 2017.

A AB-MIFE for Inner Products Against Bounded Collusions

A.1 Definitions for MIFE with Corruptions

We recall the definition of Multi-Input Functional Encryption (MIFE). Let $\{\mathcal{U}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\{\mathcal{V}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of domain and range spaces, where $\mathcal{U}_\lambda = \mathcal{U}_{\lambda, \text{pub}} \times \mathcal{U}_{\lambda, \text{pri}}$ consists of a public and a private component. We consider a function class $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ where each $\mathcal{F}_\lambda$ contains functions $f: \mathcal{U}_\lambda^n \rightarrow \mathcal{V}_\lambda$ and $n = \text{poly}(\lambda)$.

Definition 37 (Syntax of MIFE). A MIFE scheme for the functionality $\mathcal{F}$ consists of four efficient algorithms:

Setup(1^λ) $\rightarrow$ ($\{\text{SK}_i\}_{i \in [n]}$, MSK): On input the security parameter 1^λ , this algorithm outputs a set of secret keys $\{\text{SK}_i\}_{i \in [n]}$ and a master secret key MSK.

Enc($\text{SK}_i, u_{i, \text{pub}}, u_{i, \text{pri}}$) $\rightarrow$ CT_i : On input a secret key SK_i , a public input $u_{i, \text{pub}} \in \mathcal{U}_{\lambda, \text{pub}}$ and a private input $u_{i, \text{pri}} \in \mathcal{U}_{\lambda, \text{pri}}$, this algorithm outputs a ciphertext CT_i . We assume that CT_i implicitly contains $u_{i, \text{pub}}$.

KeyGen(MSK, f) $\rightarrow$ DK: On input the master secret key MSK and a function $f \in \mathcal{F}_\lambda$, this algorithm outputs a decryption key DK. We assume that DK implicitly contains f .

Dec(DK, $\{\text{CT}_i\}_{i \in [n]}$) $\rightarrow$ v : On input a decryption key DK and a set of ciphertexts $\{\text{CT}_i\}_{i \in [n]}$, this algorithm outputs a value $v \in \mathcal{V}_\lambda$.

Note that, in contrast to our definition of MC-ABE, we use slot indices $i \in [n]$ instead of $[0; n-1]$ because this choice seems more common and natural in the general MIFE setting.

Correctness. A MIFE scheme for a functionality $\mathcal{F}$ is correct if for all $\lambda \in \mathbb{N}$, $u_1, \dots, u_n \in \mathcal{U}_\lambda$ and $f \in \mathcal{F}_\lambda$, it holds that

$$\Pr \left[v = f(u) \mid \begin{array}{l} (\{\text{SK}_i\}_{i \in [n]}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda) \\ \forall i \in [n]: \text{CT}_i \leftarrow \text{Enc}(\text{SK}_i, u_i) \\ \text{DK} \leftarrow \text{KeyGen}(\text{MSK}, f) \\ v := \text{Dec}(\text{DK}, \{\text{CT}_i\}_{i \in [n]}) \end{array} \right] = 1,$$

where the probability is taken over the random coins of the algorithms.

Security. We define IND-CPA security with static corruptions.

Definition 38 (Security Against Bounded Collusions). Let $\text{xxx} \in \{\text{adp}, \text{stat}\}$. A MIFE scheme FE for a functionality $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ is secure against Q key queries if $\text{Exp}_{\text{FE}, \mathcal{A}}^{\text{mife-0}}(1^\lambda) \approx_c \text{Exp}_{\text{FE}, \mathcal{A}}^{\text{mife-1}}(1^\lambda)$ for all PPT adversaries $\mathcal{A}$, where $\text{Exp}_{\text{FE}, \mathcal{A}}^{\text{mife-}b}$ for $b \in \{0, 1\}$ proceeds as follows.

- **Setup.** Launch $\mathcal{A}(1^\lambda)$ and receive from it a set $\mathcal{C} \subseteq [n]$ of corrupted slots. Run $(\{\text{SK}_i\}_{i \in [n]}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda)$ and send $\{\text{SK}_i\}_{i \in \mathcal{C}}$ to $\mathcal{A}$. Initialize a counter $m = 0$.
- **Query.** Repeat the following for arbitrarily many rounds determined by $\mathcal{A}$. In each round, $\mathcal{A}$ has two options.
 - **QEnc**(i, u_i^0, u_i^1): upon $\mathcal{A}$ submitting an index $i \in [n]$ and a pair of messages $u_i^0, u_i^1 \in \mathcal{U}_\lambda$, run $\text{CT}_i \leftarrow \text{Enc}(\text{SK}_i, u_i^b)$ and return CT_i to $\mathcal{A}$.
 - **QKeyGen**(f): upon $\mathcal{A}$ submitting a function $f \in \mathcal{F}_\lambda$, run $\text{DK} \leftarrow \text{KeyGen}(\text{MSK}, f)$ and return DK to $\mathcal{A}$. Set $m := m + 1$.
- **Guess.** The adversary outputs a bit $b' \in \{0, 1\}$. The outcome of the experiment is b' if $\mathcal{A}$ satisfies the following admissibility condition. Otherwise, the outcome is set to 0. The adversary $\mathcal{A}$ is admissible if

- $m \leq Q$,
- for $i \in \mathcal{C}$, each query $\text{QEnc}(i, u_i^0, u_i^1)$ satisfies $u_i^0 = u_i^1$,
- for $i \in [n] \setminus \mathcal{C}$, each query $\text{QEnc}(i, u_i^0, u_i^1)$ satisfies $u_{i,\text{pub}}^0 = u_{i,\text{pub}}^1$,
- for all sequences $(x_1^0, \dots, x_n^0, x_1^1, \dots, x_n^1, f)$ such that
 - * for all $i \in \mathcal{C}$, we have $u_i^0 = u_i^1$,
 - * for all $i \in [n] \setminus \mathcal{C}$, $\text{QEnc}(i, u_i^0, u_i^1)$ is queried, and
 - * $\text{QKeyGen}(f)$ is queried,
 it holds $f(u_1^0, \dots, u_n^0) = f(u_1^1, \dots, u_n^1)$, and
- if $\text{xxx} = \text{stat}$, then $\mathcal{A}$ submits all queries to QEnc and QKeyGen upfront in one shot.

If we do not explicitly specify Q , then the adversary can submit any number of queries to QKeyGen and we say that iFE is secure against *unbounded* collusions.

Function Class. We define the attribute-based inner-product functionality (AB-IP) for *conjunctions* of a predicate family.

Definition 39 (MIFE for AB-IP). Let $d, \ell, \ell' \in \mathbb{N}$ and $\{p_\lambda\}_{\lambda \in \mathbb{N}}$, $\{P_\lambda\}_{\lambda \in \mathbb{N}}$, $\{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ be sequences of prime numbers, predicates, ciphertext attribute spaces and key attribute spaces, respectively. For each $\lambda \in \mathbb{N}$, we let $\mathcal{U}_{\lambda,\text{pub}} = \mathcal{X}_\lambda \cup \{\star\}$, $\mathcal{U}_{\lambda,\text{pri}} = \mathbb{Z}_{p_\lambda}^\ell$ and $\mathcal{V}_\lambda = \mathbb{Z}_{p_\lambda} \cup \{\perp\}$. The AB-IP functionality is a family of functions $\{\mathcal{F}_\lambda = (\mathcal{V}_\lambda \times \mathbb{Z}_{p_\lambda}^\ell)^n\}_{\lambda \in \mathbb{N}}$, where $f = ((y_1, \mathbf{v}_1), \dots, (y_n, \mathbf{v}_n)) \in \mathcal{F}_\lambda$ represents the function $f: \mathcal{U}_\lambda^n \rightarrow \mathcal{V}_\lambda$ defined as

$$f((x_1, \mathbf{u}_1), \dots, (x_n, \mathbf{u}_n)) = \begin{cases} \sum_{i \in [n]} \mathbf{u}_i \mathbf{v}_i^\top & \text{if } P_\lambda^\star(x_i, y_i) = 0 \text{ for all } i \in [n], \\ \perp & \text{otherwise,} \end{cases}$$

where $P_\lambda^\star(x, y) = P_\lambda(x, y)$ if $x \in \mathcal{X}_\lambda$ and $P_\lambda^\star(\star, y) = 0$ for each $y \in \mathcal{Y}_\lambda$.

Definition 40 (Legitimate Keys). We say that a MIFE scheme for AB-IP is secure against legitimate key queries if it is secure against all legitimate adversaries. An adversary $\mathcal{A}$ is called legitimate if it is admissible (as per Definition 38) and satisfies the following additional condition. Let q_k denote the number of queries to QKeyGen and, for $i \in [n]$, let $q_{c,i}$ denote the number of queries of the form $\text{QEnc}(i, \star, \star)$. Let

$$(\mathcal{C}, \{(i, u_i^{j,0}, u_i^{j,1})\}_{i \in [n], j \in [q_{c,i}]}, \{f^k\}_{k \in [q_k]})$$

be the query of the adversary. Parse $u_i^{j,b} = (x_i^j, \mathbf{u}_i^{j,b})$ and $f^k = \{(y_i^k, \mathbf{v}_i^k)\}_{i \in [n]}$. We say that $\mathcal{A}$ is legitimate if for all $k \in [q_k]$ and $i \in [n] \setminus \mathcal{C}$, there exists $j_i^k \in [q_{c,i}]$ such that $P_\lambda^\star(x_i^{j_i^k}, y_i^k) = 0$.

A.2 Construction of AB-MIFE Against Legitimate Keys

Construction 41 (AB-MIFE Against Bounded Collusions). For a fixed collusion bound Q , we choose the parameters κ and τ according to Lemma 20. The construction employs an AB-IPFE scheme iFE = (iSetup, iEnc, iKeyGen, iDec) for a predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$. The details of the MIFE scheme FE for AB-IP go as follows:

Setup(1^λ) takes as input the security parameter 1^λ , samples $\mathbf{t}_1, \dots, \mathbf{t}_{n-1} \xleftarrow{\$} \mathbb{Z}_p^\kappa$, sets $\mathbf{t}_n = -\sum_{i \in [n-1]} \mathbf{t}_i$ and runs $(\text{iMPK}_i, \text{iMSK}_i) \leftarrow \text{iSetup}(1^\lambda, 1^{\kappa+\ell})$ for $i \in [n]$. Then it outputs secret keys $\{\text{SK}_i = (\text{iMPK}_i, \mathbf{t}_i)\}_{i \in [n]}$ and the master secret key $\text{MSK} = \{\text{iMSK}_i\}_{i \in [n]}$.

Enc($\text{SK}_i, x_i, \mathbf{u}_i$) takes as input SK_i , an attribute $x_i \in \{0, 1\}^{\ell'}$ and a message vector $\mathbf{u}_i \in \mathbb{Z}_p^\ell$. The algorithm runs $\text{iCT}_i \leftarrow \text{iEnc}(\text{iMPK}_i, x_i, (\mathbf{u}_i, \mathbf{t}_i))$ and outputs the ciphertext $\text{CT}_i = \text{iCT}_i$.

$\text{KeyGen}(\text{MSK}, \{(y_i, \mathbf{v}_i)\}_{i \in [n]})$ takes as input MSK and n policy-message pairs $(y_i, \mathbf{v}_i) \in \mathcal{Y} \times \mathbb{Z}_p^\ell$, samples a subset $\Delta \leftarrow \text{SamplerSet}(\kappa, Q, \tau)$ of $[\kappa]$ and defines the vector $\mathbf{e}_\Delta = \sum_{k \in \Delta} \mathbf{e}_k$, where $\mathbf{e}_k$ denotes the k -th unit vector in $\mathbb{Z}_p^\kappa$. Then the algorithm generates $\text{iDK}_i \leftarrow \text{iKeyGen}(\text{iMSK}_i, y_i, (\mathbf{v}_i, \mathbf{e}_\Delta))$ for $i \in [n]$ and outputs $\text{DK} = \{\text{iDK}_i\}_{i \in [n]}$.

$\text{Dec}(\text{DK}, \{\text{CT}_i\}_{i \in [\ell]})$ takes as input a decryption key $\text{DK} = \{\text{iDK}_i\}_{i \in [\ell]}$ and ciphertexts $\{\text{CT}_i = \text{iCT}_i\}_{i \in [n]}$. For each $i \in [n]$, the algorithm runs $d_i \leftarrow \text{iDec}(\text{iDK}_i, \text{iCT}_i)$ and immediately outputs $\perp$ if $d_i = \perp$. Otherwise, it outputs $d = \sum_{i \in [n]} d_i$.

Correctness. Correctness readily follows from the correctness of iFE and the fact that $\sum_{i \in [n]} \mathbf{t}_i = \mathbf{0}$.

Security. We first prove security against legitimate keys.

Proposition 42. *If iFE is stat-secure against Q key queries, then the AB-MIFE scheme FE in Construction 41 is stat-secure against Q legitimate key queries.*

Proof of Proposition 42. The proof considers the following sequence of hybrids.

Game G_0^b for $b \in \{0, 1\}$: This is the same as the experiment $\text{Exp}_{\text{FE}, \mathcal{A}}^{\text{mife-}b}(1^\lambda)$.

Game G_1^b for $b \in \{0, 1\}$: This game is the same as G_0^b except that the challenger picks random subsets $\Delta_k \leftarrow \text{SamplerSet}(\kappa, Q, \tau)$ for $k \in [q_k]$ upfront. Note that the admissibility condition requires $q_k \leq Q$. Upon receiving the k -th key query $\text{QKeyGen}(\{(y_i^k, \mathbf{v}_i^k)\}_{i \in [n]})$, the challenger returns $\text{DK}_k = \{\text{iDK}_i^k\}_{i \in [n]}$ computed as follows:

$$\text{iDK}_i^k \leftarrow \text{iKeyGen}(\text{iMSK}_i, y_i^k, (\mathbf{v}_i^k, \mathbf{e}_{\Delta_k})).$$

We have $G_0^b \equiv G_1^b$ as the view of the adversary does not change.

Game G_2^b for $b \in \{0, 1\}$: This game is the same as G_1^b except that, for every $k \in [q_k]$, the challenger chooses an index $\delta_k \in \Delta'_k := \Delta_k \setminus (\bigcup_{k' \in [q_k] \setminus \{k\}} \Delta_{k'})$ if $\Delta'_k \neq \emptyset$, and aborts otherwise. From Lemma 20, it follows that $\Delta'_k \neq \emptyset$ for all $k \in [q_k]$ with overwhelming probability, which implies that $G_1^b \approx_s G_2^b$.

Following the notation of Definition 40, we denote the j -th encryption query for the i -th slot by $\text{QEnc}(i, (x_i^j, \mathbf{u}_i^{j,0}), (x_i^j, \mathbf{u}_i^{j,1}))$ and the k -th key generation query by $\text{QKeyGen}(\{(y_i^k, \mathbf{v}_i^k)\}_{i \in [n]})$. For $i \in [n]$, $j \in [q_{c,i}]$ and $k \in [q_k]$, we let

$$h_i^{j,k,b} = (\mathbf{u}_i^{j,b} - \mathbf{u}_i^{j,0}) \cdot (\mathbf{v}_i^k)^\top$$

and $\mathcal{Q}_i^k = \{j \in [q_{c,i}] : y_i^k(x_i^j) = 0\}$. We note that the adversary's admissibility condition implies the following properties:

1. For $i \in [n]$ and $k \in [q_k]$, the value $h_i^{j,k,b}$ is constant for all $j \in \mathcal{Q}_i^k$. Therefore, we can omit the index j in this case and simply write $h_i^{k,b} = h_i^{j,k,b}$ for $j \in \mathcal{Q}_i^k$.
2. For all $i \in \mathcal{C}$ and $k \in [q_k]$, we have $h_i^{k,b} = 0$.
3. For all $k \in [q_k]$, we have $\sum_{i \in [n]} h_i^{k,b} = 0$.

Game G_3^b for $b \in \{0, 1\}$: This game is the same as the previous game G_2^b except that when receiving an encryption query $\text{QEnc}(i, (x_i^j, \mathbf{u}_i^{j,0}), (x_i^j, \mathbf{u}_i^{j,1}))$, the challenger computes and returns

$$\text{iCT}_i^j \leftarrow \text{Enc}(\text{MSK}, i, x_i^j, (\mathbf{u}_i^{j,0}, \mathbf{t}_i - \sum_{k \in [q_k]} h_i^{k,b} \cdot \mathbf{e}_{\delta_k}))$$

as opposed to $\text{iCT}_i^j \leftarrow \text{Enc}(\text{MSK}, i, x_i^j, (\mathbf{u}_i^{j,b}, \mathbf{t}_i))$ in G_2^b . To argue indistinguishability, we consider three cases for $i \in [n]$, $j \in [q_{c,i}]$ and $k \in [q_k]$.

- If $i \in [n] \setminus \mathcal{C}$ and $P^*(x_i^j, y_i^k) \neq 0$, then $\text{iDec}(\text{DK}_i^k, \text{iCT}_i^j) = \perp$ and nothing is leaked about the vector encoded in iCT_i^j by the security of iFE.
- If $i \in [n] \setminus \mathcal{C}$ and $P^*(x_i^j, y_i^k) = 0$, then property 1 implies that the iFE decryption result $\text{iDec}(\text{DK}_i^k, \text{iCT}_i^j)$ is the same in G_2^b and G_3^b , so we can again rely on the security of iFE to conclude that the ciphertext iCT_i^j in the two games cannot be distinguished.
- If $i \in \mathcal{C}$, then we have $h_i^{k,b} = 0$ from property 2, so nothing changes.

In the end, we have $G_2^b \approx_c G_3^b$.

Game G_4^b for $b \in \{0, 1\}$: This game is the same as game G_3^b except that, when receiving an encryption query $\text{QEnc}(i, (x_i^j, \mathbf{u}_i^{j,0}), (x_i^j, \mathbf{u}_i^{j,1}))$, the challenger computes and returns

$$\text{iCT}_i^j \leftarrow \text{Enc}(\text{MSK}, i, x_i^j, (\mathbf{u}_i^{j,0}, \mathbf{t}_i)) .$$

We note that property 3 implies that the distributions $\{\mathbf{t}_i\}_{i \in [\ell]}$ and $\{\mathbf{t}_i - \sum_{k \in [q_k]} h_i^{k,b} \cdot \mathbf{e}_{\delta_k}\}_{i \in [\ell]}$ are identical, so $G_3^b \equiv G_4^b$. Moreover, we can observe that the challenger's responses to oracle queries in G_4^b are independent of the bit b . So we have $G_4^0 \equiv G_4^1$. This concludes the proof. □

A.3 Upgrade to Any Keys

A recent work [ATY23] presents a generic compiler to remove the restriction to legitimate keys from the security model of MIFE for AB-IP.¹⁰ The compiler uses a *public-key* ciphertext-policy ABE (CP-ABE) for the policy class supported by the AB-IP scheme (bounded-depth circuits in our case). To our knowledge, there is currently no CP-ABE for this policy class in the literature whose security can be based on the hardness of LWE. Fortunately, the work of [AY20] proposes a CP-ABE for bounded-depth circuits in the *secret-key* setting. As MIFE is a secret-key primitive anyway, this fact does not matter for us, and we observe that the compiler of [ATY23] can also be instantiated from secret-key ABE.

When instantiating (1) the AB-IPFE scheme iFE in Construction 41 with the AB-IPFE of [LLW21], and (2) the CP-ABE scheme in the conversion to any keys with the CP-ABE of [AY20], we obtain the following theorem.

Theorem 43. *Assuming the hardness of LWE, there exists a MIFE scheme for the AB-IP functionality for bounded-depth circuits that is secure against any Q key queries, i.e., bounded collusions.*

¹⁰In fact, their compiler covers the so-called *attribute-based attribute-weighted sums* functionality which is a generalization of AB-IP.